

CSE213 MICROCONTROLLER PROGRAMMING

0.0 COURSE INFO

Instructor : Alper Bilge, PhD

Time : Thursdays 09.30-12.30

Labs : Wednesdays 15.30-17.30

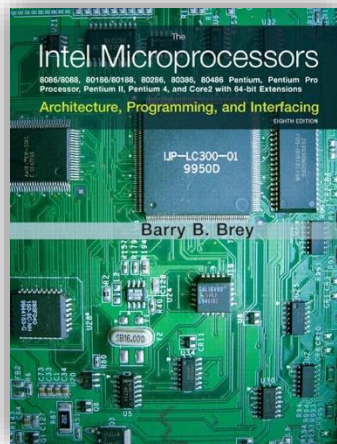
Location : Amfi 4 (courses)
B102 / YLab1 (labs)

Grading (tentative)

- 1 MT – 30%
- Labs – 30% (assessment via quiz)
- Final – 40%

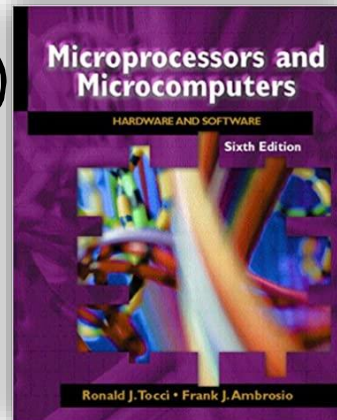
Books

The Intel Microprocessors



Brey, B.B. "The Intel Microprocessors",
Prentice Hall, 2009.

Official course textbook



Tocci, R.J. and Ambrosio, F.J.,
"Microprocessors and Microcomputers",
Prentice Hall, 2003.

Supplementary textbook

Instructor Info

Instructor : Alper Bilge, Phd
Associate Professor of Computer Science
Akdeniz University, Computer Engineering Department

E-mail : abilge@akdeniz.edu.tr

Office Hours : No office hours, please reach on MS Teams.

Course Schedule (tentative)

1. Introduction to the Microprocessor and Computer
2. The Microprocessor and Its Architecture
3. Addressing Modes
4. Data Movement Instructions
5. Arithmetic and Logic Instructions
6. Program Control Instructions
7. Memory Interface
8. I/O Interface
9. Interrupts
10. Floating Point Numbers
11. Microprocessor Performance

Laboratory sessions



- Lab sessions will be handled in laboratory.
- Labs and project will be carried out by each student alone or in groups of 2.
- Target microprocessor platform for lab works is Intel 8086.
- 8086 Emulator will be employed for lab works.
- The emulator can be downloaded in MS Teams.
- You may install the emulator to your own computer as well.

Learning outcomes --> At the end of this course, you will be able to

1. comprehend the structure of a microprocessor-based computer system

- 1.1. summarize the development progress of microprocessors.
- 1.2. express fundamental units within microprocessors.
- 1.3. define the relationship between the microprocessor, memory, and input-output units.
- 1.4. explain bus types and their functions.

2. explain Intel 8086 microprocessors architecture.

- 2.1. recognize Intel 8086 register types.
- 2.2. define the functions of the bus interface unit and execution unit within Intel 8086.
- 2.3. summarize the addressing modes of Intel 8086.
- 2.4. recognize Intel 8086 instruction set, memory interface, input-output interface, and interrupt features.

3. define the differences between high-level and low-level programming languages.

- 3.1. recognize common features of high-level programming languages.
- 3.2. recognize common features of low-level programming languages.
- 3.3. express the tools that are used in the conversion of codes written by high-level programming languages to machine language.
- 3.4. identify usage fields of high level and low-level programming languages.

4. develop a program using low-level programming language.

- 4.1. recognize the tools that are used for conversion of the codes developed by high-level programming languages to machine language
- 4.2. realize operations such as defining variables, forming loops, procedures, and subroutines in low-level programming language.
- 4.3. analyze an instruction set that is specific to a particular microprocessor.

Academic Integrity Statement



1. All work in the classes must be your own work. **NO COPYING OR PLAGIARISM IS ALLOWED.** If such is detected, no credit for the exam or project will be given and appropriate actions for academic dishonesty will be taken.
2. Cheating, fabrication, plagiarism, and helping others to commit these acts are all forms of academic dishonesty, and they are wrong. Academic misconduct could result in disciplinary action that may include, but is not limited to, suspension or dismissal.
3. *It is the ethical responsibility of students to identify the conceptual sources of work submitted. Failure to do so is dishonest and is the basis for a charge of cheating or plagiarism, which is subject to disciplinary action.*
4. Students are sometimes surprised at what we consider plagiarism.
 - It is plagiarism to use in a homework assignment any text from the instructor's notes and slides.
 - It is plagiarism to use in a homework assignment any text found on the Web.
 - It is plagiarism to use in a computer program any code you did not write.
5. A student may use the ideas expressed in the instructor's slides or in material found on the Web, but the ideas must be expressed in the student's own words, to demonstrate understanding of the topic. Students should cite information sources whenever using ideas or information discovered outside of class (e.g., on the Web or in the library). A student is less likely to be accused of plagiarism when information sources are cited.
6. Students are also prohibited from cooperating on homework assignments unless the instructor states explicitly that cooperating is allowed. Students are allowed to discuss homework assignments, but not to collaborate in solving problems, writing answers, or writing computer software. If two students are found to have cooperated on a homework assignment, both students are considered to have cheated. It does not matter which student did the original work and which student copied.
7. Plagiarism, copying, and other forms of cheating can result in immediate failure of the course.

CSE213

MICROCONTROLLER PROGRAMMING

Introduction to the Microprocessor
and Computer

-1-

Introduction

- Overview of Intel microprocessors.
- Discussion of history of computers.
- Function of the microprocessor.
- Terms and jargon (**computerese**).

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Converse by using appropriate computer terminology such as bit, byte, data, real memory system, protected mode memory system, Windows, DOS, I/O.
- Detail history of the computer and list applications performed by computer systems.
- Provide an overview of the various 80X86 and Pentium family members.

Chapter Objectives

(*cont.*)

Upon completion of this chapter, you will be able to:

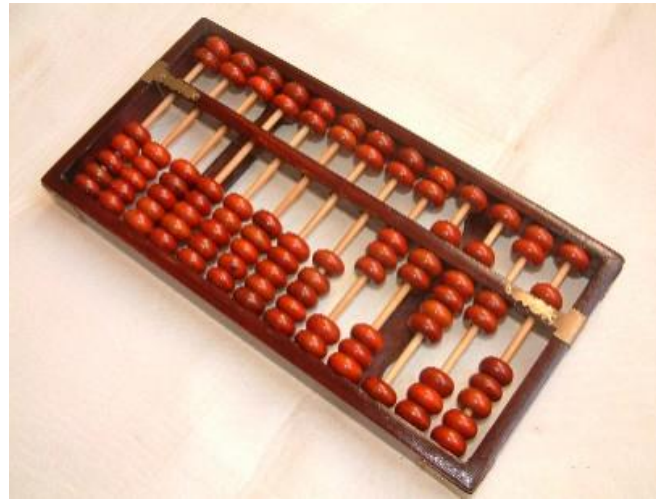
- Draw the block diagram of a computer system and explain the purpose of each block.
- Describe the function of the microprocessor and detail its basic operation.
- Define the contents of the memory system in the personal computer.

1–1 A HISTORICAL BACKGROUND

- Events leading to development of the microprocessor.
- 80X86, Pentium, Pentium Pro, Pentium III, Pentium 4, and Core2 microprocessors.
- While not essential to understand the microprocessor, furnishes:
 - interesting reading
 - historical perspective of fast-paced evolution

The Mechanical Age

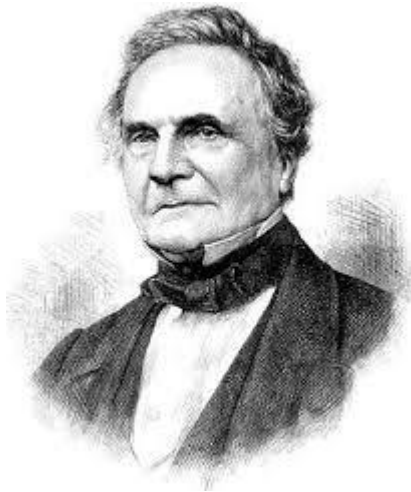
- Idea of computing system is not new.
- Calculating with a machine dates to 500 BC.
- Babylonians invented the **abacus**.
 - first mechanical calculator
 - strings of beads perform calculations



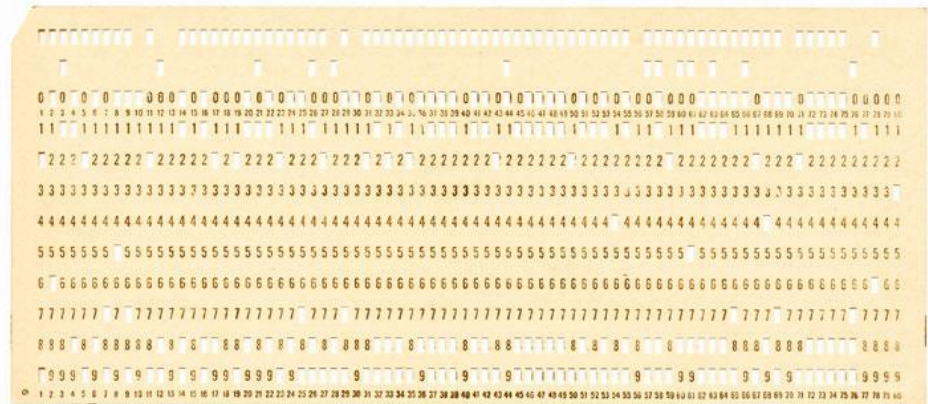
<http://en.wikipedia.org/wiki/Abacus>

- In 1642, mathematician **Blaise Pascal** invented a calculator constructed of gears and wheels.
- When moved one complete revolution, a second gear advances one place.
 - same principle used in automobile odometer
- Basis of all mechanical calculators.
- PASCAL programming language is named in honor of Blaise Pascal.

- In 1823, Charles Babbage (aided by Augusta Ada Byron) began to create his **Analytical Engine** (programmable calculating machine)
 - Steam-powered mechanical computer
 - Generated navigational tables
 - Used punched cards
 - Stored 1000 20-digit decimal numbers



Example of a punch card



The Electrical Age

- 1800s, The advent of electric motor
- 1889, Herman Hollerith developed a machine that counted, sorted, and collated data stored on punched cards
- 1941, Z3 calculating computer by Konrad Zuse
 - used in aircraft and missile design
- 1943, Alan Turing, Colossus
 - broke secret German military codes generated by ENIGMA
 - used vacuum tubes
 - not programmable
 - could not solve other problems (special-purpose computer)

- 1946, ENIAC, University of Pennsylvania
 - 17,000 vacuum tubes, 500 miles of wires
 - weighted over 30 tons
 - 100,000 operations per second
 - The first programmable machine
 - programmed by rewiring its circuits
- 1948, Development of the transistor
- 1958, Invention of the integrated circuit (IC)
- 1960s, Digital integrated circuits
- 1971, The first microprocessor by Intel (4004)

Programming Advancements

- Once programmable machines developed, programs and programming languages began to appear.
- As early practice of rewiring circuits proved too cumbersome, computer languages began to appear in order to control the computer.
- The first, **machine language**, was constructed of ones and zeros using binary codes.
 - stored in the computer memory system as groups of instructions called a program

- More efficient than rewiring a machine to program it.
 - still time-consuming to develop a program due to sheer number of program codes required
- Mathematician John von Neumann, first modern person to develop a system to accept instructions and store them in memory.
- Computers are often called **von Neumann machines** in his honor.
 - Recall that Babbage also had developed the concept long before von Neumann.

- **Assembly language** was then used to simplify entering binary code.
- Assembler allows programmer to use mnemonic codes...
 - such as ADD for addition
- In place of a binary number.
 - such as 0100 0111
- Assembly language an aid to programming.

- Since early days of programming, additional languages have appeared.
 - FLOWMATIC-FORTRAN-ALGOL-**COBOL**-RPG
- Some common modern programming languages are BASIC, PASCAL, C/C++, C#, Java, and ADA.
 - BASIC and PASCAL languages both designed as teaching languages, but escaped the classroom.

TOP 10 Popular Programming Languages in 2020	
1	Python
2	JavaScript
3	Java
4	C#
5	C
6	C++
7	GO
8	R
9	Swift
10	PHP
WWW.NORTHEASTERN.EDU/GRADUATE	

- Scientific community uses primarily C/C++.
- Recent survey of embedded system developers showed C was used by 60%.
 - 30% used assembly language
 - remainder used BASIC and JAVA
- These languages allow programmer almost complete control over the programming environment and computer system.
 - especially C/C++

- Assembly language still plays important role.
 - many video games written almost exclusively in assembly language
- Assembly also interspersed with C/C++ to perform machine control functions efficiently.
 - some newer parallel instructions found on Pentium and Core2 microprocessors only programmable in assembly language

Reminder

- A **bit** is a binary digit with a value of one or zero
- 4-bit-wide memory location is often called a **nibble**
- 8-bit-wide memory location is called a **byte**
- A **word** is a collection of 2 bytes (or 16 bits)
- 1 **KB** = 1024 Bytes = 2^{10} Bytes (1 KB = 1024 bytes)
- 1 **MB** = 1024 KB = 2^{20} Bytes
- 1 **GB** = 2^{30} Bytes, 1 TB = 2^{40} Bytes
- **B** is used for *byte* and **b** is used for *bit*
 - 8 Mbps = 8 Megabits per second (ADSL speed)

The Microprocessor Age

- World's first microprocessor the Intel 4004.
- A 4-bit microprocessor-programmable controller on a chip.
- Addressed 4096, 4-bit-wide memory locations.
- The 4004 instruction set contained 45 instructions.

The Microprocessor Age

- 1971, Intel 4004 and 4040
 - 4-bit microprocessors
 - addressed 4096 (4-bit) memory locations
 - 45 instructions
 - most calculators and low-end applications such as microwave ovens still use 4-bit microprocessors
- 1972, Intel 8008
 - 8-bit microprocessor
 - addressed 16K memory locations
 - 48 instructions
 - 50,000 instructions per second

The Microprocessor Age

- 1973, Intel 8080 (compatible with TTL)
 - addressed 64K memory locations
 - 500,000 instructions per second (2 μ s per instr.)
- 1974, Motorola 6800
- 1974, The first personal computer
 - (MITS Altair 8800) -> BASIC interpreter by Gates&Allen
- 1977, Intel 8085 (last 8-bit μ p by Intel)
 - 769,230 instructions per second (1.3 μ s per instr.)
 - internal clock generator and system controller

The Modern Microprocessors

- 1978, Intel 8086 and 8088
 - 16-bit microprocessors
 - 400 ns per instruction
 - 2,5 million instructions per second
 - addressed 1M memory locations
 - multiply and divide instructions are added
 - over 20,000 instruction variations
 - such microprocessors are called **CISC** (complex instruction set computers)
 - Alternative is **RISC** (reduced instruction set computers)

The Modern Microprocessors

- 1983, Intel 80286
 - addressed 16M memory locations
 - increased clock speed (8 MHz)
 - some instructions took 250 ns. to be executed
- 1986, Intel 80386
 - 32-bit microprocessor
 - addressed 4GB memory
- 1989, Intel 80486
 - same as 80386 but half of its instructions are executed in one clock cycle instead of two
 - 8 KB cache

The Pentium Microprocessor

- 1993, Intel Pentium (P5 or 80586)
 - Introductory versions operated with a clocking frequency 60 & 66 MHz, and a speed of 110 MIPs
 - Over-clocked versions operated on 120, 133, and 233 MHz clocking frequencies
 - 16 KB cache
 - Full-frame video displays at scan rates of 30 Hz or higher (which is comparable to commercial television)

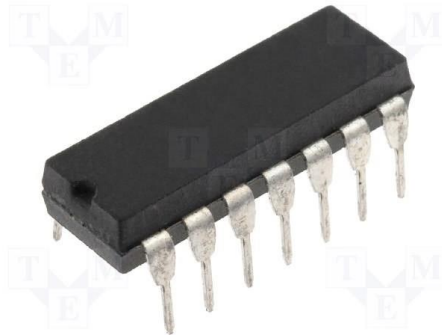
Next Microprocessors

- Pentium Pro
- Pentium II
- Pentium III
- Pentium IV
- Core2
- Quad Core
- 64-bit microprocessors
- ...

Some Definitions

Integrated Circuit

- a.k.a. IC, microcircuit, microchip, silicon chip, or chip
- Integrated Circuit is a miniaturized electronic circuit (consisting mainly of semiconductor devices, as well as passive components) that has been manufactured in the surface of a thin substrate of semiconductor material.



Microprocessor

- A microprocessor is a programmable digital electronic component that incorporates the functions of a central processing unit (CPU) on a single semiconducting integrated circuit (IC).
- 8-bit, 16-bit, 32-bit, and 64-bit microprocessor: refers to number of bits manipulated in one operation. It requires external memory to execute programs. It cannot directly interface to I/O devices, peripheral chips are needed.

Microcomputer and Microcontroller

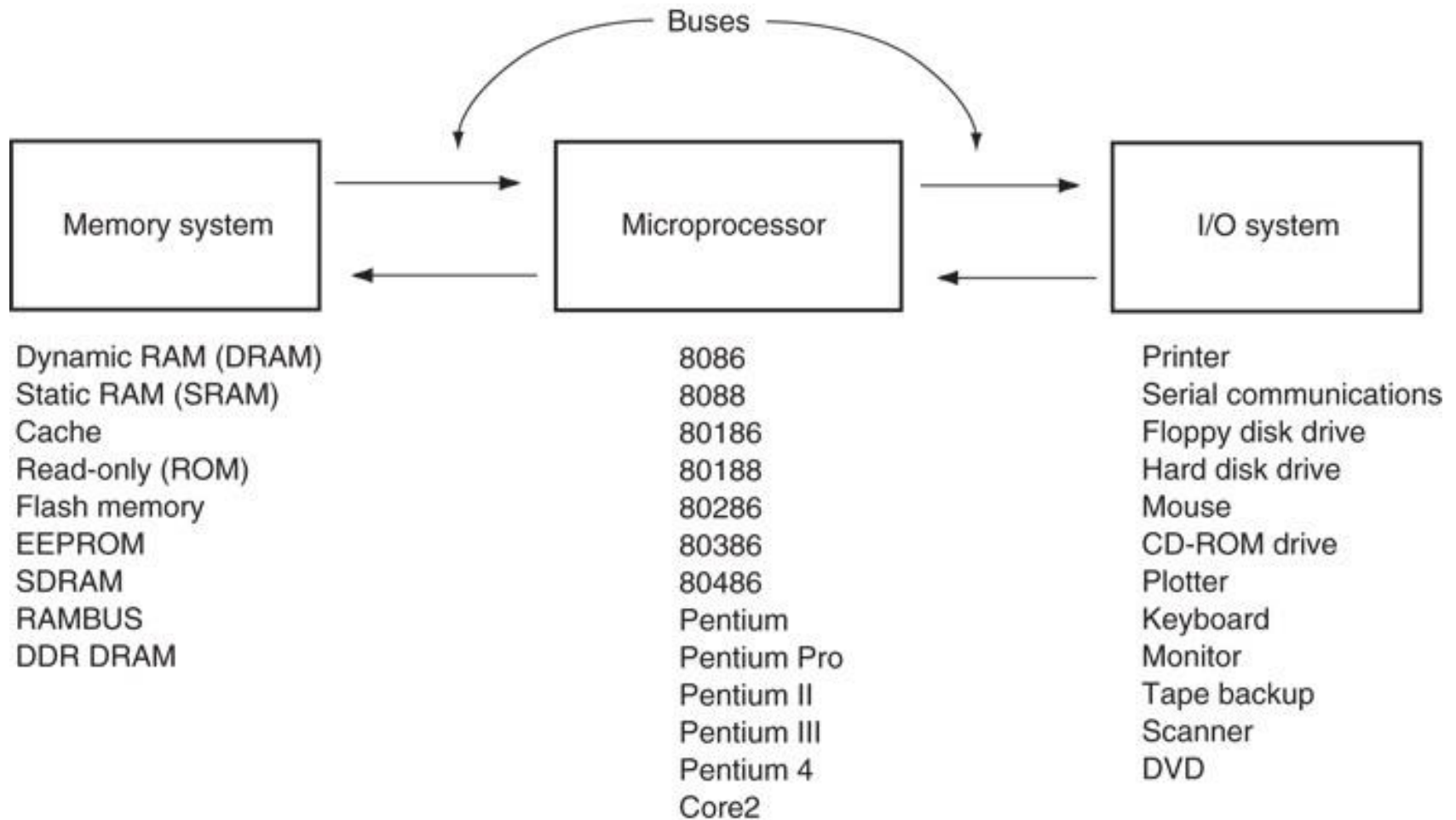
- A **microcomputer** is a computer that uses a microprocessor as its **CPU**.
- A **microcontroller** (or MCU) is a computer-on-a-chip. It is a type of microprocessor emphasizing self-sufficiency and cost-effectiveness, in contrast to a general-purpose microprocessor (the kind used in a PC).
- The only difference between a **microcontroller** and a **microprocessor** is that a microprocessor has three parts - **ALU**, **Control Unit** and **registers**, while the microcontroller has additional elements like **ROM**, **RAM**, peripherals (timer, I/O ports, etc).

THE MICROPROCESSOR-BASED PERSONAL COMPUTER SYSTEM

- Computers have undergone many changes recently.
- Machines that once filled large areas reduced to small desktop computer systems because of the microprocessor.

- Figure 1–6 shows block diagram of the personal computer.
- Applies to any computer system, from early mainframe computers to the latest systems.
- Diagram composed of three blocks interconnected by buses.
 - a **bus** is the set of common connections that carry the same type of information

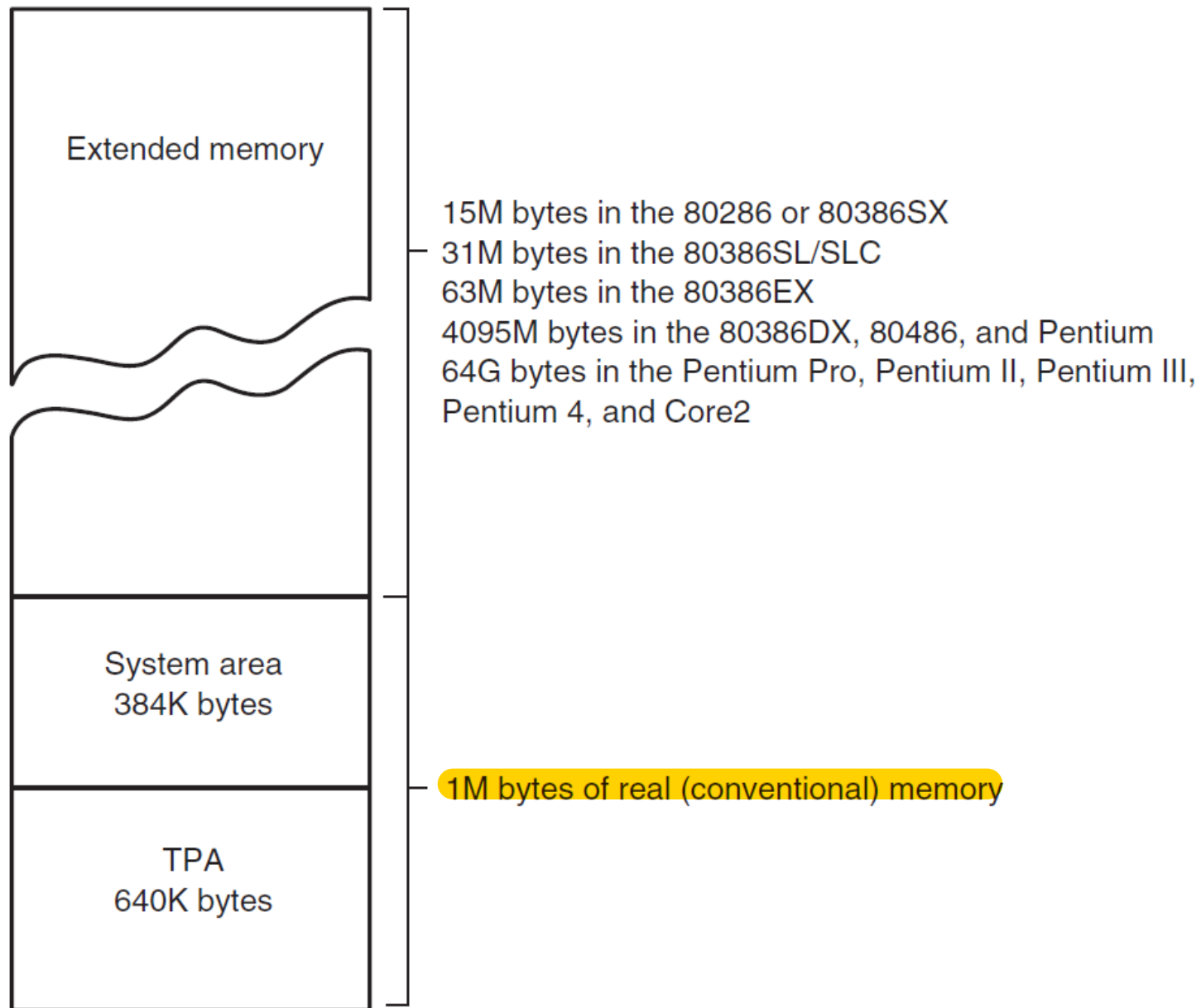
Figure 1–6 The block diagram of a microprocessor-based computer system.



The Memory and I/O System

- Memory structure of all Intel-based personal computers similar.
- Figure 1–7 illustrates memory map of a personal computer system.
- This map applies to any IBM personal computer.
 - also any IBM-compatible clones in existence

Figure 1–7 The memory map of a personal computer.

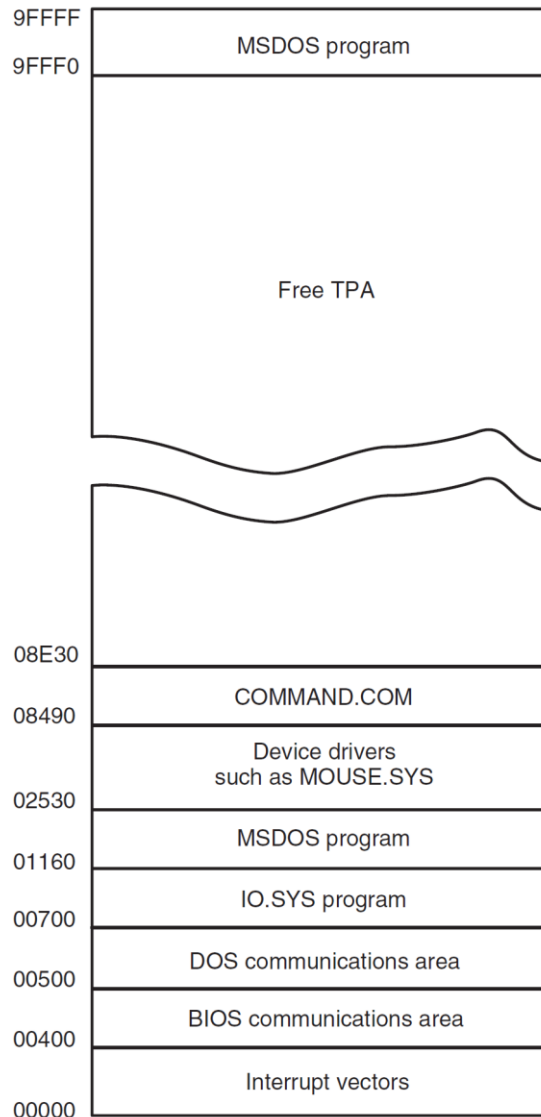


- Main memory system divided into three parts:
 - TPA (transient program area)
 - system area
 - XMS (extended memory system)
- Type of microprocessor present determines whether an extended memory system exists.
- First 1M byte of memory often called the real or conventional memory system.
 - Intel microprocessors designed to function in this area using real mode operation

The TPA

- The transient program area (TPA) holds the DOS (disk operating system) operating system; other programs that control the computer system.
 - the TPA is a DOS concept and not really applicable in Windows
 - also stores any currently active or inactive DOS application programs
 - length of the TPA is 640K bytes

Figure 1–8 The memory map of the TPA in a personal computer. (Note that this map will vary between systems.)

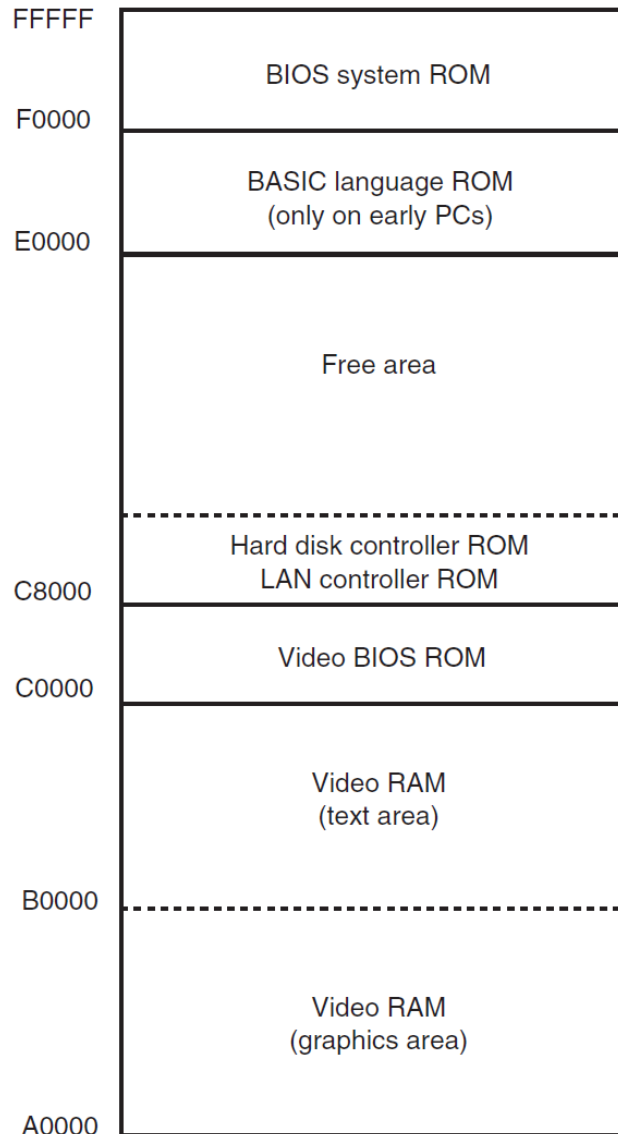


- DOS memory map shows how areas of TPA are used for system programs, data and drivers.
 - also shows a large area of memory available for application programs
 - hexadecimal number to left of each area represents the memory addresses that begin and end each data area

The System Area

- Smaller than the TPA; just as important.
- The **system area** contains programs on read-only (ROM) or flash memory, and areas of read/write (RAM) memory for data storage.
- Figure 1–9 shows the system area of a typical personal computer system.
- As with the map of the TPA, this map also includes the hexadecimal memory addresses of the various areas.

Figure 1–9 The system area of a typical personal computer.

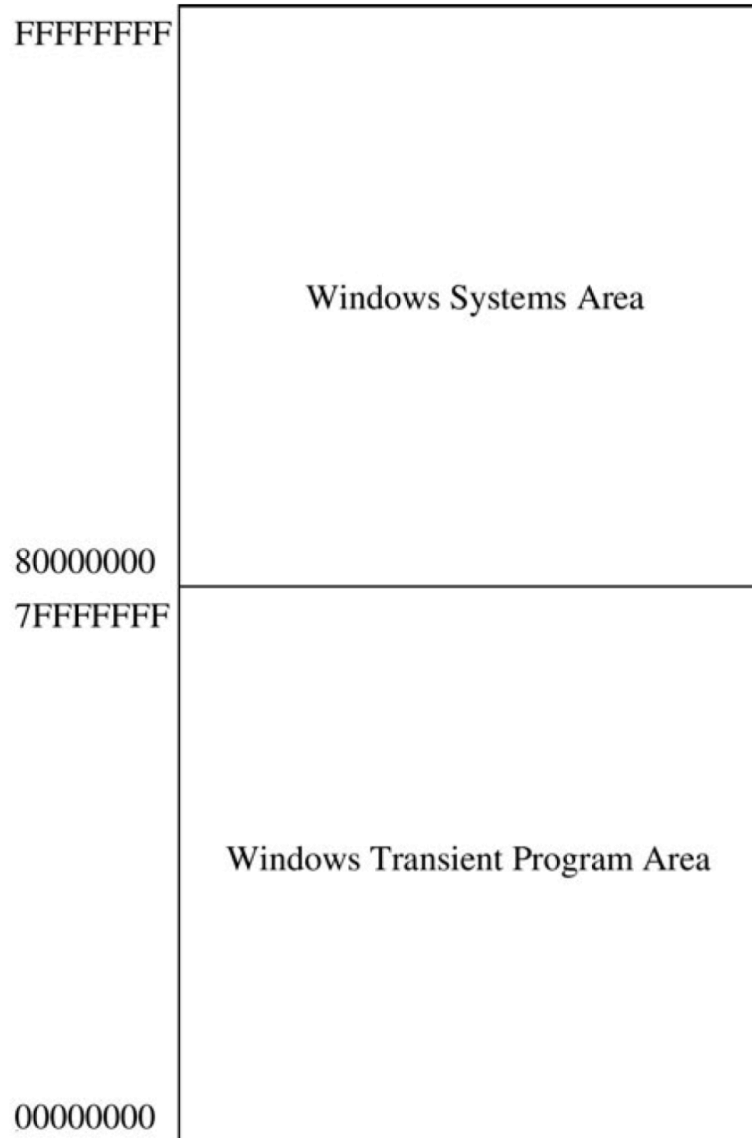


- First area of system space contains video display RAM and video control programs on ROM or flash memory.
 - area starts at location A0000H and extends to C7FFFFH
 - size/amount of memory depends on type of video display adapter attached

Windows Systems

- Modern computers use a different memory map with Windows than DOS memory maps.
- The Windows memory map in Figure 1–10 has two main areas; a TPA and system area.
- The difference between it and the DOS memory map are sizes and locations of these areas.

Figure 1–10 The memory map used by Windows XP.

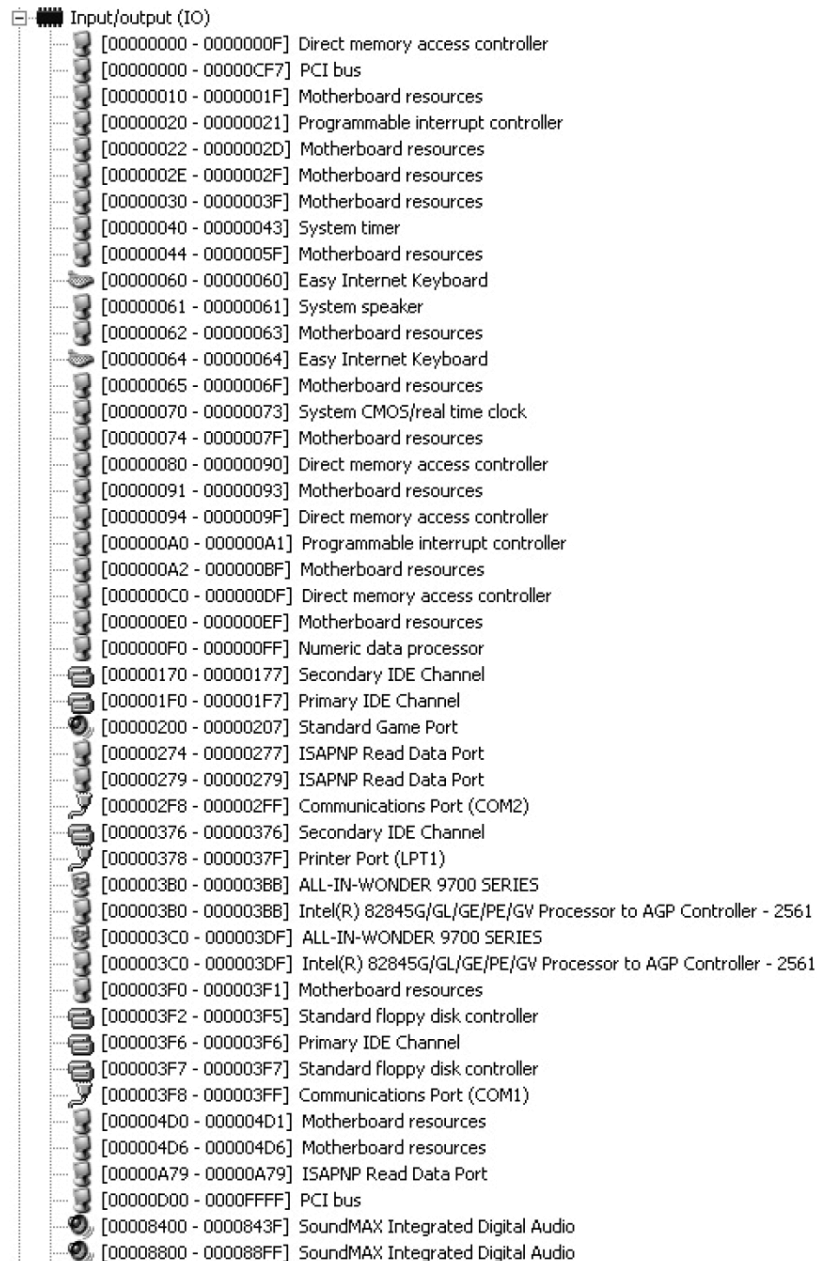


- TPA is first 2G bytes from locations `00000000H` to `7FFFFFFFH`.
- Every Windows program can use up to 2G bytes of memory located at linear addresses `00000000H` through `7FFFFFFFH`.
- System area is last 2G bytes from `80000000H` to `FFFFFFFFFH`.

I/O Space

- I/O devices allow the microprocessor to communicate with the outside world.
- I/O (input/output) space in a computer system extends from I/O port 0000H to port FFFFH.
 - **I/O port address** is similar to a memory address
 - instead of memory, it addresses an I/O device
- Figure 1–11 shows the I/O map found in many personal computer systems.

Figure 1–11 Some I/O locations in a typical personal computer.



- Access to most I/O devices should always be made through Windows, DOS, or BIOS function calls.
- The map shown is provided as a guide to illustrate the I/O space in the system.

- The area below I/O location 0400H is considered reserved for system devices
- Area available for expansion extends from I/O port 0400H through FFFFH.
- Generally, 0000H - 00FFH addresses main board components; 0100H - 03FFH handles devices located on plug-in cards or also on the main board.
- The limitation of I/O addresses between 0000 and 03FFH comes from original standards specified by IBM for the PC standard.



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Introduction to the Microprocessor
and Computer

-2-

Introduction

- How data is stored in memory
- Numeric data representations
- Alphanumeric representations

Chapter Objectives

(cont.)

Upon completion of this chapter, you will be able to:

- Convert between binary, decimal, and hexadecimal numbers.
- Differentiate and represent numeric and alphabetic information as integers, floating-point, BCD, and ASCII data.

The Microprocessor

- Called the **CPU (central processing unit)**.
- The controlling element in a computer system.
- Controls memory and I/O through connections called **buses**.
 - buses select an I/O or memory device, transfer data between I/O devices or memory and the microprocessor, control I/O and memory systems
- Memory and I/O controlled via instructions stored in memory, executed by the microprocessor.

- Microprocessor performs three main tasks:
 - data transfer between itself and the memory or I/O systems
 - simple arithmetic and logic operations
 - program flow via simple decisions
- Power of the microprocessor is capability to execute billions of millions of instructions per second from a program or software (**group of instructions**) stored in the memory system.
 - stored programs make the microprocessor and computer system very powerful devices

- Another powerful feature is the ability to make simple decisions based upon numerical facts.
 - a microprocessor can decide if a number is zero, positive, and so forth
- These decisions allow the microprocessor to modify the program flow, so programs appear to think through these simple decisions.

Simple arithmetic and logic operations

<i>Operation</i>	<i>Comment</i>
Addition	
Subtraction	
Multiplication	
Division	
AND	Logical multiplication
OR	Logic addition
NOT	Logical inversion
NEG	Arithmetic inversion
Shift	
Rotate	

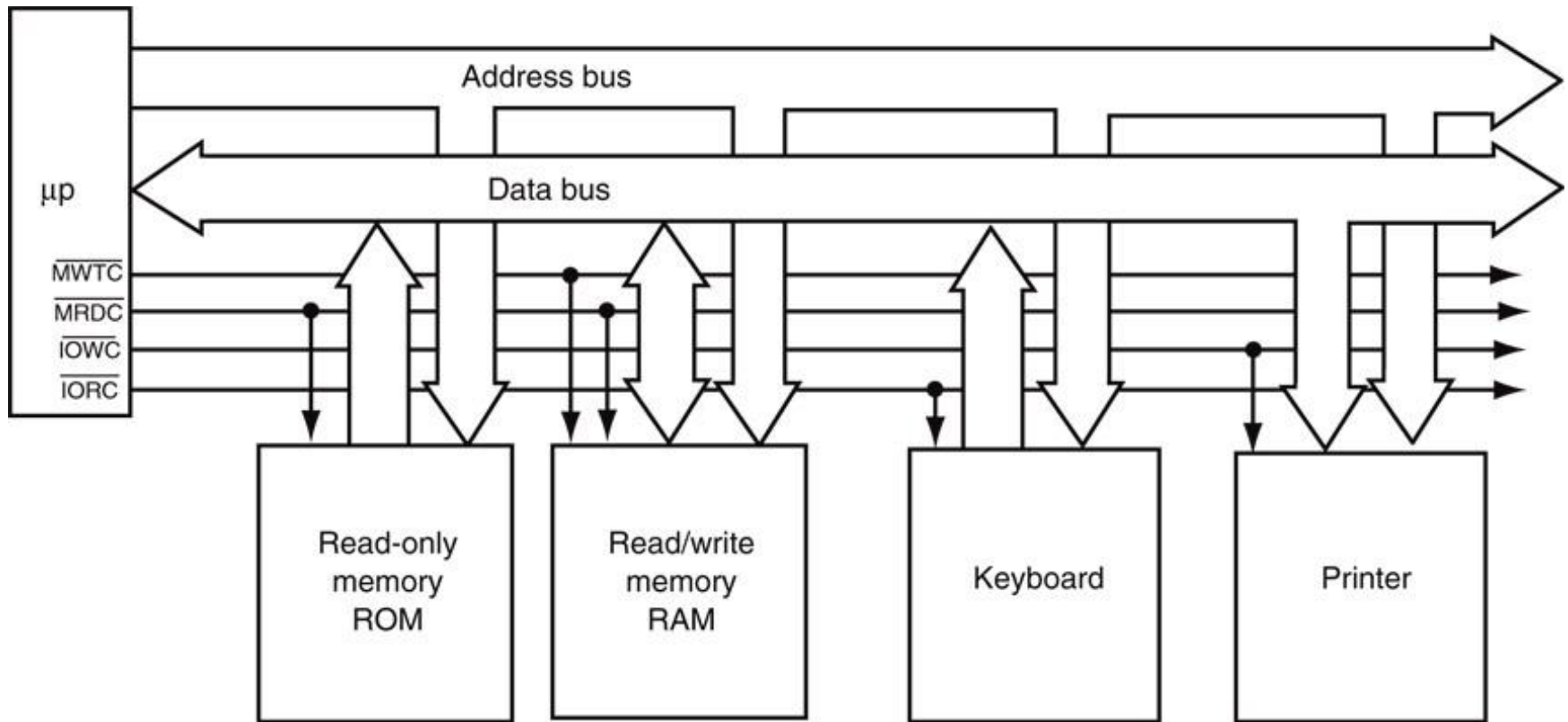
Decisions found in the 8086 through Core2 microprocessors

<i>Decision</i>	<i>Comment</i>
Zero	Test a number for zero or not-zero
Sign	Test a number for positive or negative
Carry	Test for a carry after addition or a borrow after subtraction
Parity	Test a number for an even or an odd number of ones
Overflow	Test for an overflow that indicates an invalid result after a signed addition or a signed subtraction

Buses

- A common group of wires that interconnect components in a computer system.
- Transfer address, data, & control information between microprocessor, memory and I/O.
- Three buses exist for this transfer of information: address, data, and control.
- Figure 1–12 shows how these buses interconnect various system components.

Figure 1–12 The block diagram of a computer system showing the address, data, and control bus structure.



- The address bus requests a memory location from the memory or an I/O location from the I/O devices.
 - if I/O is addressed, the address bus contains a 16-bit I/O address from 0000H through FFFFH.
 - if memory is addressed, the bus contains a memory address, varying in width by type of microprocessor (20-bits for 8086).
- 64-bit extensions to Pentium provide 40 address pins, allowing up to 1T byte of memory to be accessed.

- The data bus transfers information between the microprocessor and its memory and I/O address space.
- Data transfers vary in size, from 8 bits wide to 64 bits wide in various Intel microprocessors.
 - 8088 has an 8-bit data bus that transfers 8 bits of data at a time
 - 8086, 80286, 80386SL, 80386SX, and 80386EX transfer 16 bits of data
 - 80386DX, 80486SX, and 80486DX, 32 bits
 - Pentium through Core2 microprocessors transfer 64 bits of data

- Control bus lines select and cause memory or I/O to perform a read or write operation.
- In most computer systems, there are four control bus connections:
- $\overline{MRDC}$ (memory read control)
- $\overline{MWTC}$ (memory write control)
- $\overline{IORC}$ (I/O read control)
- $\overline{IOWC}$ (I/O write control).
- overbar indicates the control signal is active-low; (active when logic zero appears on control line)

- The microprocessor reads a memory location by sending the memory an address through the address bus.
- Next, it sends a memory read control signal to cause the memory to read data.
- Data read from memory are passed to the microprocessor through the data bus.
- Whenever a memory write, I/O write, or I/O read occurs, the same sequence ensues.

1–3 NUMBER SYSTEMS

- Use of a microprocessor requires working knowledge of numbering systems.
 - binary, decimal, and hexadecimal
- This section provides a background for these numbering systems.
- Conversions are described.
 - decimal and binary
 - decimal and hexadecimal
 - binary and hexadecimal

Digits

- Before converting numbers between bases, digits of a number system must be understood.
- First digit in any numbering system is always zero.
- A decimal (base 10) number is constructed with 10 digits: 0 through 9.
- A base 8 (**octal**) number; 8 digits: 0 through 7.
- A base 2 (**binary**) number; 2 digits: 0 and 1.

- If the base exceeds 10, additional digits use letters of the alphabet, beginning with an A.
 - a base 12 number contains 11 digits: 0 through 9, followed by A for 10 and B for 11
- Note that a base 10 number does not contain a *10* digit.
 - a base 8 number does not contain an 8 digit
- Common systems used with computers are decimal, binary, and hexadecimal (base 16).
 - many years ago octal numbers were popular

Positional Notation

- Once digits are understood, larger numbers are constructed using positional notation.
 - position to the left of the units position is the tens position
 - left of tens is the hundreds position, and so forth
- An example is decimal number 132.
 - this number has 1 hundred, 3 tens, and 2 units
- Exponential powers of positions are critical for understanding numbers in other systems.

- Exponential value of each position:
 - the units position has a weight of 10^0 , or 1
 - tens position a weight of 10^1 , or 10
 - hundreds position has a weight of 10^2 , or 100
- Position to the left of the radix (**number base**) point is always the units position in system.
 - called a decimal point only in the decimal system
 - position to left of the binary point always 2^0 , or 1
 - position left of the octal point is 8^0 , or 1
- Any number raised to its zero power is always one (1), or the units position.

- Position to the left of the units position always the number base raised to the first power.
 - in a decimal system, this is 10^1 , or 10
 - binary system, it is 2^1 , or 2
 - 11 decimal has a different value from 11 binary
- 11 decimal has different value from 11 binary.
 - decimal number composed of 1 ten, plus 1 unit; a value of 11 units
 - binary number 11 is composed of 1 two, plus 1 unit: a value of 3 decimal units
 - 11 octal has a value of 9 decimal units

- In the decimal system, positions right of the decimal point have negative powers.
 - first digit to the right of the decimal point has a value of 10^{-1} , or 0.1.
- In the binary system, the first digit to the right of the binary point has a value of 2^{-1} , or 0.5.
- Principles applying to decimal numbers also generally apply to those in any other system.
- To convert a binary number to decimal, add weights of each digit to form its decimal equivalent.

Conversion to Decimal Examples

Power	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	
Weight	4	2	1	.5	.25	.125	
Number	1	1	0	1	0	1	$(110.101)_2 = (6.625)_{10}$
Numeric Value	4	+	2	+	0	+	.5 + 0 + .125 = 6.625

Power	6^1	6^0	6^{-1}	
Weight	6	1	.167	
Number	2	5	.2	$(25.2)_6 = (17.333)_{10}$
Numeric Value	12	+	5	+
			.333	= 17.333

Power	8^2	8^1	8^0	8^{-1}	
Weight	64	8	1	.125	
Number	1	2	5	.7	$(125.7)_8 = (85.875)_{10}$
Numeric Value	64	+	16	+	5 + .875 = 85.875

Power	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
Weight	16	8	4	2	1	.5	.25	.125	.0625
Number	1	1	0	1	1	0	1	1	1
Numeric Value	$16 + 8 + 0 + 2 + 1 + 0 + .25 + .125 + .0625 = 27.4375$								

$$(11011.0111)_2 = (27.4375)_{10}$$

Power	16^1	16^0	16^{-1}	
Weight	16	1	.0625	
Number	6	A	.C	$(6A.C)_{16} = (106.75)_{10}$
Number Value	96	+	10	+
			.75	= 106.75

Conversion to Decimal

- To convert from any number base to decimal, determine the weights or values of each position of the number.
- Sum the weights to form the decimal equivalent.

Conversion from Decimal

- Conversions from decimal to other number systems more difficult to accomplish.
- To convert the whole number portion of a number to decimal, divide by 1 radix.
- To convert the fractional portion, multiply by the radix.

Whole Number Conversion from Decimal

- To convert a decimal whole number to another number system, divide by the radix and save remainders as significant digits of the result.
- An algorithm for this conversion:
 - divide the decimal number by the radix (number base)
 - save the remainder
(first remainder is the least significant digit)
 - repeat steps 1 and 2 until the quotient is zero

- To convert 10 decimal to binary, divide it by 2.
 - the result is 5, with a remainder of 0
- First remainder is units position of the result.
 - in this example, a 0
- Next, divide the 5 by 2; result is 2, with a remainder of 1.
 - the 1 is the value of the twos (2^1) position
- Continue division until the quotient is a zero.
- The result is written as 1010_2 from the bottom to the top.

- To convert 10 decimal to base 8, divide by 8.
 - a 10 decimal is a 12 octal.
- For decimal to hexadecimal, divide by 16.
 - remainders will range in value from 0 through 15
 - any remainder of 10 through 15 is converted to letters A through F for the hexadecimal number
 - decimal number 109 converts to a 6DH

Whole Number Conversion from Decimal Examples

2) 10	remainder = 0	
2) 5	remainder = 1	
2) 2	remainder = 0	
2) 1	remainder = 1	result = 1010
0		

8) 10	remainder = 2	
8) 1	remainder = 1	result = 12
0		

16) 109	remainder = 13 (D)	
16) 6	remainder = 6	result = 6D
0		

Converting from a Decimal Fraction

- Conversion is accomplished with multiplication by the radix.
- Whole number portion of result is saved as a significant digit of the result.
 - fractional remainder again multiplied by the radix
 - when the fraction remainder is zero, multiplication ends
- Some numbers are never-ending (repetend).
 - a zero is never a remainder

- Algorithm for conversion from a decimal fraction:
 - multiply the decimal fraction by the radix (number base).
 - save the whole number portion of the result (even if zero) as a digit; first result is written immediately to the right of the radix point
 - repeat steps 1 and 2, using the fractional part of step 2 until the fractional part of step 2 is zero
- Same technique converts a decimal fraction into any number base.

Converting from a Decimal Fraction Examples

$$\begin{array}{r} .125 \\ \times \quad 2 \\ \hline 0.25 \end{array} \quad \text{digit is 0}$$
$$\begin{array}{r} .25 \\ \times \quad 2 \\ \hline 0.5 \end{array} \quad \text{digit is 0}$$
$$\begin{array}{r} .5 \\ \times \quad 2 \\ \hline 1.0 \end{array} \quad \text{digit is 1} \quad \text{result} = 0.001_2$$

$$\begin{array}{r} .125 \\ \times \quad 8 \\ \hline 1.0 \end{array} \quad \text{digit is 1} \quad \text{result} = 0.1_8$$

$$\begin{array}{r} .046875 \\ \times \quad 16 \\ \hline 0.75 \end{array} \quad \text{digit is 0}$$
$$\begin{array}{r} .75 \\ \times \quad 16 \\ \hline 12.0 \end{array} \quad \text{digit is 12(C)} \quad \text{result} = 0.0C_{16}$$

Binary-Coded Hexadecimal

- **Binary-coded hexadecimal (BCH)** is a hexadecimal number written each digit is represented by a 4-bit binary number.
- BCH code allows a binary version of a hexadecimal number to be written in a form easily converted between BCH and hexadecimal.
- Hexadecimal represented by converting digits to BCH code with a space between each digit.

BCH Table and Example

<i>Hexadecimal Digit</i>	<i>BCH Code</i>
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

2AC = 0010 1010 1100

1000 0011 1101 . 1110 = 83D.E

Complements

- At times, data are stored in complement form to represent negative numbers.
- Two systems used to represent negative data:
 - radix
 - **radix – 1** complement (earliest - 1's complement)

$$\begin{array}{r} 1111 \ 1111 \\ - 0100 \ 1100 \\ \hline 1011 \ 0011 \end{array}$$

radix-1 complement

$$\begin{array}{r} 1111 \ 1111 \\ - 0100 \ 1000 \\ \hline 1011 \ 0111 \quad (\text{one's complement}) \\ + 1 \\ \hline 1011 \ 1000 \quad (\text{two's complement}) \end{array}$$

radix complement

1–4 COMPUTER DATA FORMATS

- Successful programming requires a precise understanding of data formats.
- Commonly, data appear as ASCII, Unicode, BCD, signed and unsigned integers, and floating-point numbers (real numbers).
- Other forms are available but are not commonly found.

ASCII and Unicode Data

- **ASCII (American Standard Code for Information Interchange)** data represent alphanumeric characters in computer memory.
- Standard ASCII code is a 7-bit code.
 - eighth and most significant bit used to hold parity
- If used with a printer, most significant bits are 0 for alphanumeric printing; 1 for graphics.

ASCII Codes

TABLE 1–8 ASCII code.

		<i>Second</i>														
<i>First</i>	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	XA	XB	XC	XD	XE	XF
0X	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1X	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EMS	SUB	ESC	FS	GS	RS	US
2X	SP	!	“	#	\$	%	&	'	(	)	*	+	,	–	.	/
3X	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4X	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5X	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6X	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7X	p	q	r	s	t	u	v	w	x	y	z	{		}	~	⋮

- In PC, an extended ASCII character set is selected by placing 1 in the leftmost bit.
- Extended ASCII characters store:
 - some foreign letters and punctuation
 - Greek & mathematical characters
 - box-drawing & other special characters
- ASCII control characters perform control functions in a computer system.
 - clear screen, backspace, line feed, etc.
- Enter control codes through the keyboard.
 - hold the Control key while typing a letter

Extended ASCII Codes

TABLE 1–9 Extended ASCII code, as printed by the IBM ProPrinter.

First	Second															
	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	XA	XB	XC	XD	XE	XF
0X		☺	☻	♥	♦	♣	♠	●	◼	○	◐	♂	♀	♪	♫	⚙
1X	▶	◀	↑	!!	¶	§	■	↕	↑	↓	→	←	└	↔	▲	▼
8X	Ç	ü	é	â	ä	à	å	ç	ê	ë	è	ï	î	ì	Ä	Å
9X	É	æ	Æ	ô	ö	ò	û	ù	ÿ	Ö	Ü	¢	£	¥	₣	ƒ
AX	á	í	ó	ú	ñ	Ñ	ª	º	¿	¬	¬	½	¼	¿	«	»
BX	⌘	⌘	⌘													
CX	⌘	⌘	⌘													
DX	⌘	⌘	⌘													
EX	α	β	Γ	π	Σ	σ	μ	γ	Φ	Θ	Ω	δ	∞	φ	€	∩
FX	≡	±	≥	≤	∫	∫	÷	≈	°	·	·	√	n	2	■	

- Many Windows-based applications use the **Unicode** system to store alphanumeric data.
 - stores each character as 16-bit data
- Codes 0000H–00FFH are the same as standard ASCII code.
- Remaining codes, 0100H–FFFFH, store all special characters from many character sets.
- Allows software for Windows to be used in many countries around the world.
- For complete information on Unicode, visit:
<http://www.unicode.org>

BCD (Binary-Coded Decimal) Data

- The range of a BCD digit extends from 0000_2 to 1001_2 , or 0–9 decimal, stored in two forms:
- Stored in packed form:
 - packed BCD data stored as two digits per byte;
 - used for BCD addition and subtraction in the instruction set of the microprocessor
- Stored in unpacked form:
 - unpacked BCD data stored as one digit per byte
 - returned from a keypad or keyboard

Packed and Unpacked BCD

TABLE 1–10 Packed and unpacked BCD data.

<i>Decimal</i>	<i>Packed</i>		<i>Unpacked</i>		
12	0001 0010		0000 0001	0000 0010	
623	0000 0110	0010 0011	0000 0110	0000 0010	0000 0011
910	0000 1001	0001 0000	0000 1001	0000 0001	0000 0000

- Applications requiring BCD data are point-of-sales terminals.
 - also devices that perform a minimal amount of simple arithmetic
- If a system requires complex arithmetic, BCD data are seldom used.
 - there is no simple and efficient method of performing complex BCD arithmetic

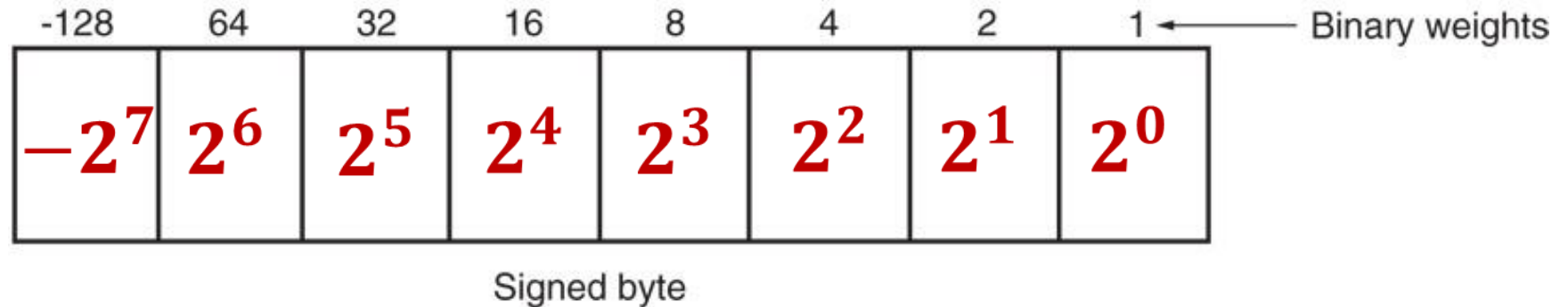
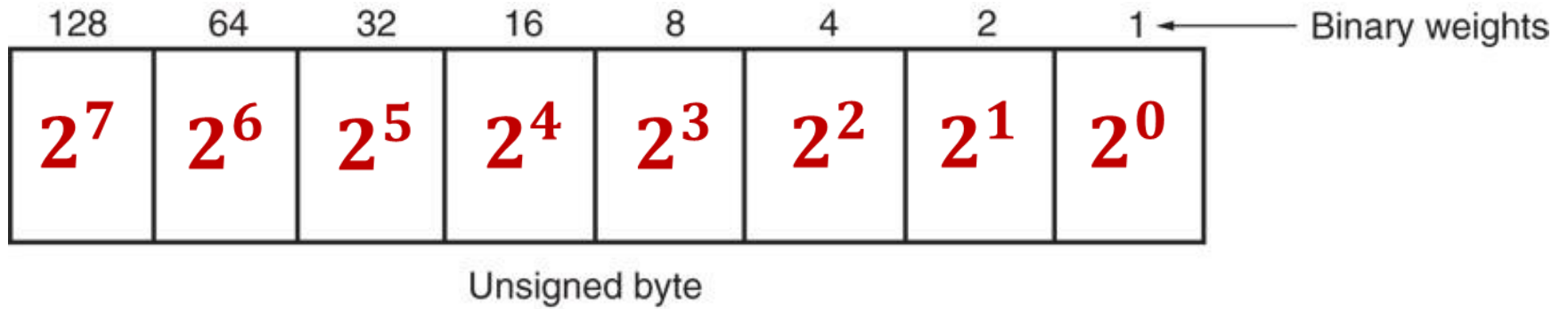
TABLE 1–10 Packed and unpacked BCD data.

<i>Decimal</i>	<i>Packed</i>		<i>Unpacked</i>		
12	0001 0010		0000 0001	0000 0010	
623	0000 0110	0010 0011	0000 0110	0000 0010	0000 0011
910	0000 1001	0001 0000	0000 1001	0000 0001	0000 0000

Byte-Sized Data

- Stored as *unsigned* and *signed* integers.
- Difference in these forms is the weight of the leftmost bit position.
 - value 128 for the unsigned integer
 - *minus* 128 for the signed integer
- In signed integer format, the leftmost bit represents the sign bit of the number.
 - also a weight of *minus* 128

Figure 1–14 The unsigned and signed bytes illustrating the weights of each binary-bit position.



- Unsigned integers range 00H to FFH (0–255)
- Signed integers from –128 to 0 to + 127.
- Negative signed numbers represented in this way are stored in the **two's complement** form.
- Evaluating a signed number by using weights of each bit position is much easier than the act of two's complementing a number to find its value.
 - especially true in the world of calculators designed for programmers

(+/-) Conversion practice

- Whenever a number is two's complemented, its sign changes from negative to positive or positive to negative.
- For example, the number 0000 1000 is a +8. Its negative value (-8) is found by two's complementing the +8.
 1. one's complement the number
 - ✓ invert each bit of a number from zero to one or from one to zero.
 2. add a one to the one's complement.

$$\begin{array}{rcl} + 8 & = & 00001000 \\ & & 11110111 \quad (\text{one's complement}) \\ + & & 1 \\ \hline - 8 & = & 11111000 \quad (\text{two's complement}) \end{array}$$

(+/-) Simpler conversion practice

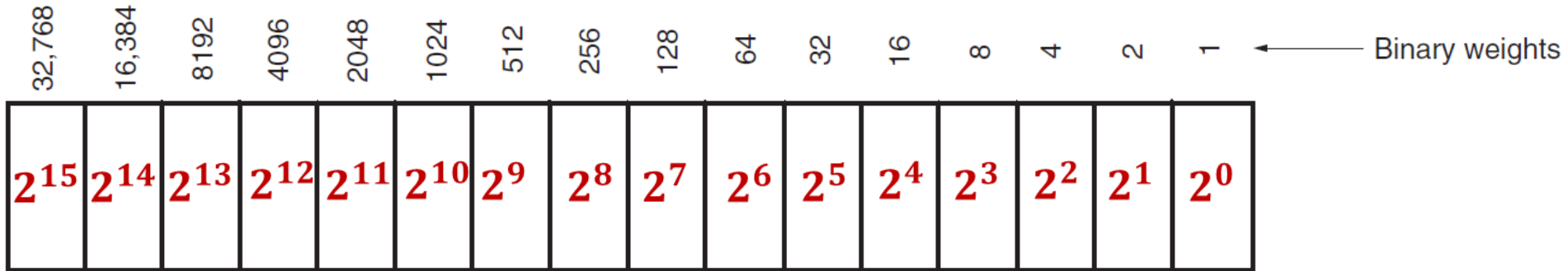
- Another, and probably simpler, technique for two's complementing a number starts with the rightmost digit.
 - Start by writing down the number from right to left.
 - Write the number exactly as it appears until the first one.
 - Write down the first one,
 - and then invert all bits to its left.

```
+8 = 00001000
      1000 (write number to first 1)
      1111 (invert the remaining bits)
-8 = 11111000
```

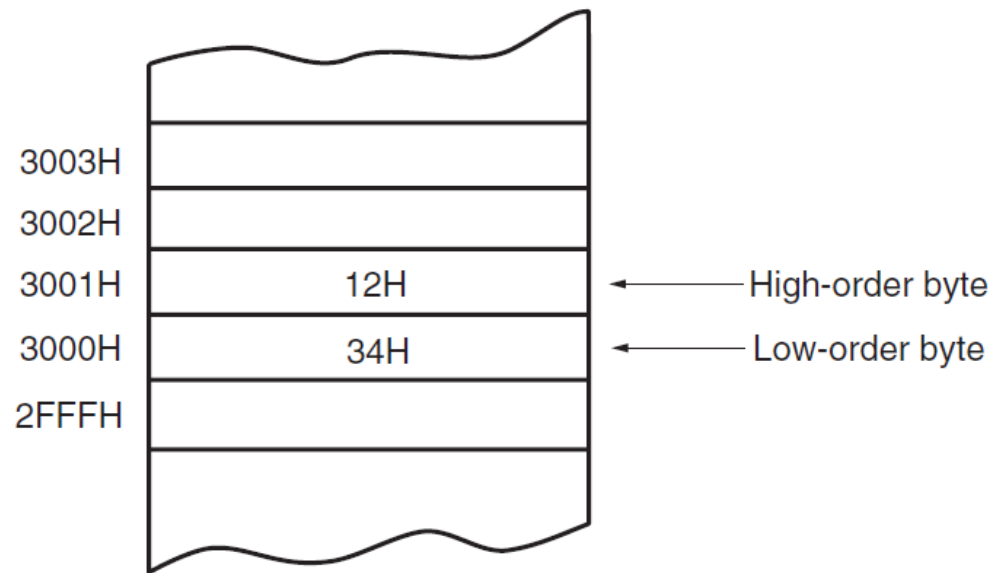
Word-Sized Data

- A word (16-bits) is formed with two bytes of data.
- The least significant byte always stored in the lowest-numbered memory location.
- Most significant byte is stored in the highest.
- This method of storing a number is called the **little-endian** format.

Figure 1–15 The storage format for a 16-bit word in (a) a register and (b) two bytes of memory.



(a) Unsigned word



(b) The contents of memory location 3000H and 3001H are the word 1234H.

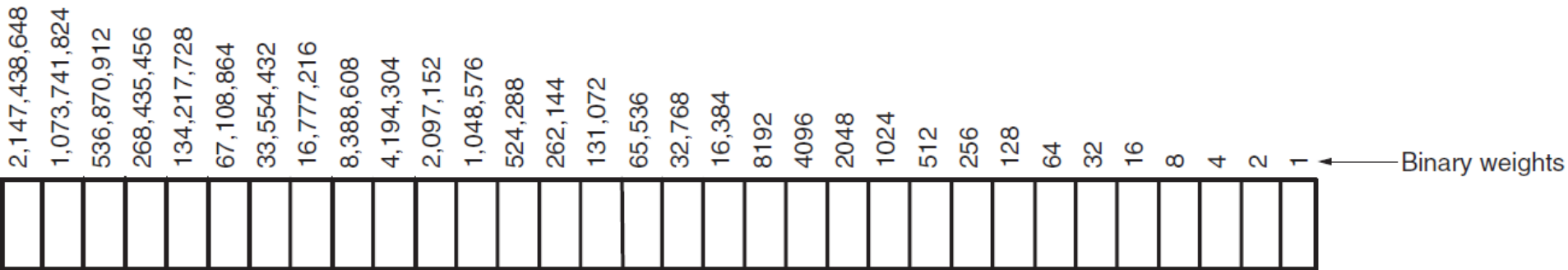
- Alternate method is called the **big-endian** format.
- Numbers are stored with the lowest location containing the most significant data.
- Not used with Intel microprocessors.
- The big-endian format is used with the Motorola family of microprocessors.

Doubleword-Sized Data

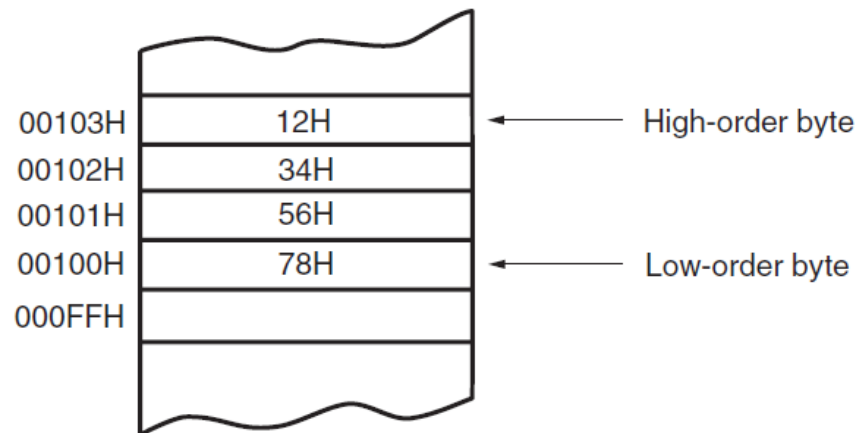
- **Doubleword-sized data** requires four bytes of memory because it is a **32-bit number**.
 - appears as a product after a multiplication
 - also as a dividend before a division
- Define using the assembler directive **define doubleword(s)**, or **DD**.
 - also use the **DWORD** directive in place of **DD**

Doubleword-Sized Data

Figure 1–16 The storage format for a 32-bit word in (a) a register and (b) 4 bytes of memory.



(a) Unsigned doubleword

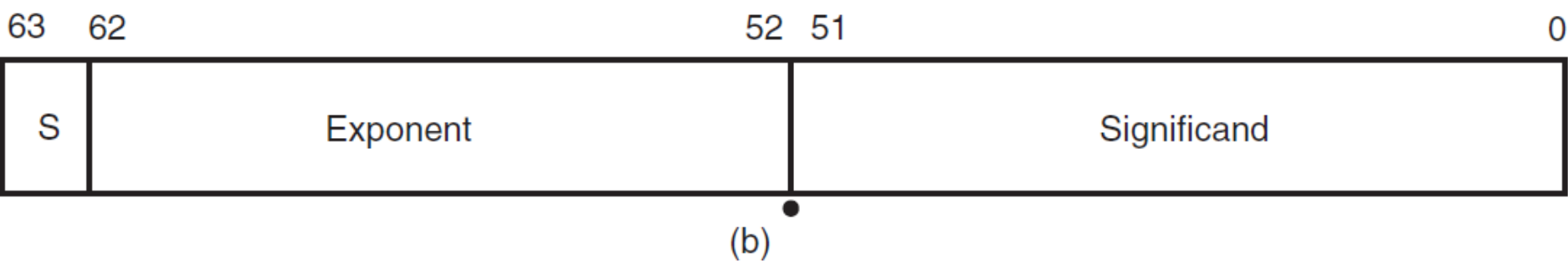
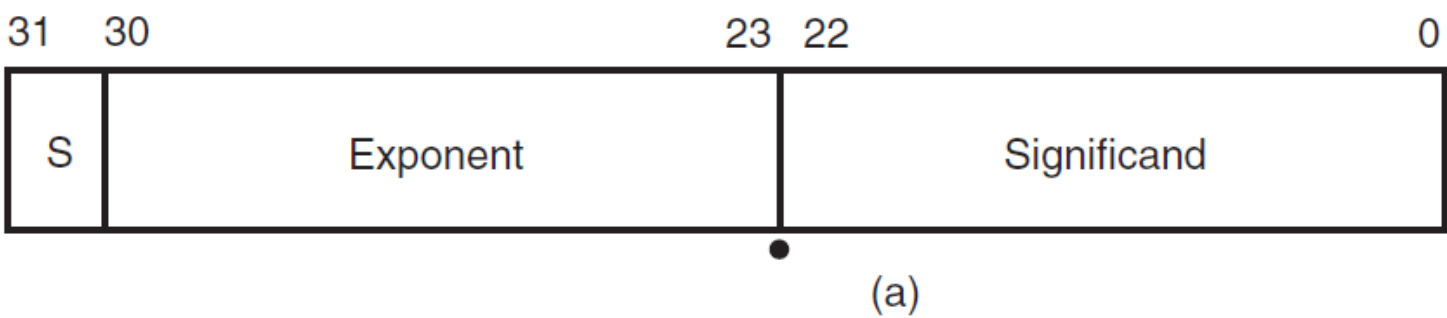


(b) The contents of memory location 00100H–00103H are the doubleword 12345678H.

Real Numbers

- Since many high-level languages use Intel microprocessors, real numbers are often encountered.
- A real, or a **floating-point number** contains two parts:
 - a mantissa, significand, or fraction
 - an exponent.
- A 4-byte number is called **single-precision**.
- The 8-byte form is called **double-precision**.

Figure 1–17 The floating-point numbers in (a) single-precision using a bias of 7FH and (b) double-precision using a bias of 3FFH.



- The assembler can be used to define real numbers in single- & double-precision forms:
 - use the DD directive for single-precision 32-bit numbers
 - use **define quadword(s)**, or DQ to define **64-bit** double-precision real numbers
- Optional directives are REAL4, REAL8, and REAL10.
 - for defining single-, double-, and extended precision real numbers

Decimal	Binary	Normalized	Sign	Biased Exponent	Mantissa
+12	1100	1.1×2^3	0	10000010	10000000 00000000 00000000
-12	1100	1.1×2^3	1	10000010	10000000 00000000 00000000
+100	1100100	1.1001×2^6	0	10000101	10010000 00000000 00000000
-1.75	1.11	1.11×2^0	1	01111111	11000000 00000000 00000000
+0.25	0.01	1.0×2^{-2}	0	01111101	00000000 00000000 00000000
+0.0	0	0	0	00000000	00000000 00000000 00000000

```
;single-precision real numbers
```

```
;
```

```
0000 3F9DF3B6      NUMB1      DD      1.234      ;define 1.234
0004 C1BB3333      NUMB2      DD      -23.4     ;define -23.4
0008 43D20000      NUMB3      REAL4  4.2E2     ;define 420
```

```
;
```

```
;double-precision real numbers
```

```
;
```

```
000C 405ED99999999999A NUMB4      DQ      123.4     ;define 123.4
0014 C1BB333333333333 NUMB5      REAL8  -23.4     ;define -23.4
```

```
;
```

```
;Extended-precision real numbers
```

```
;
```

```
001C 4005F6CCCCCCCCCCCCD NUMB6      REAL10 123.4     ;define 123.4
```

```
//Single-precision real numbers
```

```
//
```

```
float Numb1 = 1.234;
```

```
float Numb2 = -23.4;
```

```
float Numb3 = 4.3e2;
```

```
//
```

```
//Double-precision real numbers
```

```
//
```

```
double Numb4 = 123.4;
```

```
double Numb5 = -23.4;
```



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

The Microprocessor and Its
Architecture

Introduction

- This chapter presents the microprocessor as a programmable device by first looking at its internal programming model and then how its memory space is addressed.
- The architecture of Intel microprocessors is presented, as are the ways that the family members address the memory system.
- Addressing modes for this powerful family of microprocessors are described for the real, protected, and flat modes of operation.

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Explain basic structure of Intel 8086 microprocessor, programming structure and software development flow
- Describe function and purpose of each program-visible register in the 8086-Core2 microprocessors, including 64-bit extensions.
- Detail the flag register and the purpose of each flag bit.

Chapter Objectives

(*cont.*)

Upon completion of this chapter, you will be able to:

- Describe how memory is accessed using real mode memory-addressing techniques.
- Describe how memory is accessed using protected mode memory-addressing techniques.
- Describe how memory is accessed using the 64-bit flat memory model.

PROGRAMMING STRUCTURE

- The lowest-level programming language is the Machine language which is the only one understood by computers.
- While easily understood by computers, machine languages are almost impossible for humans to use because they consist entirely of numbers (e.g., 1s and 0s).

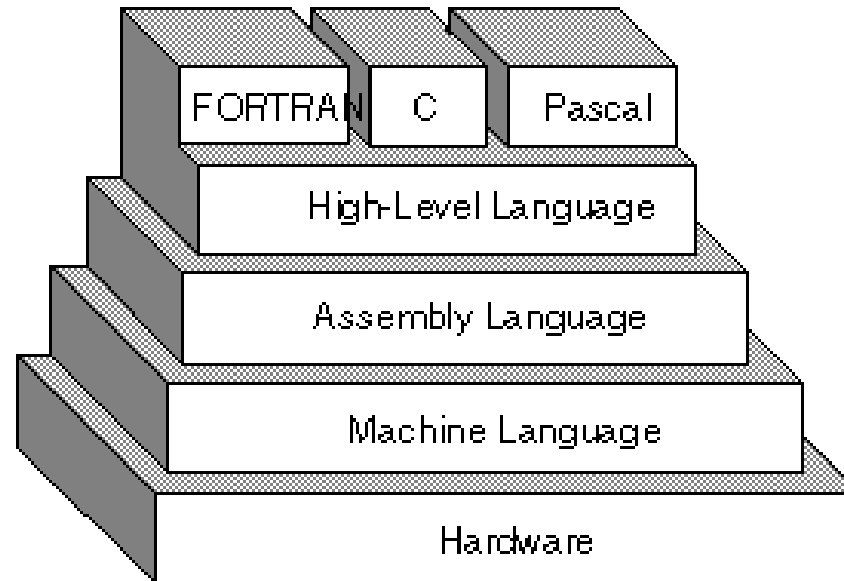
PROGRAMMING STRUCTURE

- Programmers, therefore, use either a high-level programming language or an assembly language.
- An assembly language contains the same instructions as a machine language, but the instructions and variables have names instead of being just numbers.

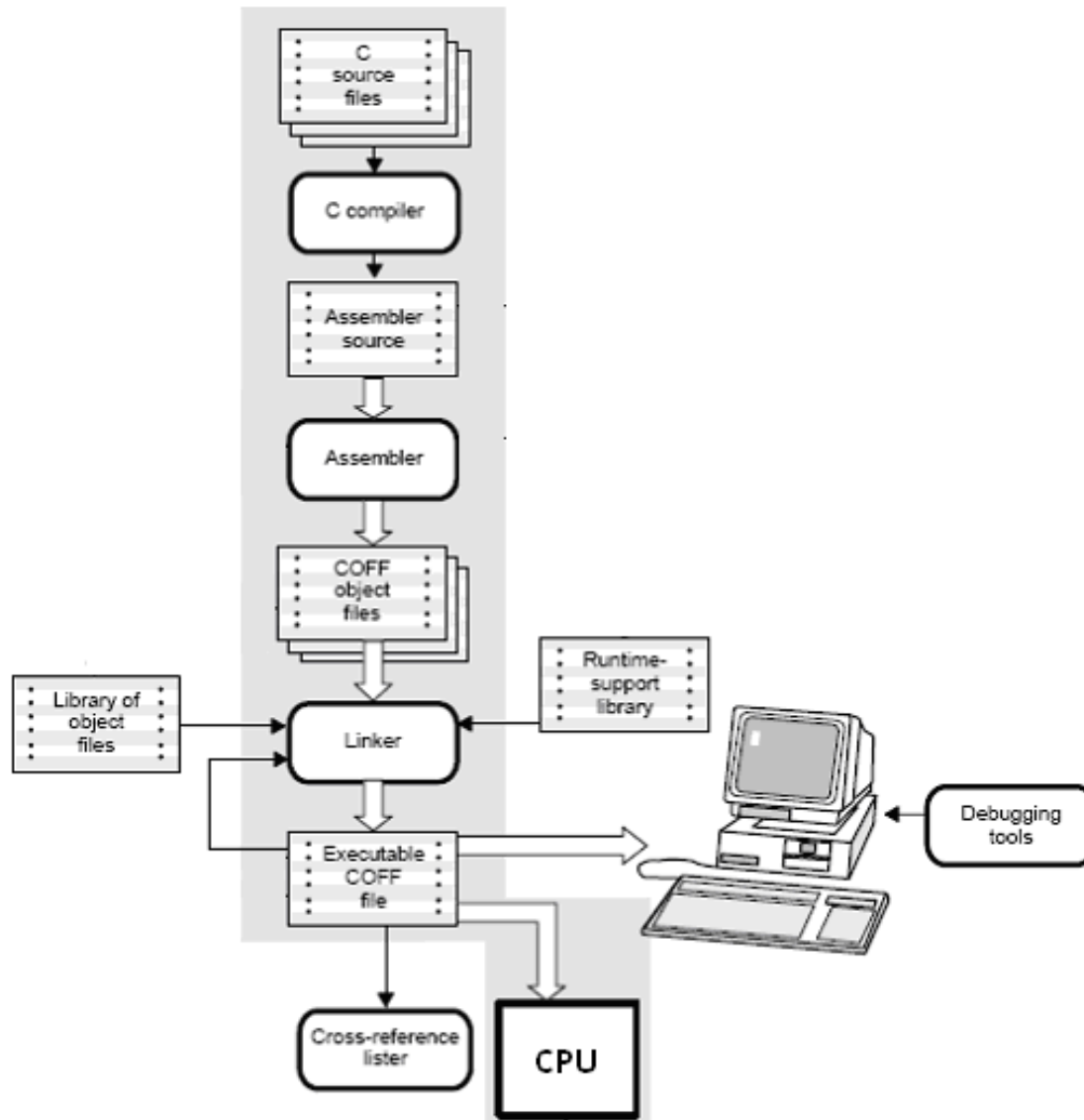
PROGRAMMING STRUCTURE

- Every CPU has its own unique machine language. Programs must be rewritten or recompiled to run on different types of computers.
- High-level languages are, however, platform independent. They can run on different types of computers.

PROGRAMMING STRUCTURE



SOFTWARE DEVELOPMENT FLOW



SOFTWARE DEVELOPMENT FLOW

- The following list describes sample tools that are used for software development.
- The **compiler (e.g., C, C++, C#)** translates source code into assembly language source code.
- The **assembler** translates assembly language source files into machine language (object files). Source files contain numbers corresponding to instructions.

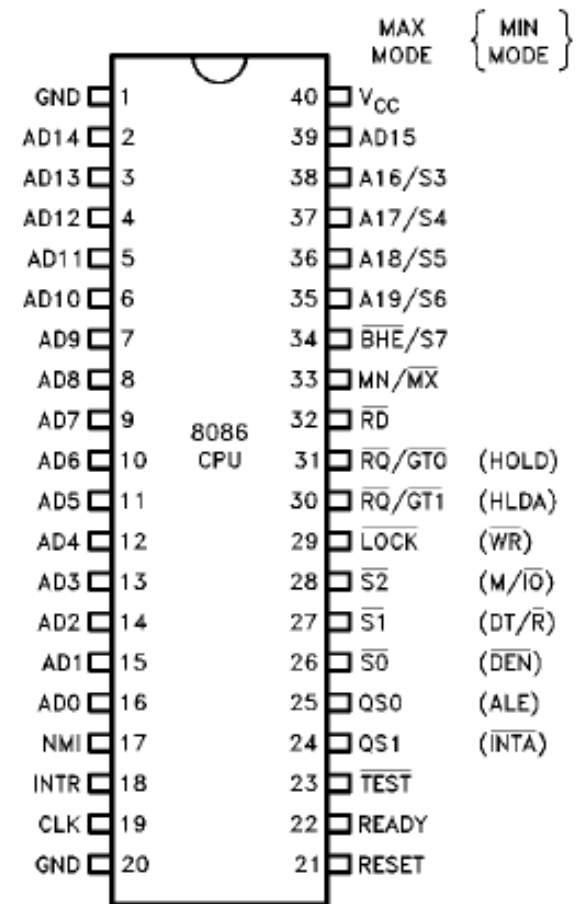
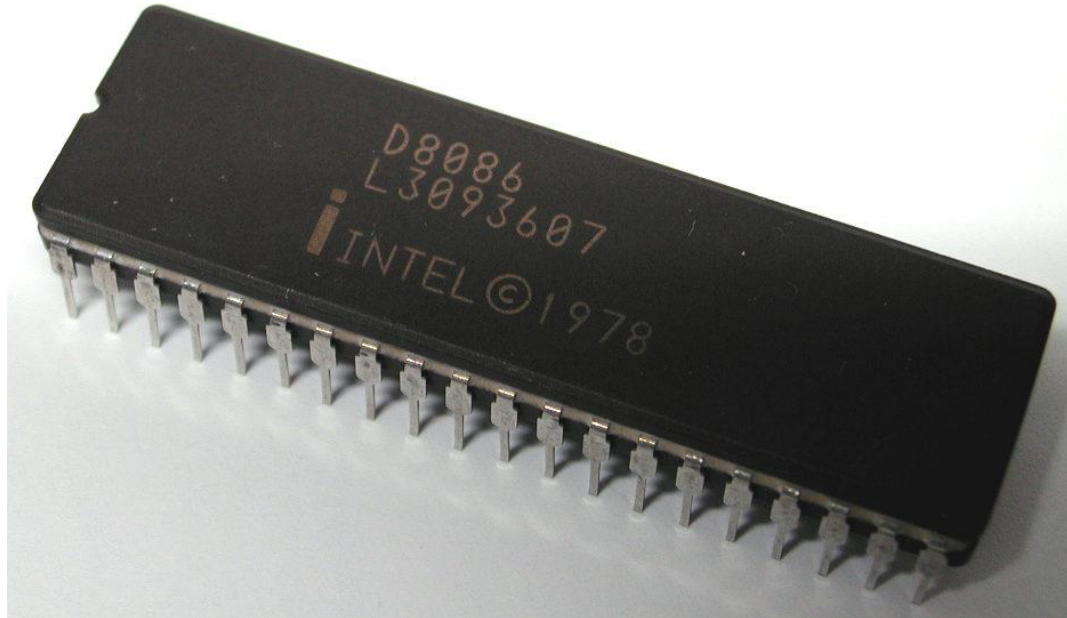
SOFTWARE DEVELOPMENT FLOW

- The **linker** combines object files created by the assembler into a single executable object module. As it creates the executable module, it binds symbols to memory locations and resolves all references to those symbols. It also accepts library members and output modules created by a previous linker run.
- The **cross-reference lister** uses object files to produce a cross-reference listing showing variable and symbols, their definitions, memory locations and their references in the linked source files.

Inside Intel 8086

- The **8086** is a **16-bit** microprocessor chip designed by Intel in 1978, which gave rise to the x86 architecture.
- Intel 8088, released in 1979, was essentially the same chip, but with an external 8-bit data bus (allowing the use of cheaper and fewer supporting logic chips), and is notable as the processor used in the original IBM PC.

Inside Intel 8086



231455-2

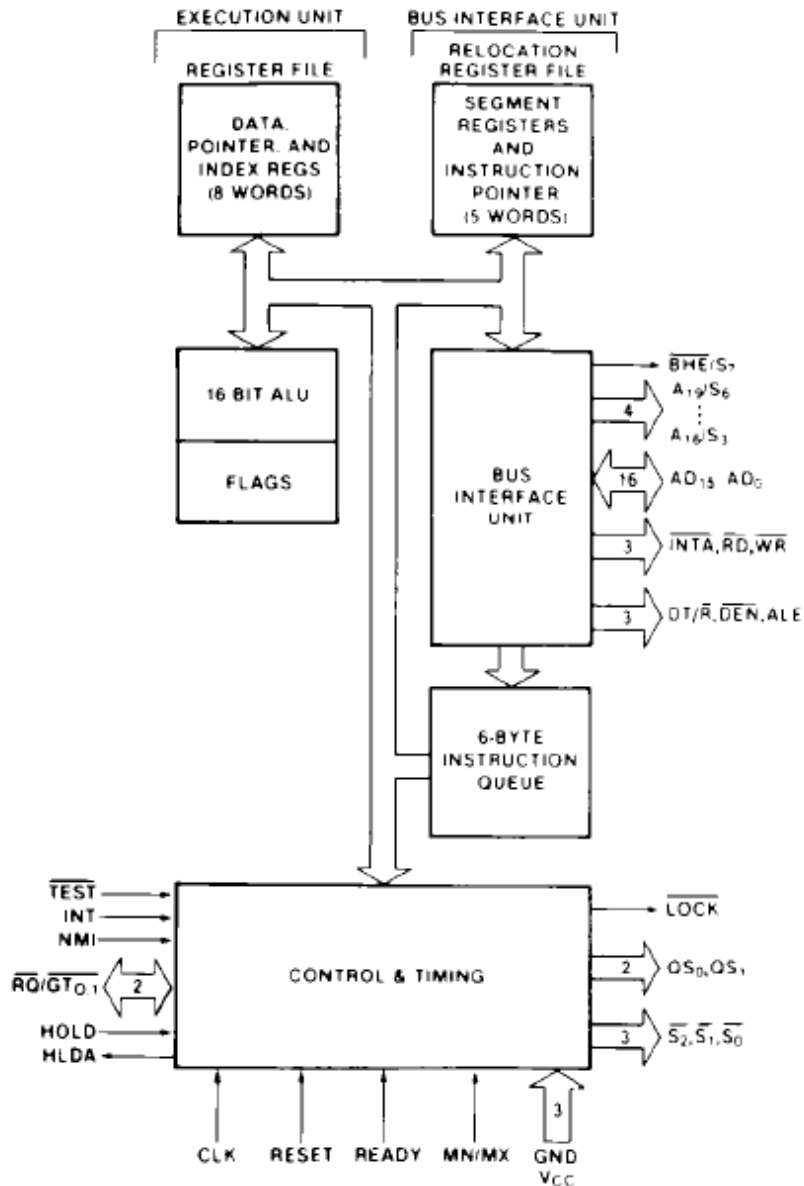
40 Lead

8086 Pin Configuration

Inside Intel 8086

- All internal registers as well as internal and external data buses of the 8086 are 16 bits wide, firmly establishing an actual "16-bit microprocessor".
- Clock rate of the first edition of 8086 was 5 MHz. Later, 8 and 10 MHz versions were also produced.
- A 20-bit external address bus gives a 1 MB (segmented) physical address space.
- 16-bit I/O addresses give 64 KB of separate I/O space.

Inside Intel 8086



- 8086 is internally divided into two separate functional units.

- Bus Interface Unit (BIU)
- Execution Unit (EU).

- This concept of dividing work between two processors and processing it simultaneously (parallel processing) speeds up the execution.

Inside Intel 8086

BUS INTERFACE UNIT

- The BIU sends out address, fetches instruction from memory, reads data from ports and memory and writes data to ports and memory. In other words the BIU handles all transfers of data and addresses on the buses for the execution unit.

Inside Intel 8086

INSTRUCTION QUEUE

- To speed up the program execution BIU fetches as many as 6 instruction bytes ahead of the memory. The pre fetch instruction bytes are held in **Instruction Queue**. The BIU can be fetching instruction bytes while EU is decoding or executing an instruction, which does not require the use of buses.
- When the EU is ready for its next instruction it simply reads the instruction from the queue in the BIU. This pre fetch and Queue scheme greatly speeds up processing. Fetching the next instruction while the current instruction executes is called **Pipelining**.

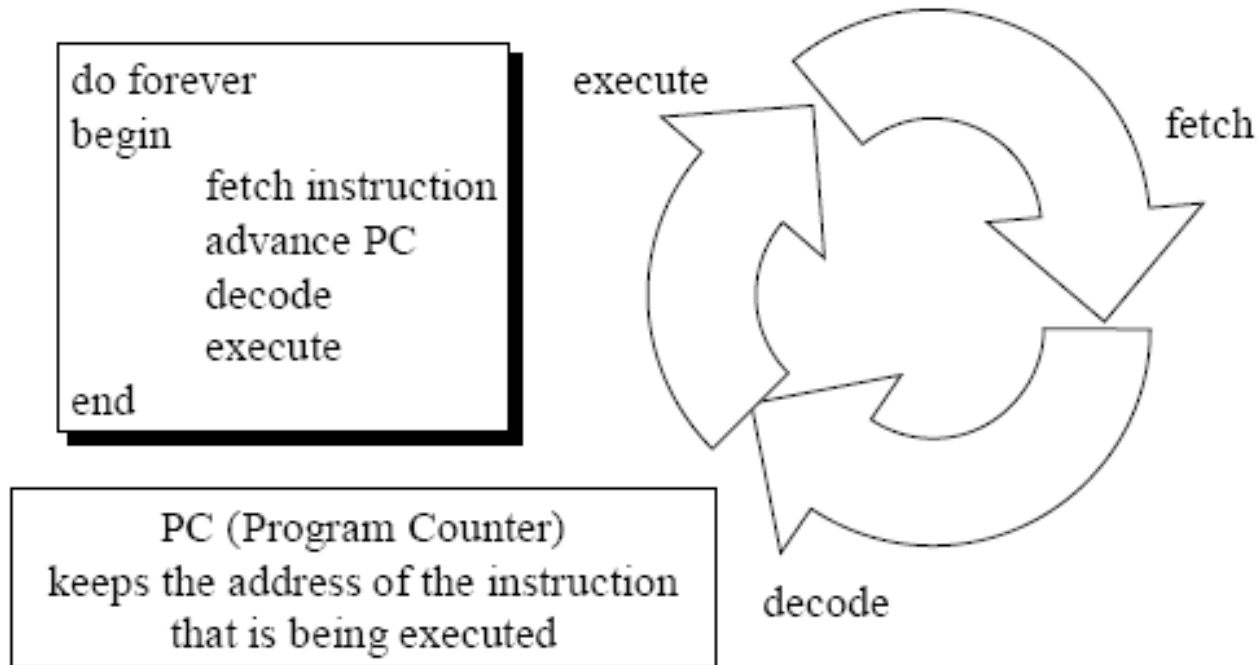
Inside Intel 8086

EXECUTION UNIT

- The EU of the 8086 tells the BIU,
 - where to fetch instructions or data from,
 - decodes instruction and executes instructions.
- The EU contains a 16 bit ALU which can add, subtract, AND, OR, XOR, increment, decrement, complement, or shift binary numbers.

Inside Intel 8086

Basic Instruction Cycle



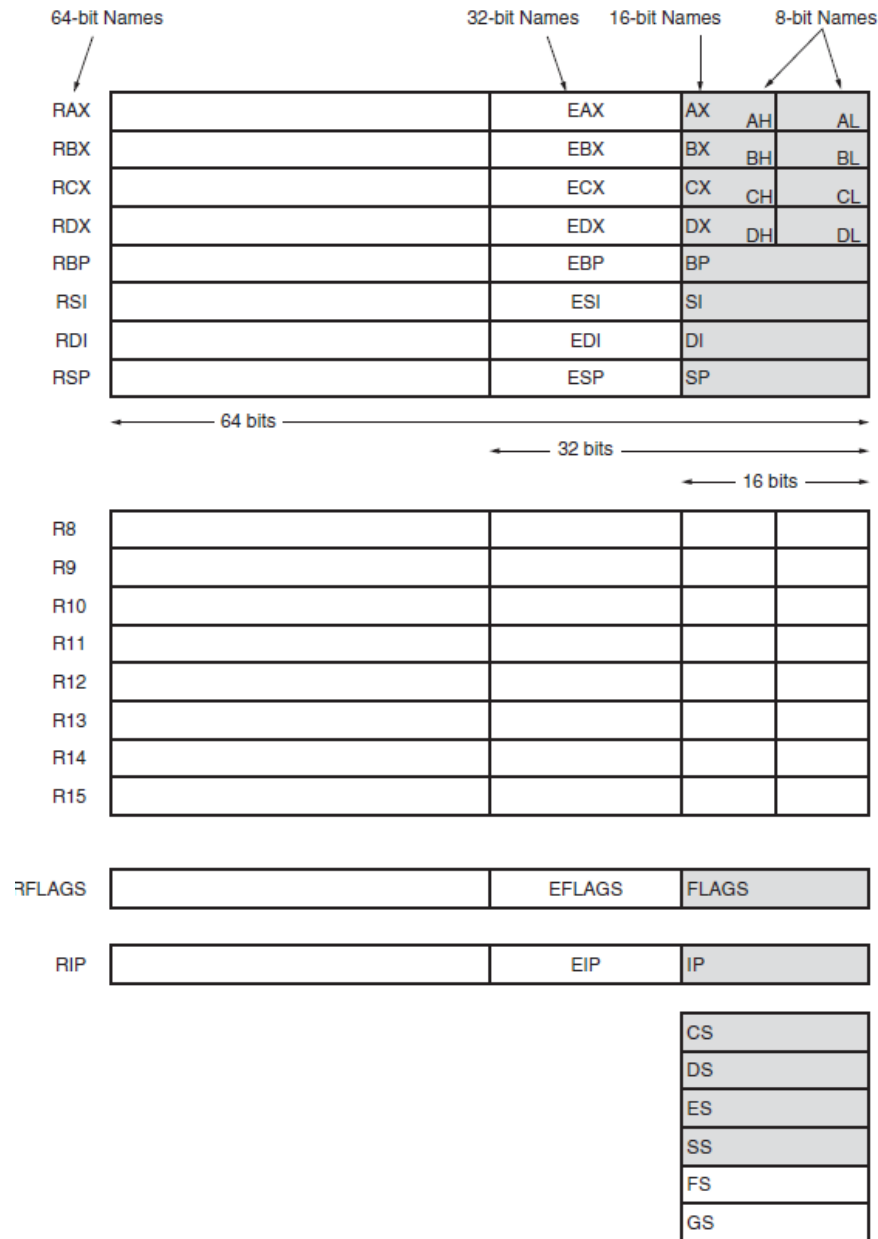
INTERNAL MICROPROCESSOR ARCHITECTURE

- Before a program is written or instruction investigated, internal configuration of the microprocessor must be known.
- In a multiple core microprocessor each core contains the same programming model.
- Each core runs a separate task or thread simultaneously.

The Programming Model

- 8086 through Core2 considered **program visible**.
 - particular registers are used during programming and are specified by the instructions
- Other registers considered to be **program invisible**.
 - not addressable directly during applications programming

Figure 2–1 The programming model of the 8086 through the Core2 microprocessor including the 64-bit extensions.



Multipurpose Registers

- **RAX** - a 64-bit register (RAX), a 32-bit register (**accumulator**) (EAX), (for 8086) a 16-bit register (AX), or as either of two 8-bit registers (AH and AL).
- The accumulator is used for instructions such as multiplication, division, and some of the adjustment instructions.
- Intel plans to expand the address bus to 52 bits to address 4P (peta) bytes of memory.

- **RBX**, addressable as RBX, EBX, **BX**, **BH**, **BL**.
 - BX register (**base index**) sometimes holds offset address of a location in the memory system in all versions of the microprocessor
- **RCX**, as RCX, ECX, **CX**, **CH**, or **CL**.
 - a (**count**) general-purpose register that also holds the count for various instructions
- **RDX**, as RDX, EDX, **DX**, **DH**, or **DL**.
 - a (**data**) general-purpose register
 - holds a part of the result from a multiplication or part of dividend before a division

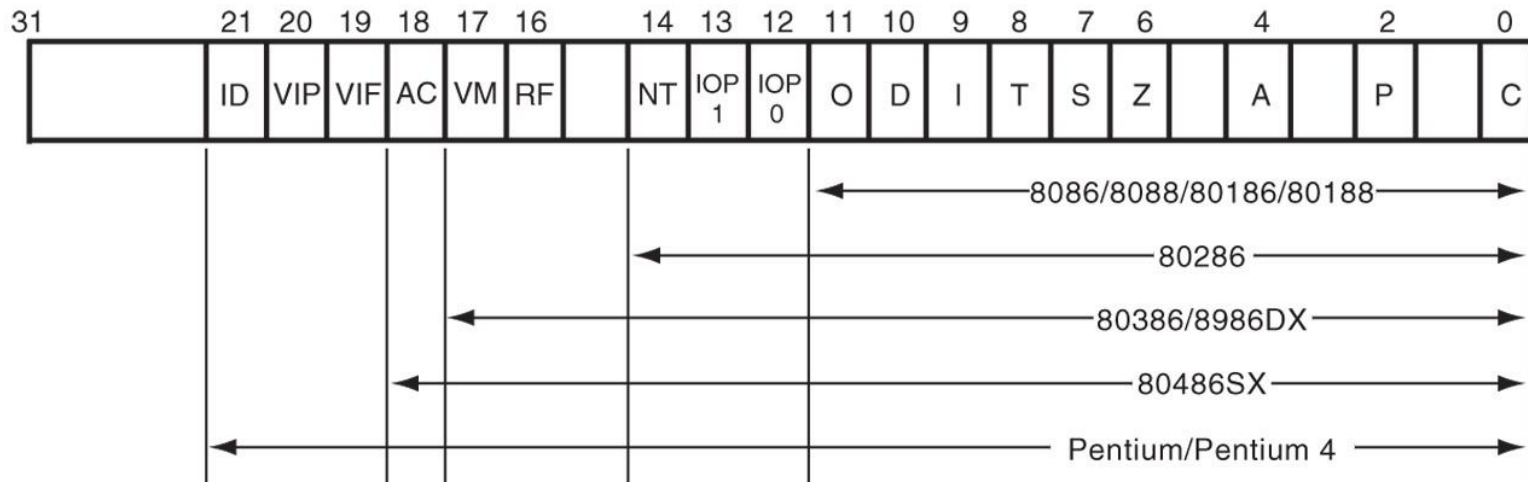
- **RBP**, as RBP, EBP, or **BP**.
 - points to a memory (**base pointer**) location for memory data transfers
- **RDI** addressable as RDI, EDI, or **DI**.
 - often addresses (**destination index**) string destination data for the string instructions
- **RSI** used as RSI, ESI, or **SI**.
 - the (**source index**) register addresses source string data for the string instructions
 - like RDI, RSI also functions as a general-purpose register

Special-Purpose Registers

- Include RIP, RSP, and RFLAGS
 - segment registers include **CS**, **DS**, **ES**, **SS**, FS, and GS
- **RIP** addresses the next instruction in a section of memory.
 - defined as **(instruction pointer)** a code segment
- **RSP** addresses an area of memory called the stack.
 - the **(stack pointer)** stores data through this pointer

- **RFLAGS** indicate the condition of the microprocessor and control its operation.
- Figure 2–2 shows the flag registers of all versions of the microprocessor.
- Flags are upward-compatible from the 8086/8088 through Core2 .
- The rightmost five and the overflow flag are changed by most arithmetic and logic operations.
 - although data transfers do not affect them

Figure 2–2 The EFLAGS and FLAG register counts for the entire 8086 and Pentium microprocessor family.



- Flags **never** change for any data transfer or program control operation.
- Some of the flags are also used to control features found in the microprocessor.

List of Each Flag bit with a brief description of function.

- **C (carry)** holds the carry after addition or borrow after subtraction.
 - also indicates error conditions
- **P (parity)** is the count of ones in a number expressed as even or odd. Logic 0 for odd parity; logic 1 for even parity.
 - if a number contains three binary one bits, it has odd parity (e.g., 1110 1010 $\rightarrow$ P=0)
 - if a number contains no one bits, it has even parity (e.g., 0000 0101 $\rightarrow$ P=1)

- **A (auxiliary carry)** holds the carry (half-carry) after addition or the borrow after subtraction between bit positions 3 and 4 of the result.
- **Z (zero)** shows that the result of an arithmetic or logic operation is zero.
- **S (sign)** flag holds the arithmetic sign of the result after an arithmetic or logic instruction executes.
- **T (trap)** The trap flag enables trapping through an on-chip debugging feature.

- **I (interrupt)** controls operation of the INTR (interrupt request) input pin.
- **D (direction)** selects increment or decrement mode for the DI and/or SI registers.
- **O (overflow)** occurs when signed numbers are added or subtracted.
 - an overflow indicates the result has exceeded the capacity of the machine

Segment Registers

- Generate memory addresses when combined with other registers in the microprocessor.
- Four or six segment registers in various versions of the microprocessor (four in 8086)
- A segment register functions differently in real mode than in protected mode.
- Following is a list of each segment register, along with its function in the system.

- **CS (code)** segment holds code (programs and procedures) used by the microprocessor.
- **DS (data)** contains most data used by a program.
 - Data are accessed by an offset address or contents of other registers that hold the offset address
- **ES (extra)** an additional data segment used by some instructions to hold destination data.

- **SS (stack)** defines the area of memory used for the stack.
 - stack entry point is determined by the stack segment and stack pointer registers
 - the BP register also addresses data within the stack segment

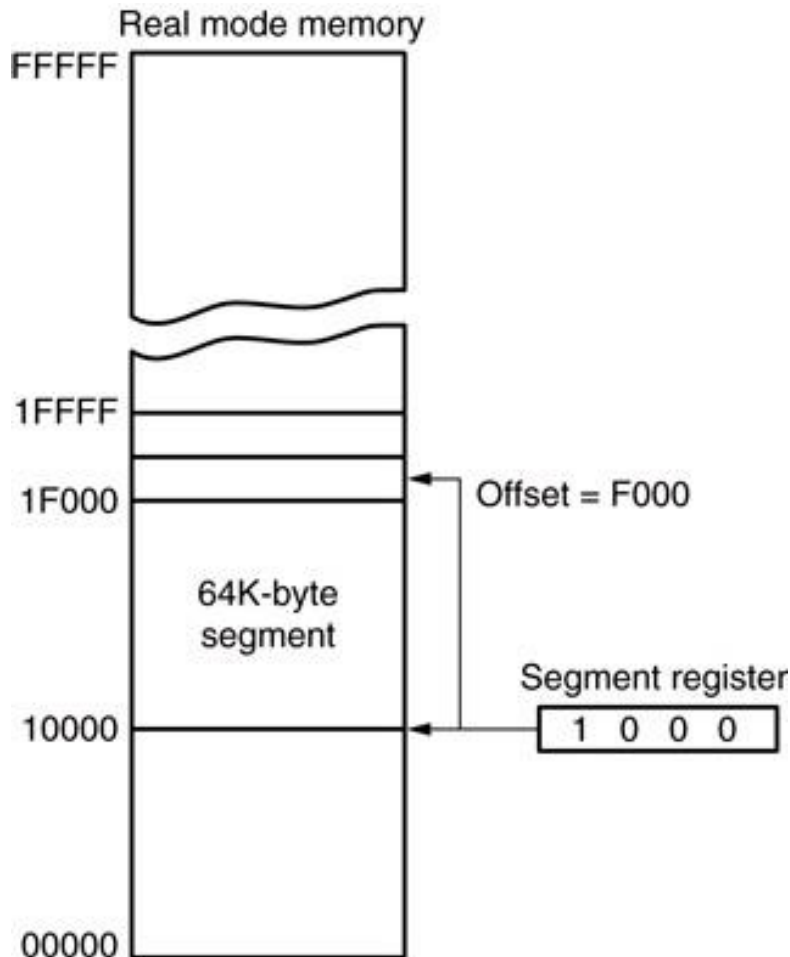
REAL MODE MEMORY ADDRESSING

- 80286 and above operate in either the real or protected mode.
- **Real mode operation** allows addressing of only the first 1M byte of memory space—even in Pentium 4 or Core2 microprocessor.
 - the first 1M byte of memory is called the **real memory, conventional memory, or DOS memory system**

Segments and Offsets

- All real mode memory addresses must consist of a segment address plus an offset address.
 - **segment address** defines the beginning address of any 64K-byte memory segment
 - **offset address** selects any location within the 64K byte memory segment
- Figure 2–3 shows how the **segment plus offset** addressing scheme selects a memory location.

Figure 2–3 The real mode memory-addressing scheme, using a segment address plus an offset.



- this shows a memory segment beginning at 10000H, ending at location 1FFFFH
 - 64K bytes in length
- also shows how an offset address, called a **displacement**, of F000H selects location 1F000H in the memory

- Once the beginning address is known, the **ending address** is found by adding FFFFH.
 - because a real mode segment of memory is 64K in length
- The offset address is always added to the segment starting address to locate the data.
- **Segment and offset address** is sometimes written as 1000:2000.
 - a segment address of 1000H; an offset of 2000H
 - yields **12000H** ($1000H \times 10H + 2000H$) real address

Default Segment and Offset Registers

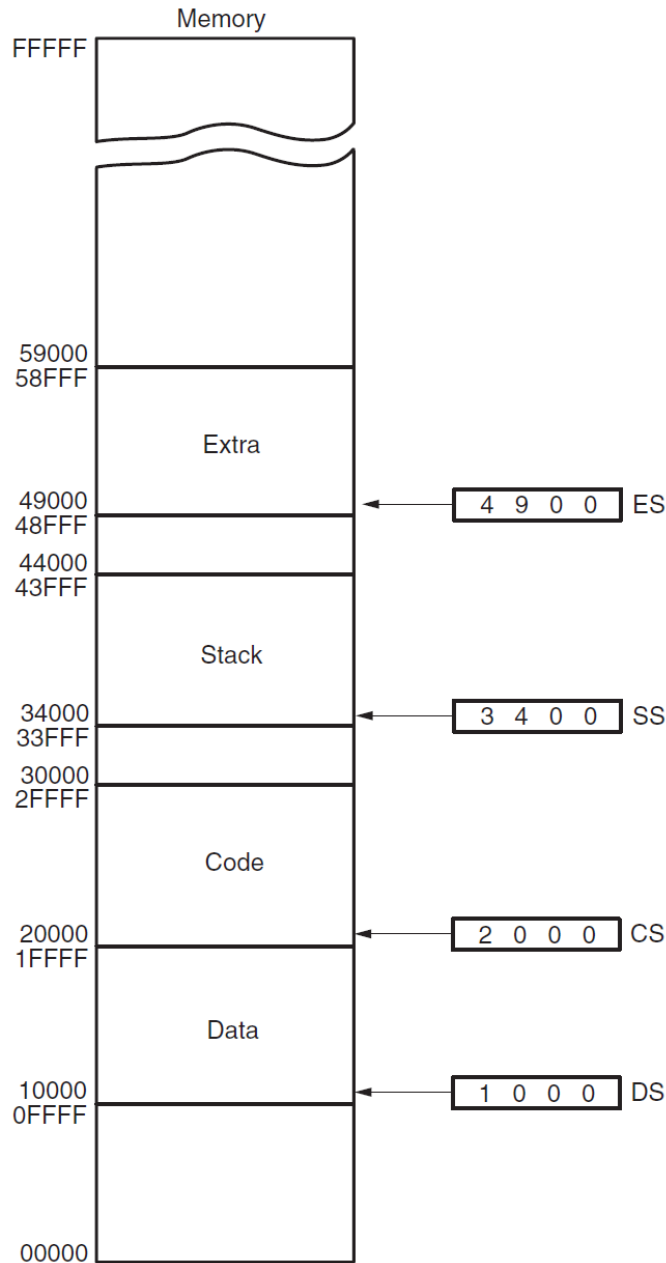
- The microprocessor has rules that apply to segments whenever memory is addressed.
 - these define the segment and offset register combination
- The **code segment** register defines the start of the code segment.
- The **instruction pointer** locates the next instruction within the code segment.

- Another of the default combinations is the **stack**.
 - stack data are referenced through the stack segment at the memory location addressed by either the stack pointer (SP/ESP) or the pointer (BP/EBP)

<i>Segment</i>		<i>Offset</i>	<i>Special Purpose</i>
CS	IP		Instruction address
SS	SP or BP		Stack address
DS	BX, DI, SI, an 8- or 16-bit number		Data address
ES	DI for string instructions		String destination address

Default 16-bit segment and offset combinations.

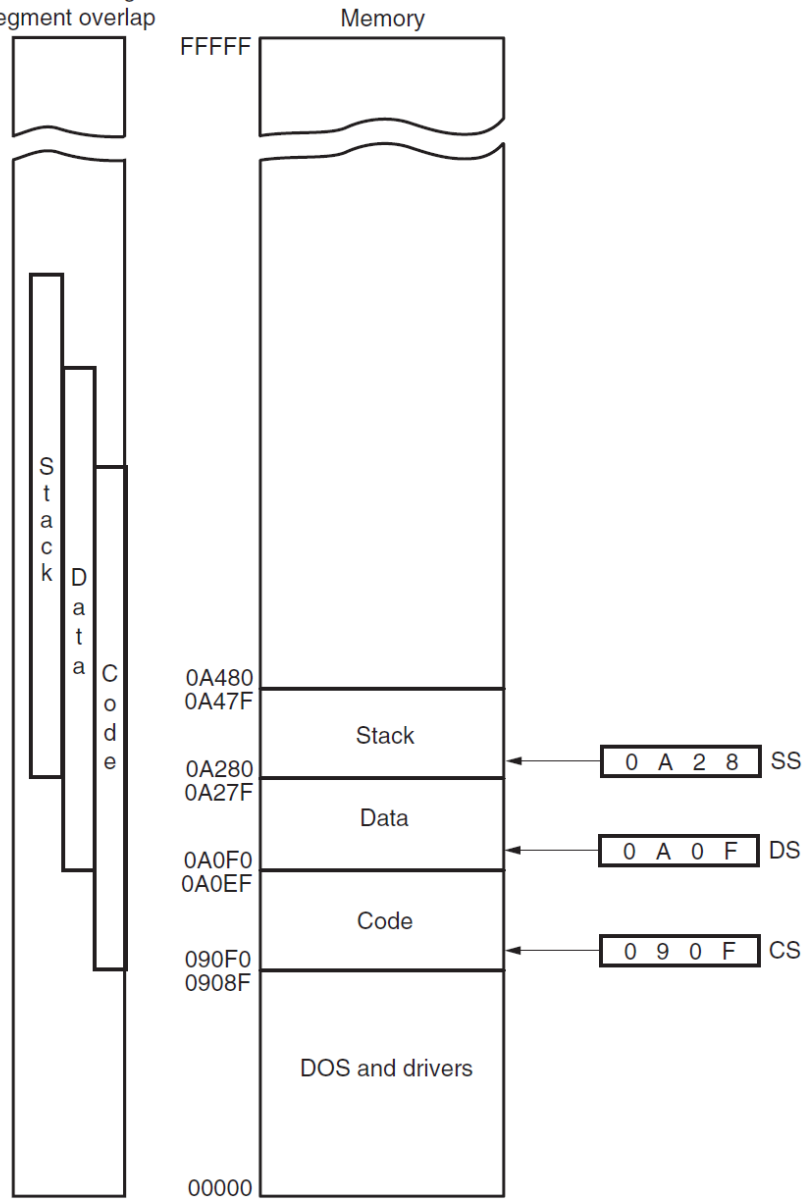
Figure 2–4 A memory system showing the placement of four memory segments.



- a memory segment can touch or overlap if 64K bytes of memory are not required for a segment
- think of segments as **windows** that can be moved over any area of memory to access data or code
- a program can have more than **four or six segments**,
 - but **only access four or six segments at a time**

Figure 2–5 An application program containing a code, data, and stack segment loaded into a DOS system memory.

Imaginary side
view detailing
segment overlap



- a program placed in memory by DOS is loaded in the TPA at the first available area of memory above drivers and other TPA programs
- area is indicated by a **free-pointer** maintained by DOS
- program loading is handled automatically by the **program loader** within DOS

Segment and Offset Addressing Scheme Allows Relocation

- Segment plus offset addressing allows DOS programs to be relocated in memory.
- A **relocatable program** is one that can be placed into **any area of memory** and **executed without change**.
- **Relocatable data** are data that can be placed in **any area of memory** and **used without any change to the program**.

- Because memory is addressed within a segment by an offset address, the memory segment can be moved to any place in the memory system without changing any of the offset addresses.
- Only the contents of the segment register must be changed to address the program in the new area of memory.
- Windows programs are written assuming that the first 2G of memory are available for code and data.

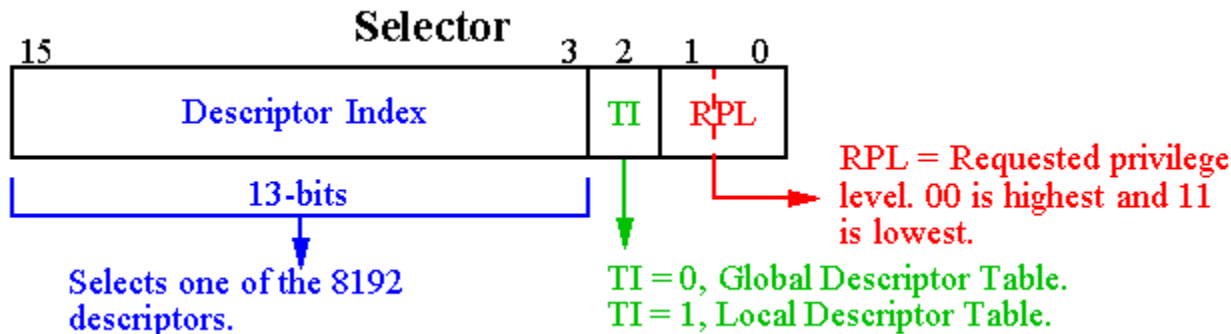
INTRO TO PROTECTED MODE MEMORY ADDRESSING

- Allows access to data and programs located within & above the first 1M byte of memory.
- **Protected mode** is where Windows operates.
- In place of a segment address, the segment register contains a **selector** that selects a descriptor from a descriptor table.
- The **descriptor** describes the **memory segment's location, length, and access rights.**

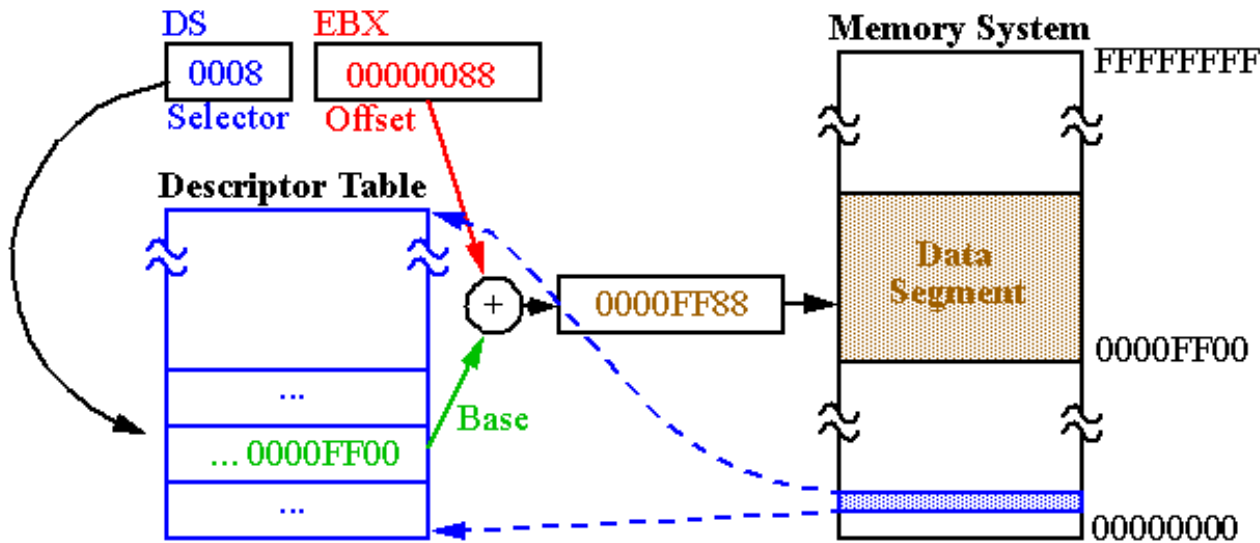
Selectors and Descriptors

- The **descriptor** is located in the segment register & describes the location, length, and access rights of the segment of memory.
 - it selects one of 8192 descriptors from one of two tables of descriptors
- In protected mode, this segment number can address any memory location in the system for the code segment.
- Indirectly, the register still selects a memory segment, but not directly as in real mode.

Protected Mode Memory Addressing



Segment Registers in Protected Mode



Segments

are interpreted differently in Protected vs. Real Mode:

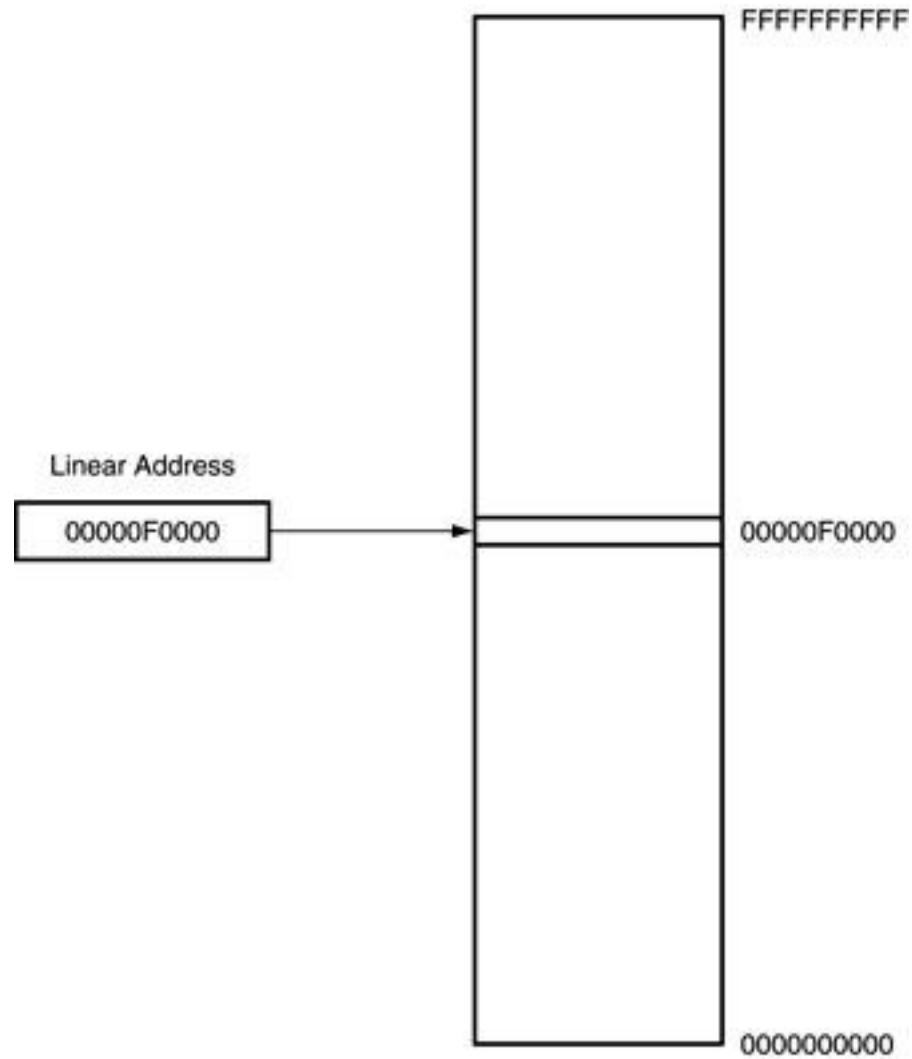
- Segment register contains a selector that selects a descriptor from the descriptor table.
- The descriptor contains information about the segment, e.g., its base address, length and access rights.

Flat Mode Memory

- A *flat mode memory* system is one in which there is no segmentation.
 - does not use a segment register to address a location in the memory
- First byte address is at 00 0000 0000H; the last location is at FF FFFF FFFFH.
 - address is 40-bits
- The segment register still selects the privilege level of the software.

- Real mode system is *not* available if the processor operates in the 64-bit mode.
- Protection and paging are allowed in the 64-bit mode.
- The CS register is still used in the protected mode operation in the 64-bit mode.
- Most programs today are operated in the IA32 compatible mode.
 - current software operates properly, but this will change in a few years as memory becomes larger and most people have 64-bit computers

Figure 2–15 The 64-bit flat mode memory model.





The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Addressing Modes

Introduction

- Efficient software development for the microprocessor requires a complete familiarity with the addressing modes employed by each instruction.
- This chapter explains the operation of the stack memory so that the PUSH and POP instructions and other stack operations will be understood.

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Explain the operation of each data-addressing mode.
- Use the data-addressing modes to form assembly language statements.
- Explain the operation of each program memory-addressing mode.
- Use the program memory-addressing modes to form assembly and machine language statements.

Chapter Objectives

(*cont.*)

Upon completion of this chapter, you will be able to:

- Select the appropriate addressing mode to accomplish a given task.
- Describe sequence of events that place data onto the stack or remove data from the stack.
- Explain how a data structure is placed in memory and used with software.

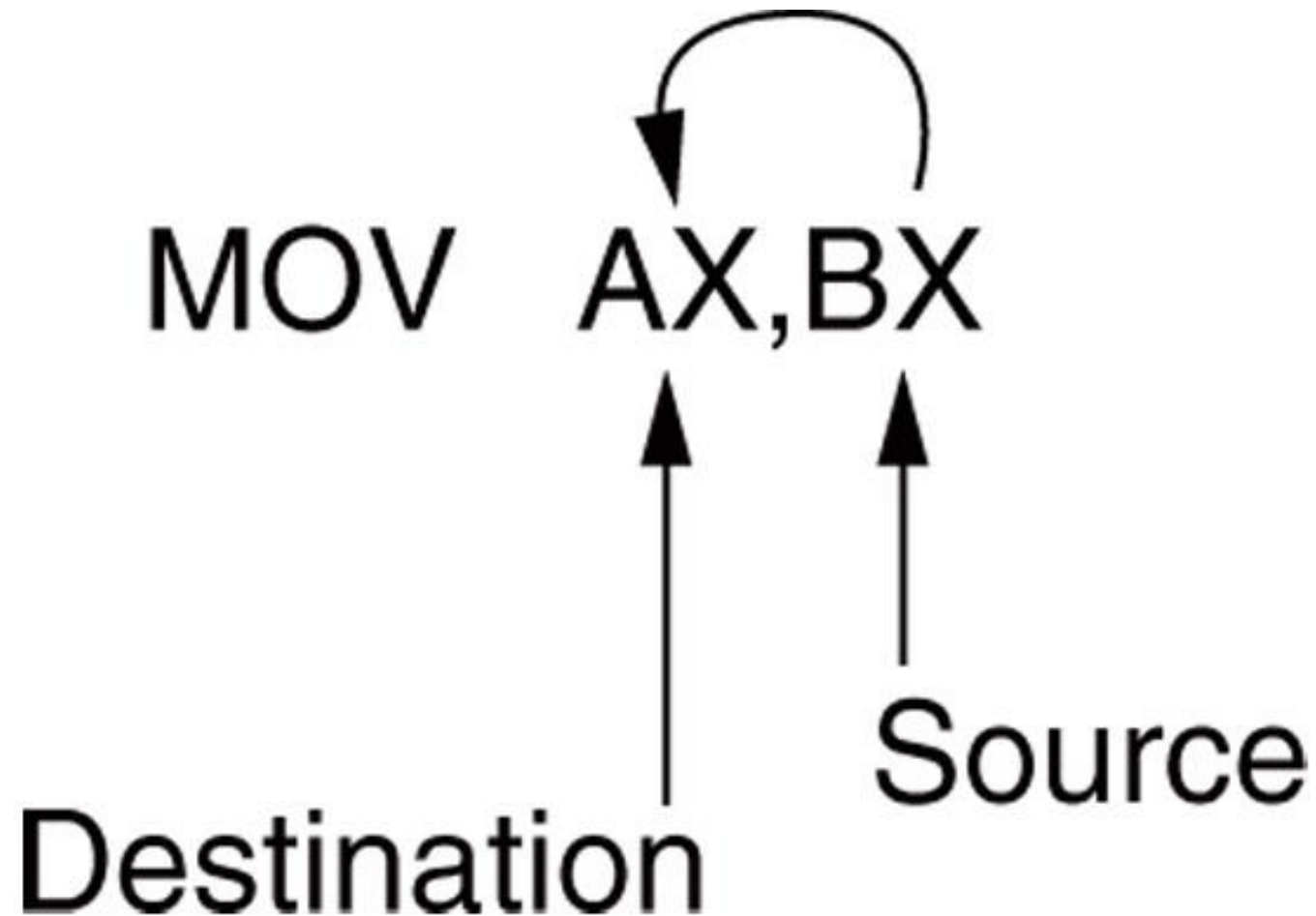
Addressing Mode Types

1. Data Addressing Modes
2. Program Memory Addressing Modes
3. Stack Memory Addressing Modes

3–1 DATA ADDRESSING MODES

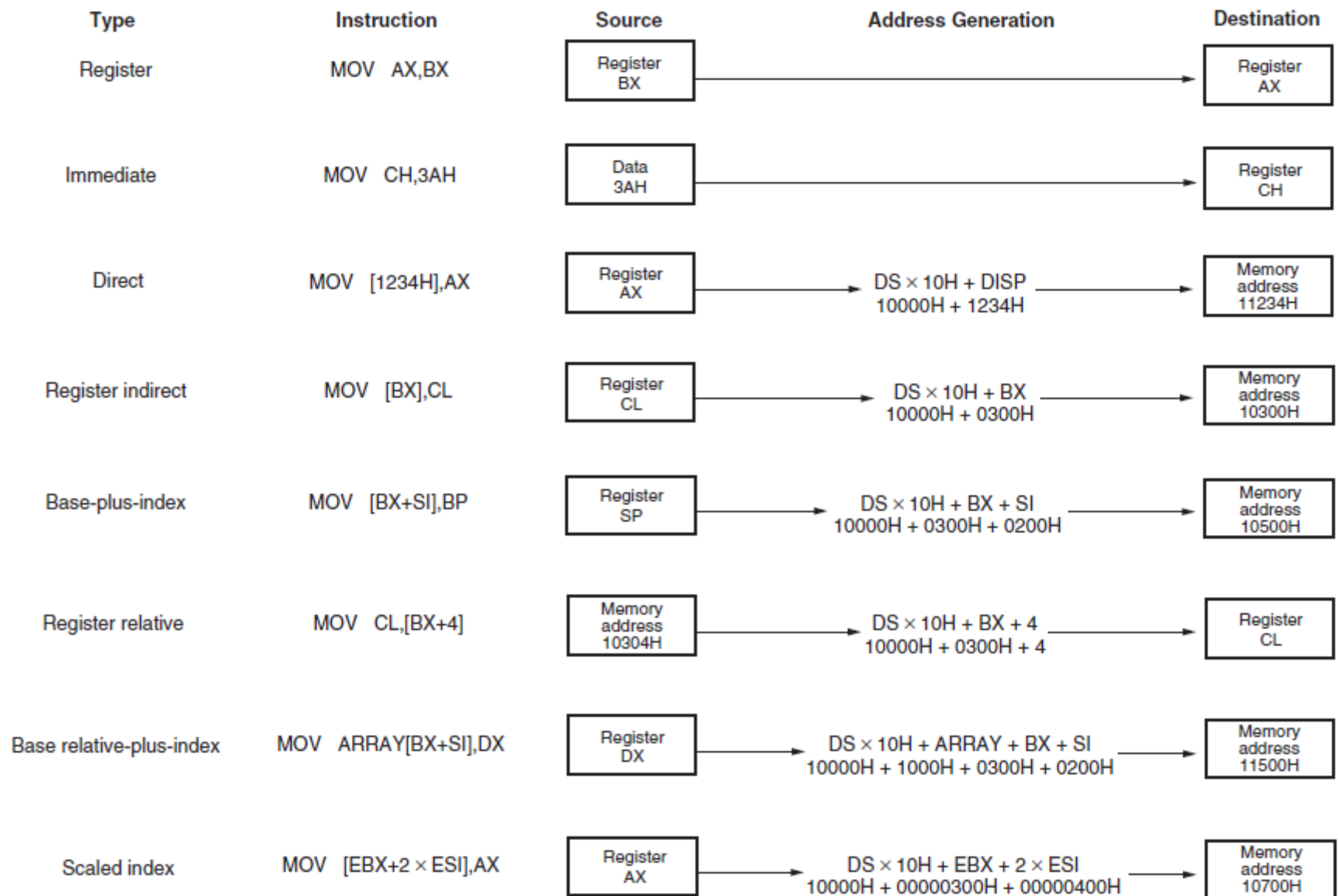
- **MOV instruction** is a common and flexible instruction.
 - provides a basis for explanation of data-addressing modes
- Figure 3–1 illustrates the MOV instruction and defines the direction of data flow.
- **Source** is to the **right** and **destination** the **left**, next to the opcode MOV.
 - an **opcode**, or operation code, tells the microprocessor **which operation to perform**

Figure 3–1 The MOV instruction showing the source, destination, and direction of data flow.



- Figure 3–2 shows all possible variations of the data-addressing modes using MOV.
- These data-addressing modes are found with all versions of the Intel microprocessor.
 - except for the scaled-index-addressing mode, found only in 80386 through Core2
- RIP relative addressing mode is not illustrated.
 - only available on the Pentium 4 and Core2 in the 64-bit mode

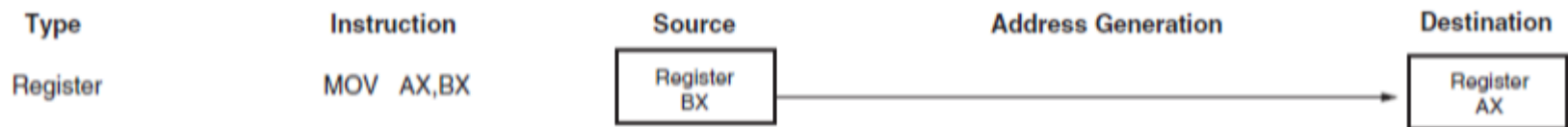
Figure 3–2 8086–Core2 data-addressing modes.



Notes: EBX = 00000300H, ESI = 00000200H, ARRAY = 1000H, and DS = 1000H

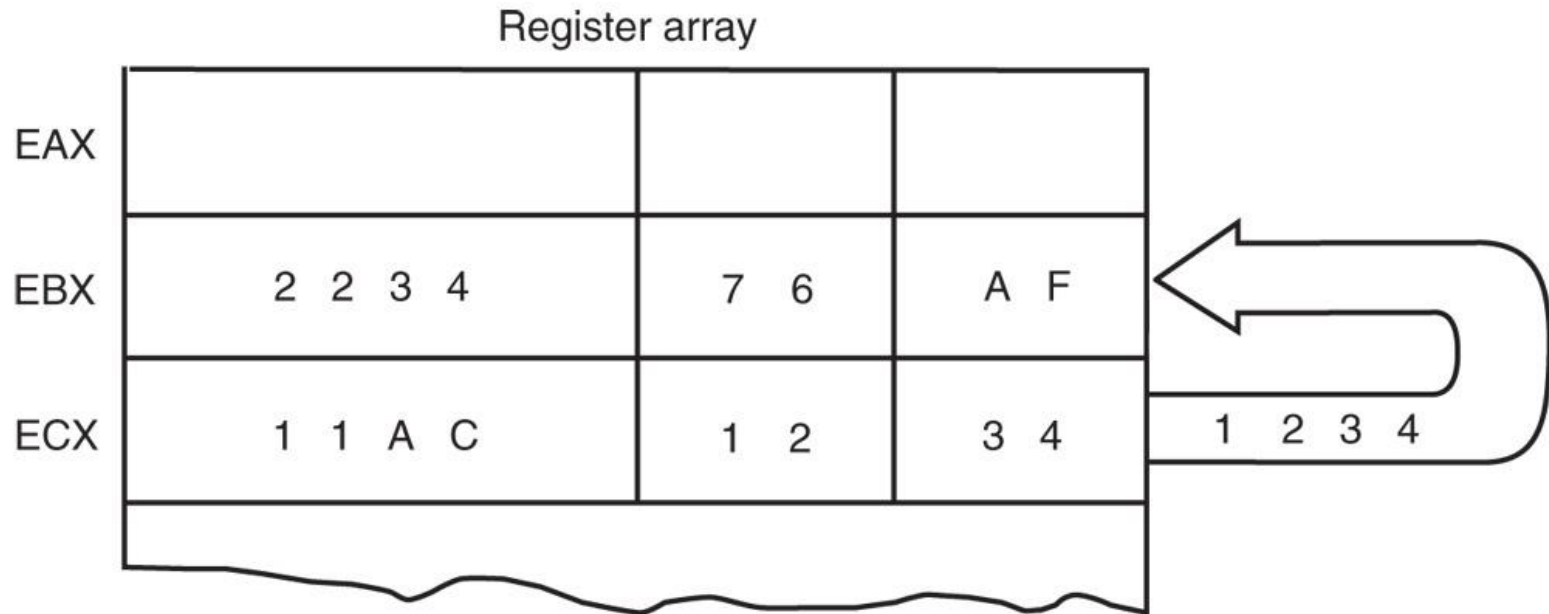
Register Addressing

- The most common form of data addressing.
 - once register names learned, easiest to apply.
- The microprocessor contains these 8-bit register names used with register addressing: AH, AL, BH, BL, CH, CL, DH, and DL.
- 16-bit register names: AX, BX, CX, DX, SP, BP, SI, and DI.



- In 80386 & above, extended 32-bit register names are: EAX, EBX, ECX, EDX, ESP, EBP, EDI, and ESI.
- 64-bit mode register names are: RAX, RBX, RCX, RDX, RSP, RBP, RDI, RSI, and R8 through R15.
- Important for instructions to use registers that are the **same size**.
 - *never* mix an 8-bit with a 16-bit register, an 8- or a 16-bit register with a 32-bit register
 - this is not allowed by the microprocessor and results in an error when assembled

Figure 3–3 The effect of executing the MOV BX, CX instruction at the point just before the BX register changes. Note that only the rightmost 16 bits of register EBX change.



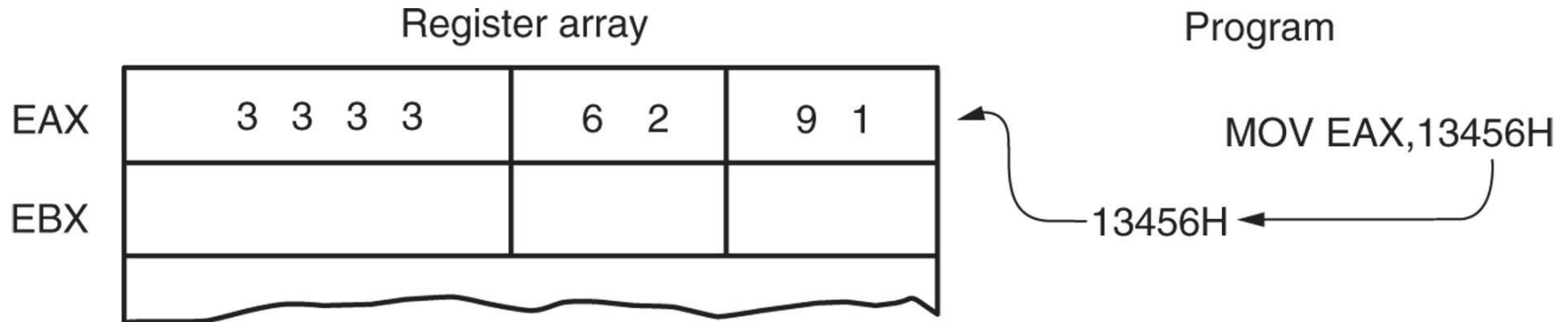
- Figure 3–3 shows the operation of the MOV BX, CX instruction.
- The source register's contents do not change.
 - the destination register's contents do change
- The contents of the destination register or destination memory location change for all instructions except the CMP and TEST instructions.
- The MOV BX, CX instruction does not affect the leftmost 16 bits of register EBX.
 - The MOV BL, CL instruction does not affect the leftmost 8 bits of register BX.

Immediate Addressing

- Term *immediate* implies that data immediately follow the hexadecimal opcode in the memory.
 - immediate data are **constant data**
 - data transferred from a register or memory location are variable data
- Immediate addressing operates upon a byte or word of data.
- Figure 3–4 shows the operation of a MOV EAX,13456H instruction.



Figure 3–4 The operation of the MOV EAX,13456H instruction. This instruction copies the immediate data (13456H) into EAX.



- As with the MOV instruction illustrated in Figure 3–3, the source data overwrites the destination data.

- In symbolic assembly language, the symbol # (number or pound sign) precedes immediate data in some assemblers.
 - MOV AX,#3456H instruction is an example
- Most assemblers do not use the # symbol, but represent immediate data as in the MOV AX,3456H instruction.
 - an older assembler used with some Hewlett-Packard logic development does, as may others
 - in this text, the # is not used for immediate data

- The symbolic assembler portrays immediate data in many ways.
- The letter **H** appends hexadecimal data.
- If hexadecimal data begin with a letter, the assembler requires the data start with a **0**.
 - to represent a hexadecimal F2, 0F2H is used in assembly language
- Decimal data are represented as is and require no special codes or adjustments.
 - an example is the 100 decimal in the MOV AL,100 instruction

- An ASCII-coded character or characters may be depicted in the immediate form if the ASCII data are enclosed in apostrophes.
 - be careful to use the apostrophe (') for ASCII data and not the single quotation mark (‘)
- Binary data are represented if the binary number is followed by the letter B.
 - in some assemblers, the letter Y
 - to represent a decimal 10, 00001010B is used in assembly language

- Each statement in an assembly language program consists of four parts or fields.
- The leftmost field is called the *label*.
 - used to store a symbolic name for the memory location it represents
- All labels must begin with a letter or one of the following special characters: @, \$, -, or ?.
 - a label may any length from 1 to 35 characters
- The label appears in a program to identify the name of a memory location for storing data and for other purposes.

- The next field to the right is the *opcode field*.
 - designed to hold the instruction, or opcode
 - the MOV part of the move data instruction is an example of an opcode
- Right of the opcode field is the *operand field*.
 - contains information used by the opcode
 - the MOV AL,BL instruction has the opcode MOV and operands AL and BL
- The *comment field*, the final field, contains a comment about the instruction(s).
 - comments always begin with a semicolon (;)

Sample Code

```
name "add-sub"

org 100h

mov al, 5          ; bin=00000101b
mov bl, 10         ; hex=0ah or bin=00001010b

; 5 + 10 = 15 <decimal> or hex=0fh or bin=00001111b
add bl, al

; 15 - 1 = 14 <decimal> or hex=0eh or bin=00001110b
sub bl, 1

; print result in binary:
mov cx, 8
print: mov ah, 2    ; print function.
       mov dl, '0'
       test bl, 10000000b ; test first bit.
       jz zero
       mov dl, '1'
zero:  int 21h
       shl bl, 1
loop print

; print binary suffix:
mov dl, 'b'
int 21h

; wait for any key press:
mov ah, 0
int 16h

ret
```

Direct Data Addressing

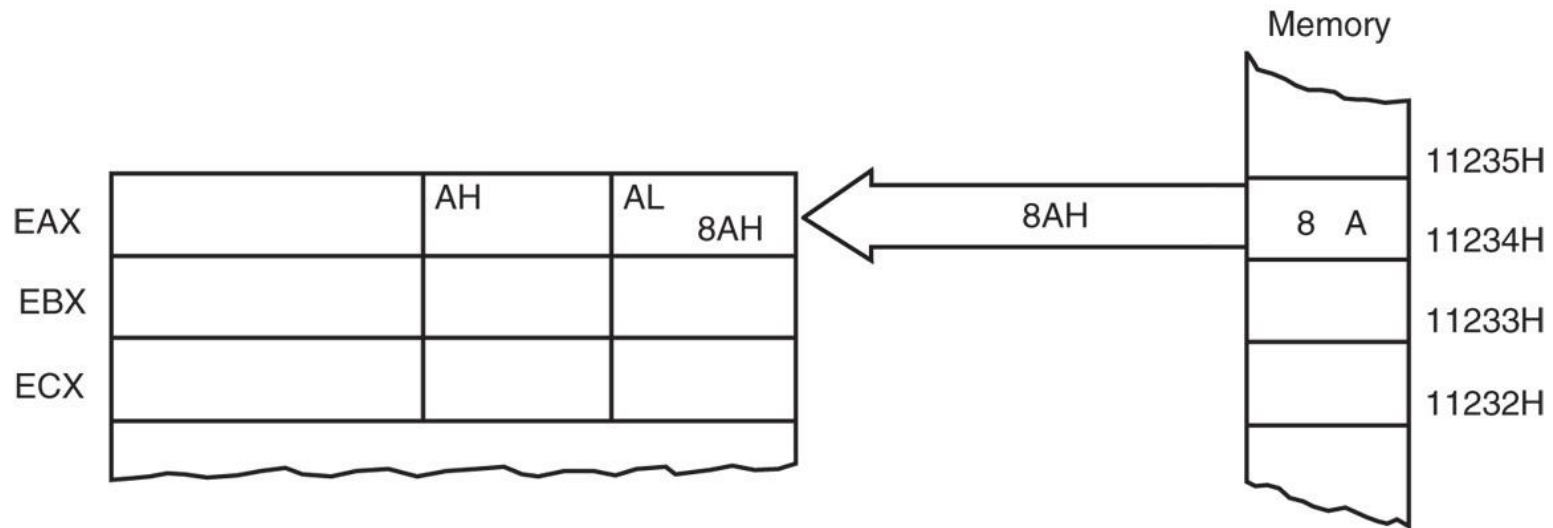
- Applied to many instructions in a typical program.
- Two basic forms of direct data addressing:
 - **direct addressing**, which applies to a MOV between a memory location and AL, AX, or EAX
 - **displacement addressing**, which applies to almost any instruction in the instruction set
- Address is formed by adding the displacement to the default data segment address or an alternate segment address.

Direct Addressing

- Direct addressing with a MOV instruction transfers data between a memory location, located within the data segment, and the AL (8-bit), AX (16-bit), or EAX (32-bit) register.
 - usually a 3-byte long instruction
- MOV AL,DATA loads AL from the data segment memory location DATA (1234H).
 - DATA is a symbolic memory location, while 1234H is the actual hexadecimal location



Figure 3–5 The operation of the MOV AL,[1234H] instruction when DS=1000H .



- This instruction transfers a copy contents of memory location 11234H into AL.
 - the effective address is formed by adding 1234H (the offset address) and 10000H (the data segment address of 1000H times 10H) in a system operating in the real mode

Displacement Addressing

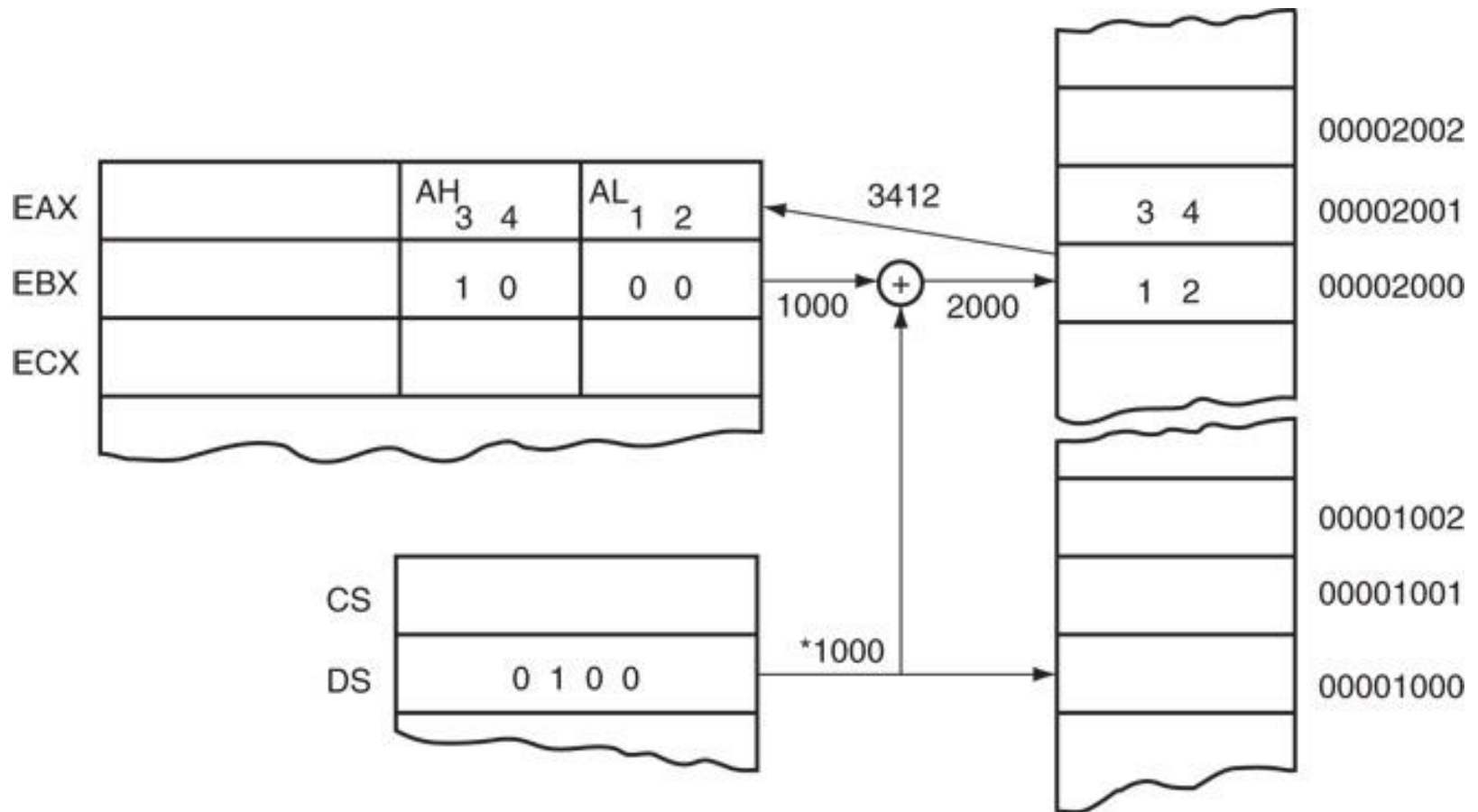
- Almost identical to direct addressing, except the instruction is 4 bytes wide instead of 3.
- In 80386 through Pentium 4, this instruction can be up to 7 bytes wide if a 32-bit register and a 32-bit displacement are specified.
- This type of direct data addressing is much more flexible because most instructions use it.

Register Indirect Addressing

- Allows data to be addressed at any memory location through an offset address held in any of the following registers: BP, BX, DI, and SI.
- In addition, 80386 and above allow register indirect addressing with any extended register except ESP.
- In the 64-bit mode, the segment registers serve no purpose in addressing a location in the flat model.



Figure 3–6 The operation of the MOV AX,[BX] instruction when BX = 1000H and DS = 0100H. Note that this instruction is shown after the contents of memory are transferred to AX.



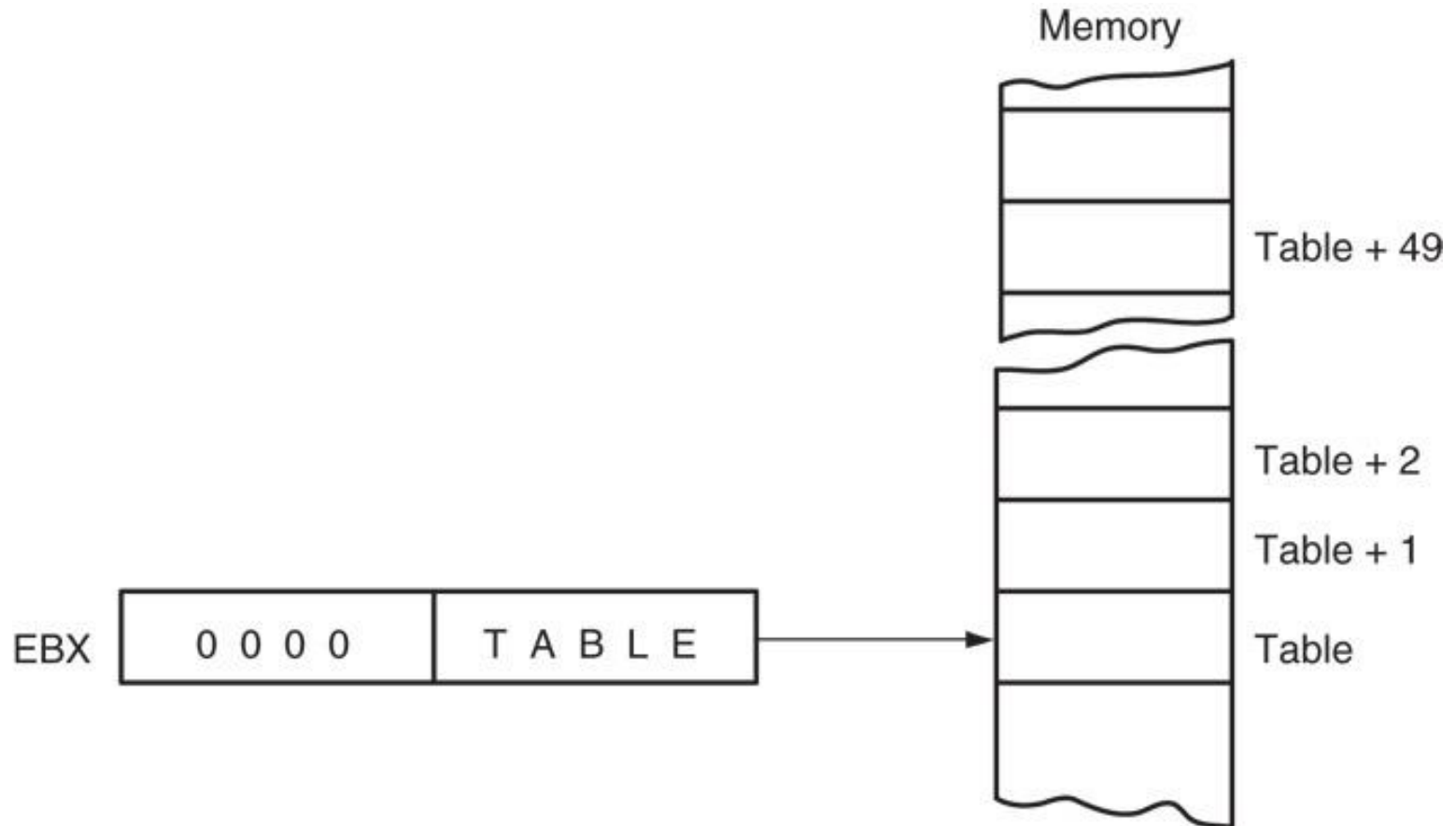
*After DS is appended with a 0.

- The **data segment** is used by default with register indirect addressing or any other mode that uses BX, DI, or SI to address memory.
- If the BP register addresses memory, the **stack segment** is used by default.
 - these settings are considered the default for these four index and base registers
- For the 80386 and above, EBP addresses memory in the stack segment by default.
- EAX, EBX, ECX, EDX, EDI, and ESI address memory in the data segment by default.

- In some cases, indirect addressing requires specifying the size of the data by the **special assembler directive** BYTE PTR, WORD PTR, DWORD PTR, or QWORD PTR.
 - these directives indicate the size of the memory data addressed by the **memory pointer** (PTR)
- The directives are with instructions that address a memory location through a pointer or index register with immediate data.
- With SIMD instructions, the octal OWORD PTR, represents a **128-bit-wide number**.

- Indirect addressing often allows a program to refer to tabular data located in memory.
- Figure 3–7 shows the table and the BX register used to sequentially address each location in the table.
- To accomplish this task, load the starting location of the table into the BX register with a MOV immediate instruction.
- After initializing the starting address of the table, use register indirect addressing to store the 50 samples sequentially.

Figure 3–7 An array (TABLE) containing 50 bytes that are indirectly addressed through register BX.



Base-Plus-Index Addressing

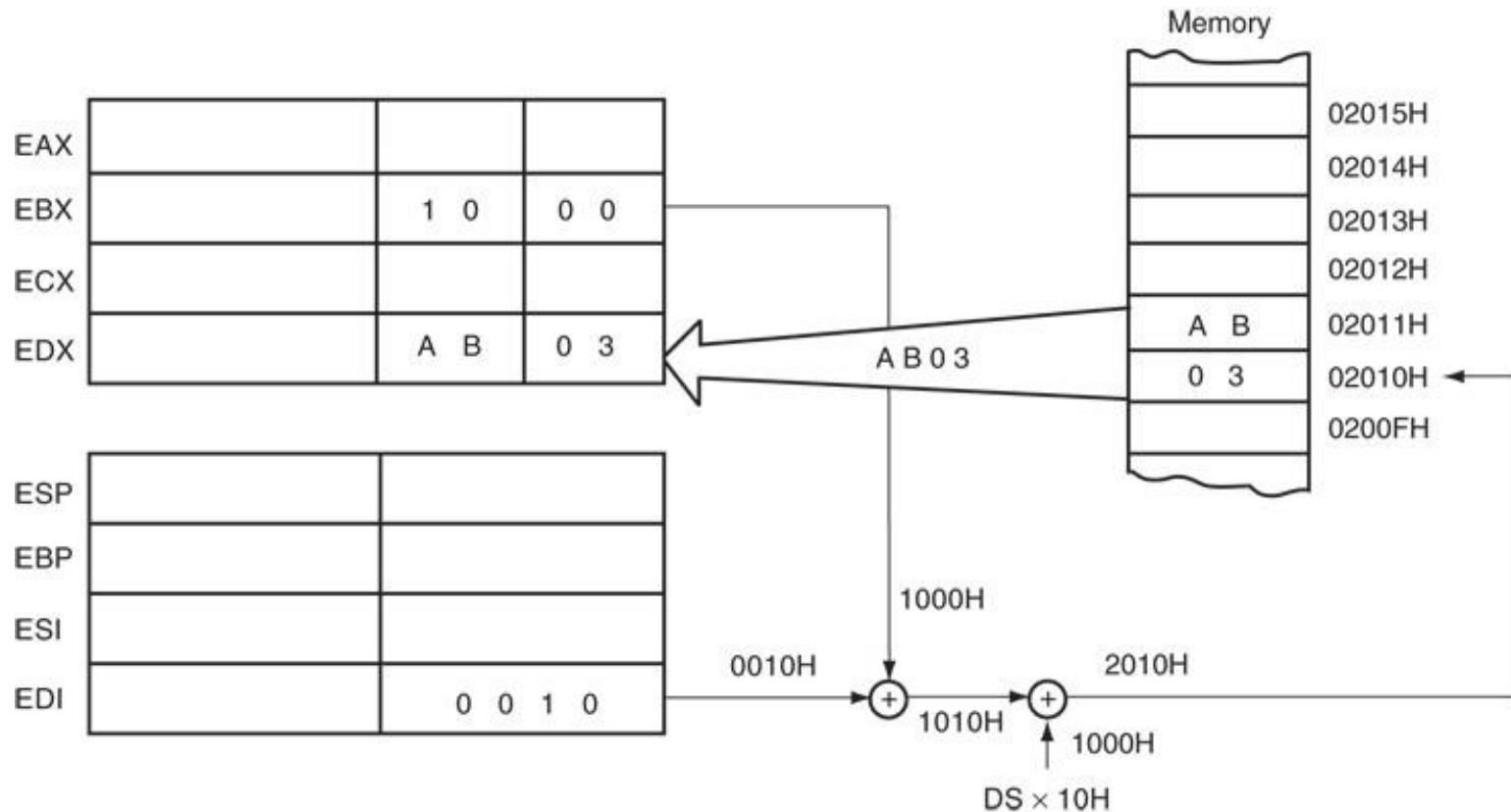
- Similar to indirect addressing because it indirectly addresses memory data.
- The base register often holds the beginning location of a memory array.
 - the index register holds the relative position of an element in the array
 - whenever BP addresses memory data, both the stack segment register and BP generate the effective address



Locating Data with Base-Plus-Index Addressing

- Figure 3–8 shows how data are addressed by the `MOV DX,[BX + DI]` instruction when the microprocessor operates in the real mode.
- The Intel assembler requires this addressing mode appear as `[BX][DI]` instead of `[BX + DI]`.
- The `MOV DX,[BX + DI]` instruction is `MOV DX,[BX][DI]` for a program written for the Intel ASM assembler.

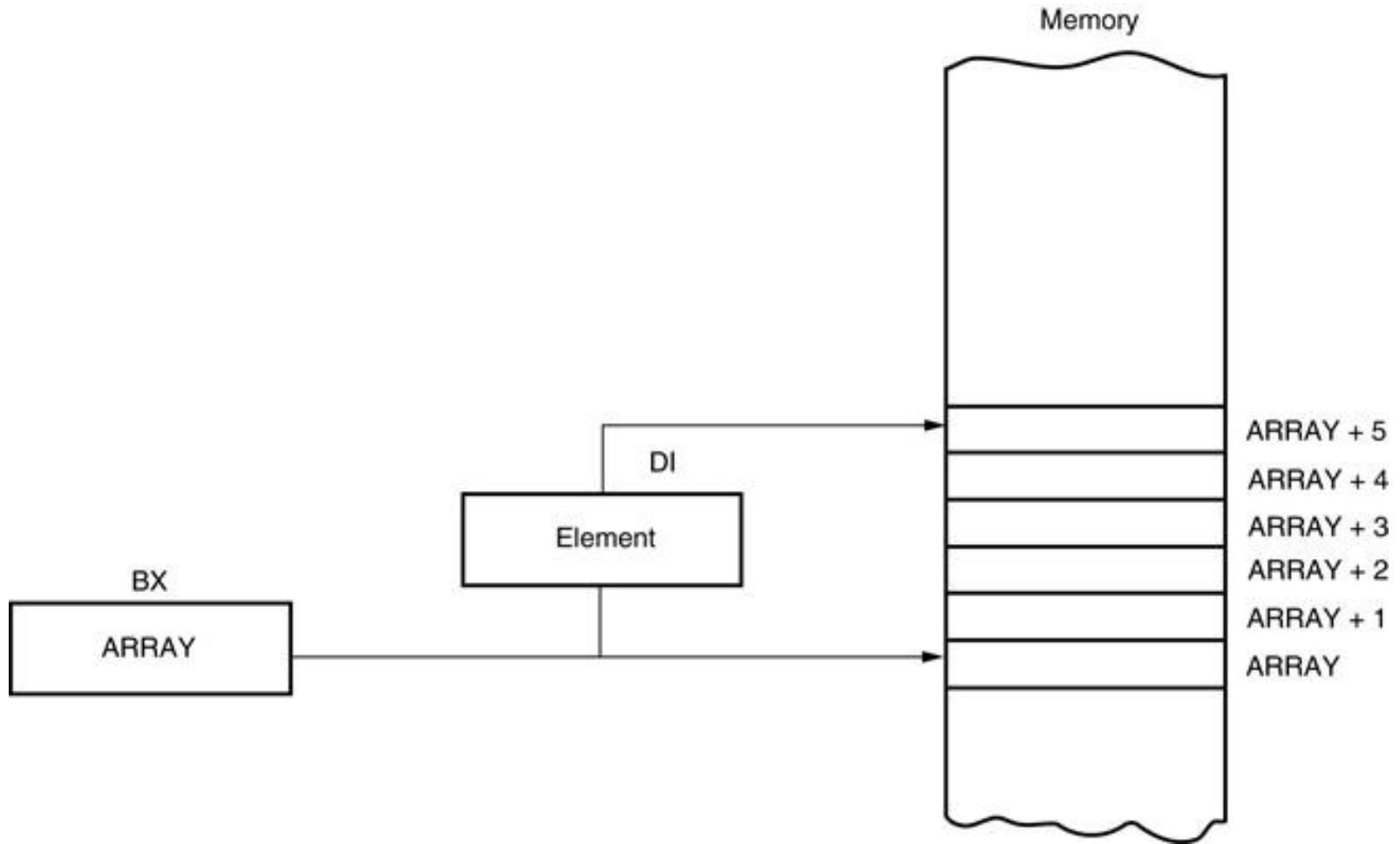
Figure 3–8 An example showing how the base-plus-index addressing mode functions for the MOV DX,[BX + DI] instruction. Notice that memory address 02010H is accessed because DS=0100H, BX=100H and DI=0010H.



Locating Array Data Using Base-Plus-Index Addressing

- A major use is to address elements in a memory array.
- To accomplish this, load the BX register (base) with the beginning address of the array and the DI register (index) with the element number to be accessed.
- Figure 3–9 shows the use of BX and DI to access an element in an array of data.

Figure 3–9 An example of the base-plus-index addressing mode. Here an element (DI) of an ARRAY (BX) is addressed.



Register Relative Addressing

- Similar to base-plus-index addressing and displacement addressing.
 - data in a segment of memory are addressed by adding the displacement to the contents of a base or an index register (BP, BX, DI, or SI)
- Figure 3–10 shows the operation of the MOV AX,[BX+1000H] instruction.
- A real mode segment is 64K bytes long.

Register relative

MOV CL,[BX+4]

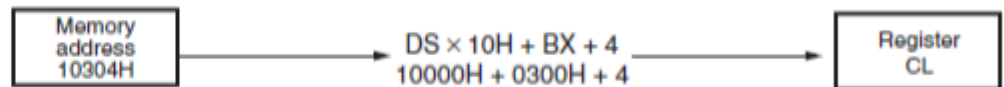
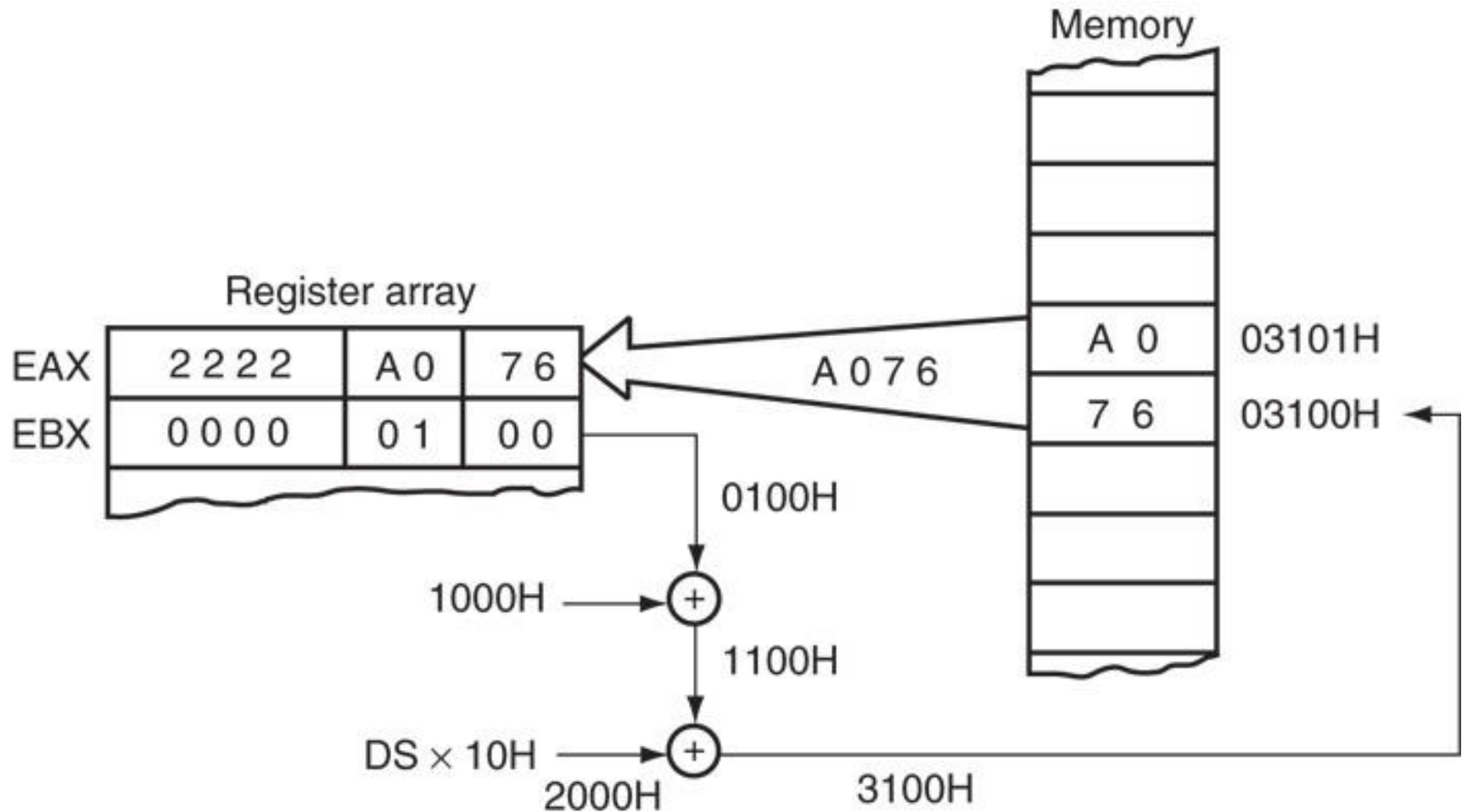


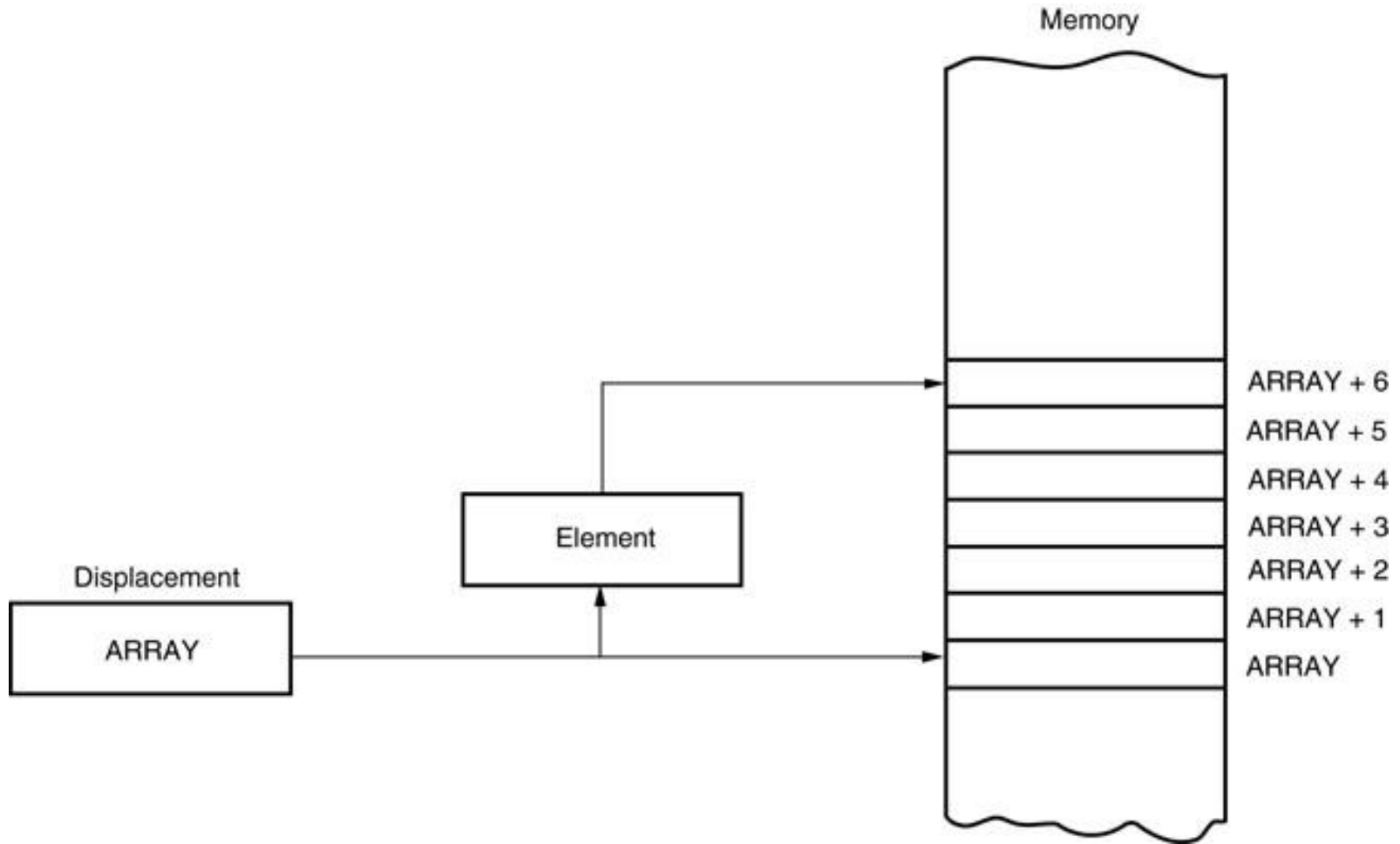
Figure 3–10 The operation of the MOV AX, [BX+1000H] instruction, when BX=0100H and DS=0200H .



Addressing Array Data with Register Relative

- It is possible to address array data with register relative addressing.
 - such as with base-plus-index addressing
- In Figure 3–11, register relative addressing is illustrated with the same example as for base-plus-index addressing.
 - this shows how the displacement ARRAY adds to index register DI to generate a reference to an array element

Figure 3–11 Register relative addressing used to address an element of ARRAY. The displacement addresses the start of ARRAY, and DI accesses an element.



Base Relative-Plus-Index Addressing

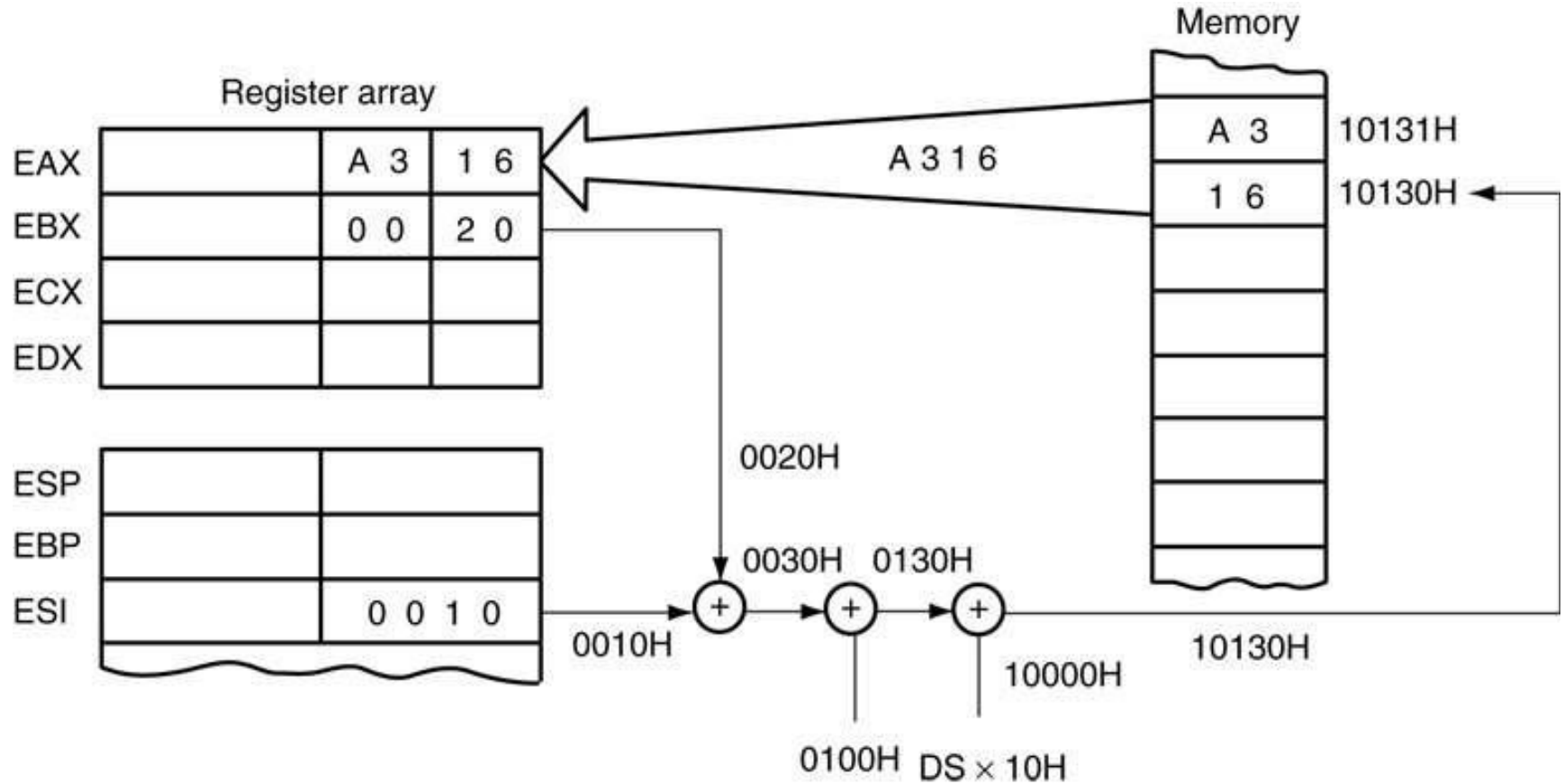
- Similar to base-plus-index addressing.
 - adds a displacement
 - uses a base register and an index register to form the memory address
- This type of addressing mode often addresses a two-dimensional array of memory data.



Addressing Data with Base Relative-Plus-Index

- Least-used addressing mode.
- Figure 3–12 shows how data are referenced if the instruction executed by the microprocessor is `MOV AX,[BX + SI + 100H]`.
 - displacement of 100H adds to BX and SI to form the offset address within the data segment
- This addressing mode is too complex for frequent use in programming.

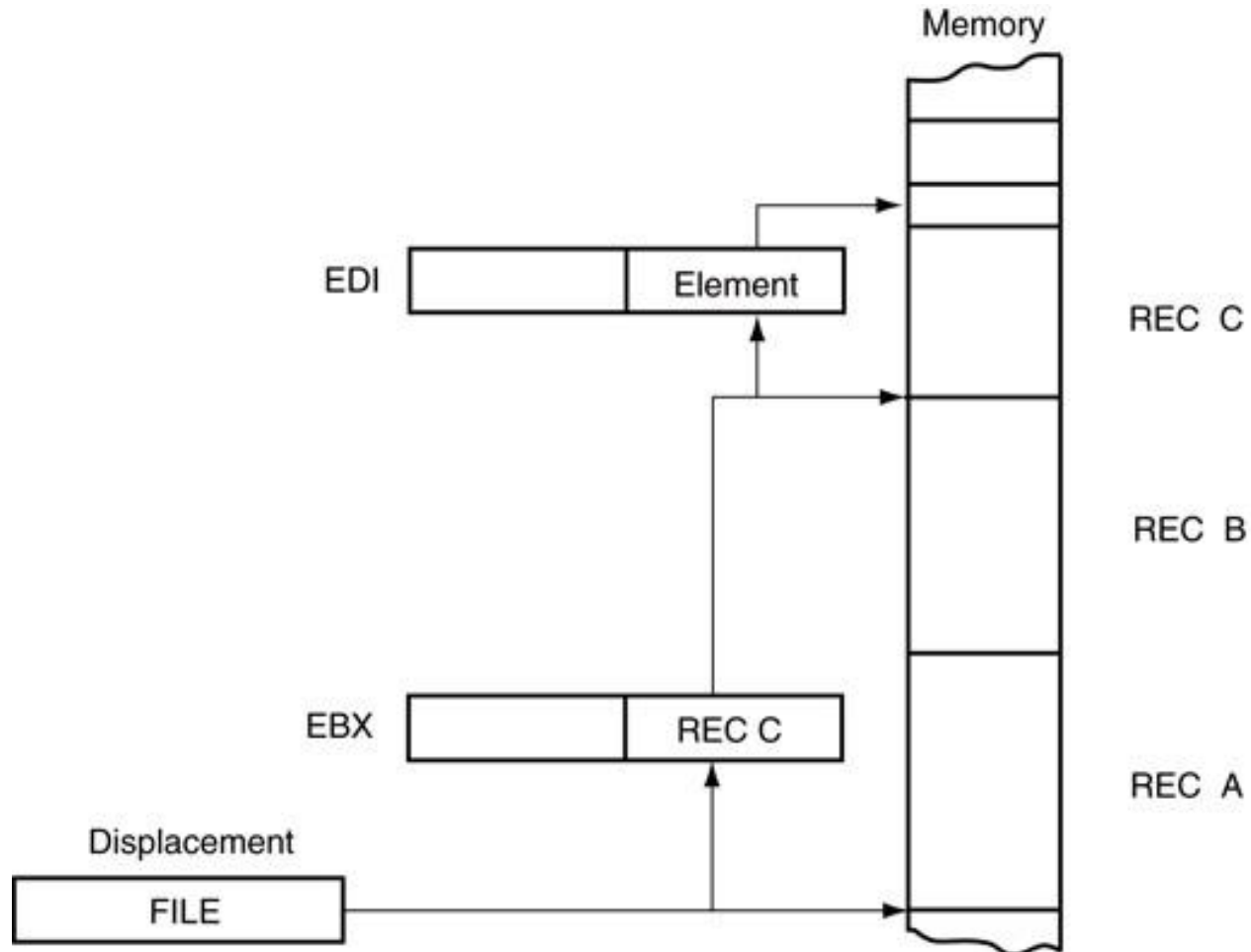
Figure 3–12 An example of base relative-plus-index addressing using a MOV AX,[BX+SI+100H] instruction. Note: DS=1000H



Addressing Arrays with Base Relative-Plus-Index

- Suppose a file of many records exists in memory, each record with many elements.
 - displacement addresses the file, base register addresses a record, the index register addresses an element of a record
- Figure 3–13 illustrates this very complex form of addressing.

Figure 3–13 Base relative-plus-index addressing used to access a FILE that contains multiple records (REC).



Data Structures

- Used to specify how information is stored in a memory array.
 - a template for data
- The start of a structure is identified with the **STRUC** assembly language directive and the end with the **ENDS** statement.

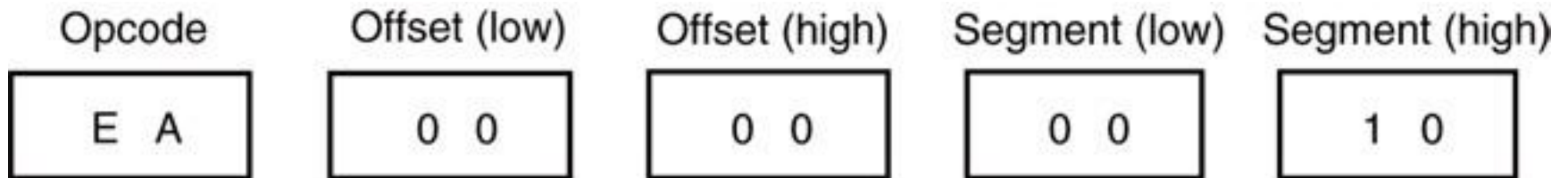
3–2 PROGRAM MEMORY-ADDRESSING MODES

- Used with the **JMP** (jump) and **CALL** instructions.
- Consist of three distinct forms:
 - **direct, relative, and indirect**

Direct Program Memory Addressing

- Used for all jumps and calls by early microprocessor; also used in high-level languages, such as BASIC.
 - GOTO and GOSUB instructions
- The microprocessor uses this form, but not as often as relative and indirect program memory addressing.
- The instructions for direct program memory addressing store the address with the opcode.

Figure 3–14 The 5-byte machine language version of a JMP [1000H] instruction.



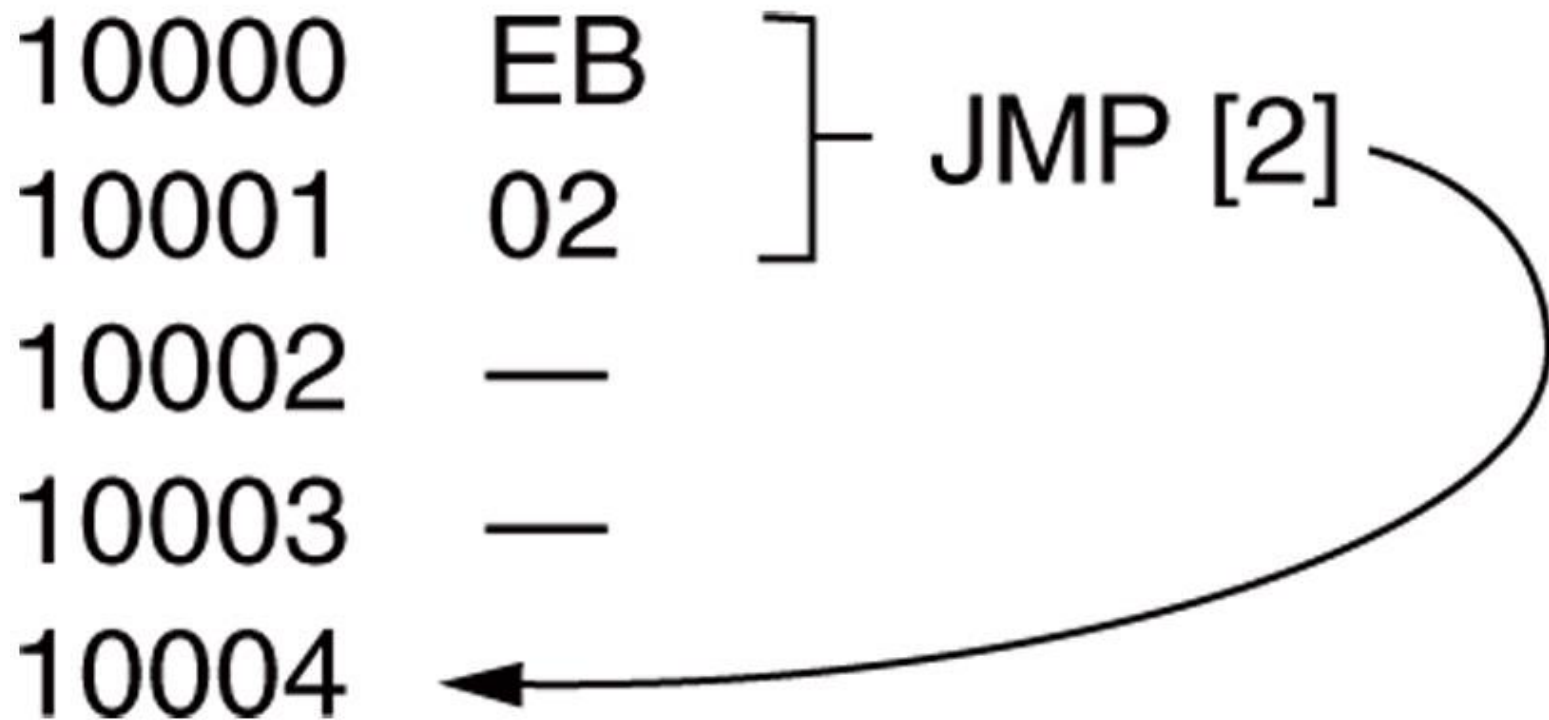
- This JMP instruction loads CS with 1000H and IP with 0000H to jump to memory location 10000H for the next instruction.
 - an **intersegment jump** is a jump to any memory location within the entire memory system
- Often called a *far jump* because it can jump to any memory location for the next instruction.
 - in real mode, any location within the **first 1M byte**
 - In protected mode operation, the far jump can jump to any location in the 4G-byte address range in the 80386 - Core2 microprocessors

- The only other instruction using direct program addressing is the intersegment or far **CALL** instruction.
- Usually, the name of a memory address, called a *label*, refers to the location that is called or jumped to instead of the actual numeric address.
- When using a label with the CALL or JMP instruction, most assemblers select the best form of program addressing.

Relative Program Memory Addressing

- Not available in all early microprocessors, but it is available to this family of microprocessors.
- The term *relative* means “relative to the instruction pointer (IP)”.
- The JMP instruction is a 1-byte instruction, with a 1-byte or a 2-byte displacement that adds to the instruction pointer.
- An example is shown in Figure 3–15.

Figure 3–15 A JMP [2] instruction. This instruction skips over the 2 bytes of memory that follow the JMP instruction.



Indirect Program Memory Addressing

- The microprocessor allows several forms of program indirect memory addressing for the JMP and CALL instructions.
- In 80386 and above, an extended register can be used to hold the address or indirect address of a relative JMP or CALL.
 - for example, the JMP EAX jumps to the location address by register EAX

- If a relative register holds the address, the jump is considered to be an indirect jump.
- For example, JMP [BX] refers to the memory location within the data segment at the offset address contained in BX.
 - at this offset address is a 16-bit number used as the offset address in the intrasegment jump
 - this type of jump is sometimes called an *indirect-indirect* or *double-indirect jump*
- Figure 3–16 shows a jump table that is stored, beginning at memory location TABLE.

Table 3–10 Examples of indirect program memory addressing.

<i>Assembly Language</i>	<i>Operation</i>
JMP AX	Jumps to the current code segment location addressed by the contents of AX
JMP CX	Jumps to the current code segment location addressed by the contents of CX
JMP NEAR PTR[BX]	Jumps to the current code segment location addressed by the contents of the data segment location addressed by BX
JMP NEAR PTR[DI+2]	Jumps to the current code segment location addressed by the contents of the data segment memory location addressed by DI plus 2
JMP TABLE[BX]	Jumps to the current code segment location addressed by the contents of the data segment memory location address by TABLE plus BX
JMP ECX	Jumps to the current code segment location addressed by the contents of ECX
JMP RDI	Jumps to the linear address contained in the RDI register (64-bit mode)

Figure 3–16 A jump table that stores addresses of various programs. The exact address chosen from the TABLE is determined by an index stored with the jump instruction.

```
TABLE  DW  LOC0
        DW  LOC1
        DW  LOC2
        DW  LOC3
```

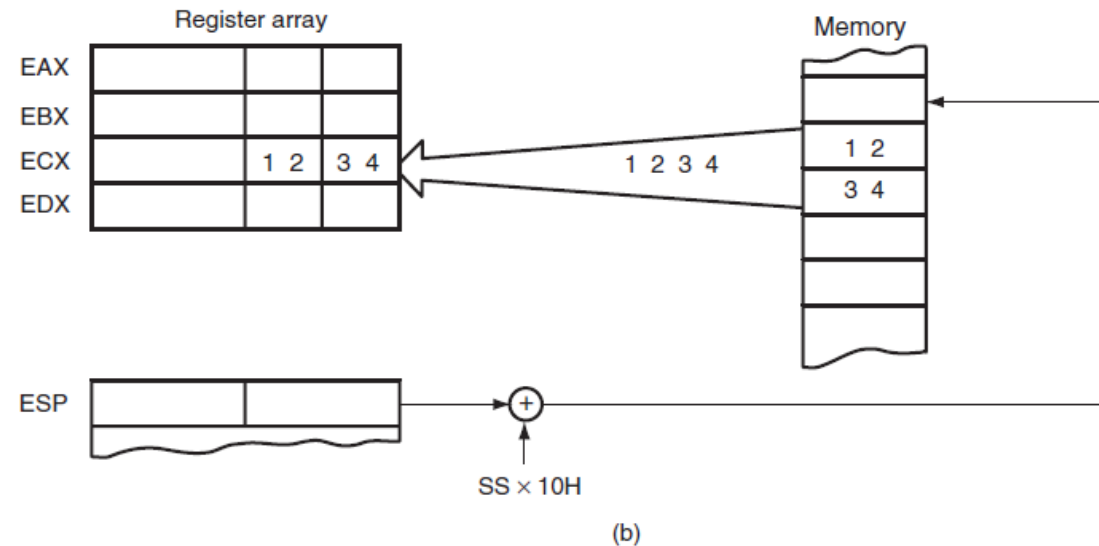
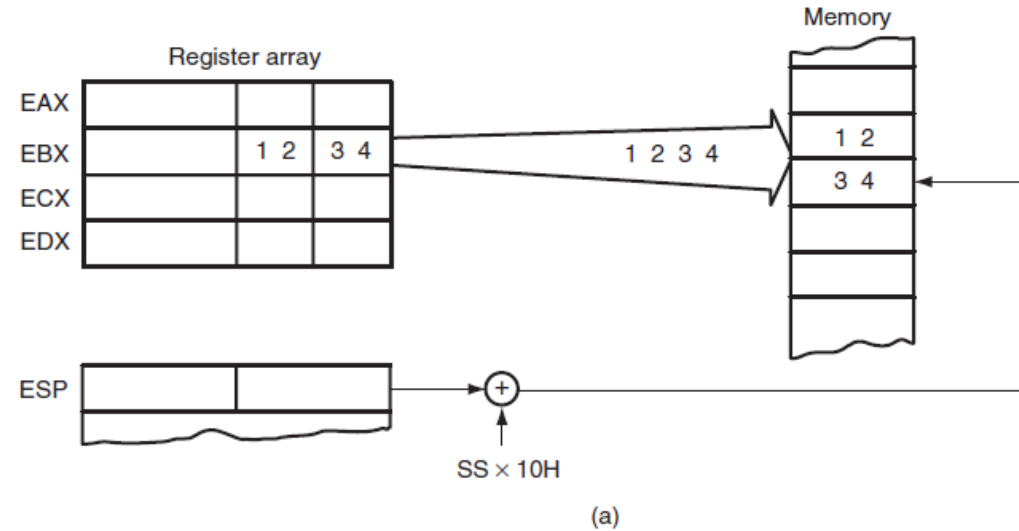
3–3 STACK MEMORY-ADDRESSING MODES

- The stack plays an important role in all microprocessors.
 - holds data temporarily and stores return addresses used by procedures
- Stack memory is LIFO (**last-in, first-out**) memory
 - describes the way data are stored and removed from the stack

- Data are placed on the stack with a **PUSH instruction**; removed with a **POP instruction**.
- Stack memory is maintained by two registers:
 - the stack pointer (SP or ESP)
 - the stack segment register (SS)
- Whenever a word of data is pushed onto the stack, the high-order 8 bits are placed in the location addressed by $SP - 1$.
 - low-order 8 bits are placed in the location addressed by $SP - 2$

- The SP is decremented by 2 so the next word is stored in the next available stack location.
 - the SP/ESP register always points to an area of memory located within the stack segment.
- When data are popped from the stack, the low-order 8 bits are removed from the location addressed by SP.
- Then, high-order 8 bits are removed;
- And, SP register is incremented by 2

Figure 3–17 The PUSH and POP instructions: (a) PUSH BX places the contents of BX onto the stack; (b) POP CX removes data from the stack and places them into CX. Both instructions are shown after execution.



- Note that **PUSH and POP store or retrieve words of data—never bytes—in 8086 - 80286.**
- 80386 and above allow words or doublewords to be transferred to and from the stack.
- Data may be pushed onto the stack from any **16-bit register or segment register.**
 - in 80386 and above, from any 32-bit extended register
- Data may be popped off the stack into any register or any segment register except CS.

- **PUSHA** and **POPA** instructions push or pop all **except segment registers**, on the stack.
- Not available on early 8086/8088 processors.
- 80386 and above allow extended registers to be pushed or popped.
 - 64-bit mode for Pentium and Core2 does not contain a PUSHA or POPA instruction



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Data Movement Instructions

Introduction

- This chapter concentrates on common data movement instructions.

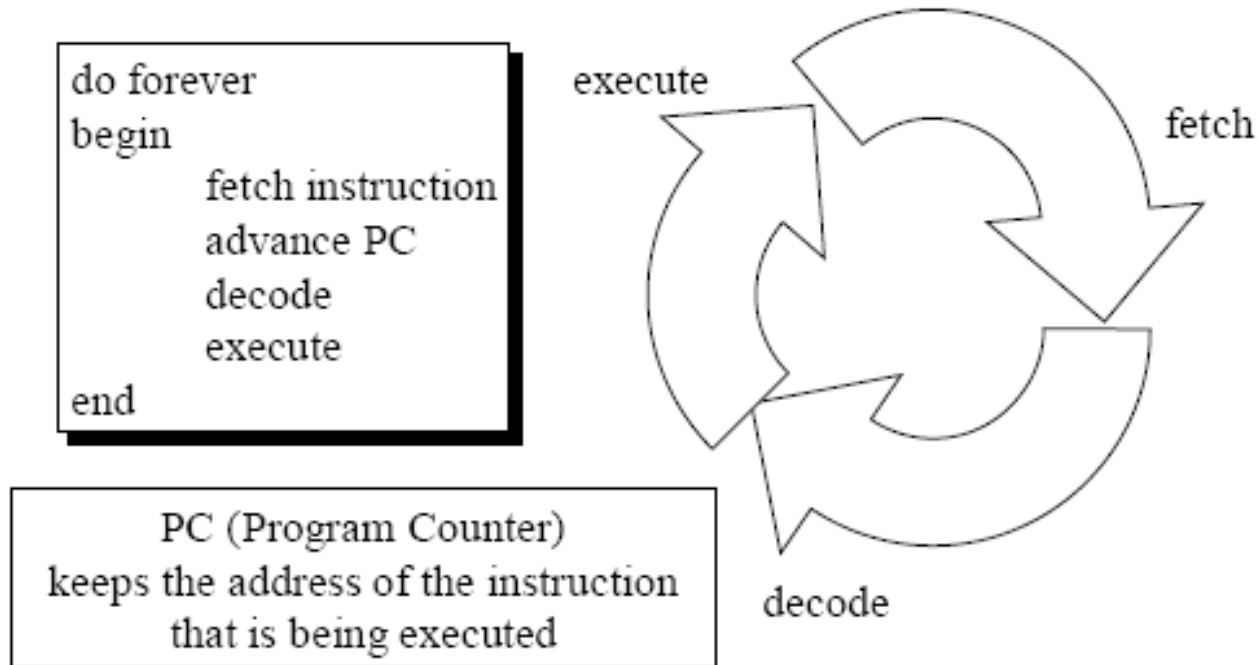
Chapter Objectives

Upon completion of this chapter, you will be able to:

- Explain the operation of each data movement instruction with applicable addressing modes.
- Select the appropriate assembly language instruction to accomplish a specific data movement task.

Instruction Cycle

Basic Instruction Cycle



Machine Language

- Native binary code microprocessor uses as its instructions to control its operation.
 - instructions vary in length from 1 to 13 bytes
- Over 100,000 variations of machine language instructions.
 - there is no complete list of these variations
- Some bits in a machine language instruction are given; remaining bits are determined for each variation of the instruction.

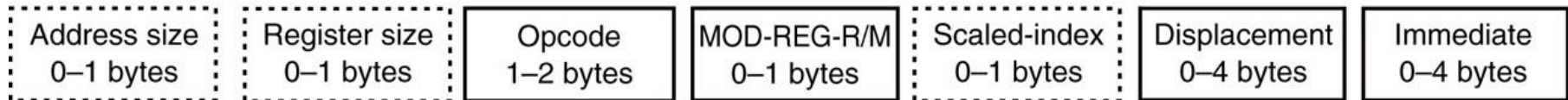
Instruction Format

16-bit instruction mode



(a)

32-bit instruction mode (80386 through Pentium 4 only)



(b)

Figure 4–1 The formats of the 8086–Core2 instructions. (a) The 16-bit form and (b) the 32-bit form.

The Opcode

- Selects the operation (addition, subtraction, etc.,) performed by the microprocessor.
- Figure below illustrates the general form of the first opcode byte of many instructions.
 - first 6 bits of the first byte are the binary opcode
 - remaining 2 bits indicate the **direction** (D) of the data flow, and indicate whether the data are a byte or a word (W)

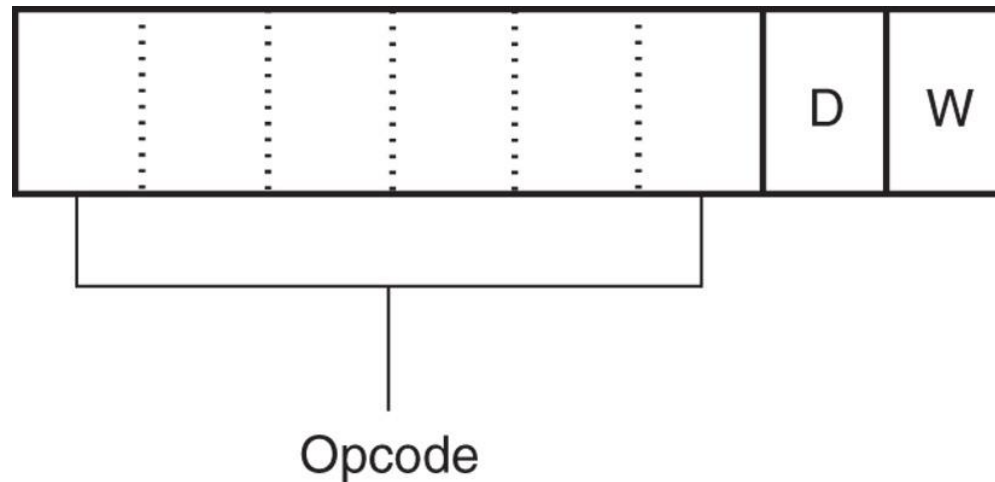


Figure 4–3 Byte 2 of many machine language instructions, showing the position of the MOD, REG, and R/M fields.



MOD Field

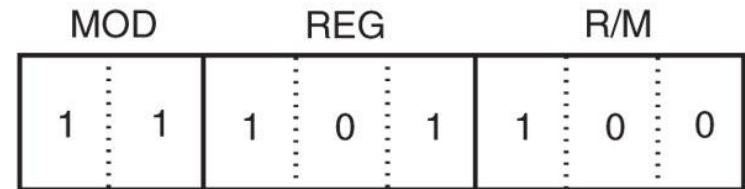
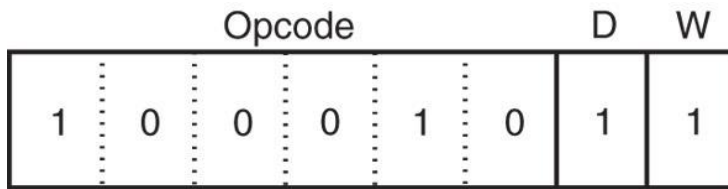
- Specifies addressing mode (MOD) and whether a displacement is present with the selected type.
 - If MOD=11, register-addressing mode is selected
 - Register addressing specifies a register instead of a memory location, using the R/M field
- If the MOD field contains a 00, 01, or 10, the R/M field selects one of the data memory-addressing modes.

- All 8-bit displacements are sign-extended into 16-bit displacements when the processor executes the instruction.
 - if the 8-bit displacement is 00H–7FH (positive), it is sign-extended to 0000H–007FH before adding to the offset address
 - if the 8-bit displacement is 80H–FFH (negative), it is sign-extended to FF80H–FFFFH

Register Assignments

- Suppose a 2-byte instruction, 8BECH, appears in a machine language program.
- In 16-bit mode, this instruction is converted to binary and placed in the instruction format of bytes 1 and 2, as illustrated in Figure 4–4.

Figure 4-4 The 8BEC instruction placed into bytes 1 and 2 formats from Figures 4-2 and 4-3. This instruction is a MOV BP,SP.



Opcode = MOV

D = Transfer to register (REG)

W = Word

MOD = R/M is a register

REG = BP

R/M = SP

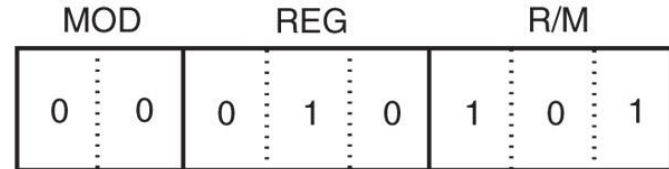
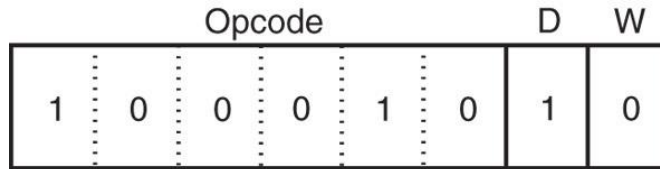
— the opcode is 100010, a MOV instruction

- D and W bits are a logic 1, so a word moves into the destination register specified in the REG field
- REG field contains 101, indicating register BP, so the MOV instruction moves data into register BP

R/M Memory Addressing

- If the MOD field contains a 00, 01, or 10, the R/M field takes on a new meaning.
- Figure 4–5 illustrates the machine language version of the 16-bit instruction MOV DL,[DI] or instruction (8A15H).
- This instruction is 2 bytes long and has an opcode 100010, D=1 (to REG from R/M), W=0 (byte), MOD=00 (no displacement), REG=010 (DL), and R/M=101 ([DI]).

Figure 4–5 A MOV DL,[DI] instruction converted to its machine language form.



Opcode = MOV

D = Transfer to register (REG)

W = Byte

MOD = No displacement

REG = DL

R/M = DS:[DI]

- If the instruction changes to MOV DL, [DI+1], the MOD field changes to 01 for 8-bit displacement
- first 2 bytes of the instruction remain the same
- instruction now becomes 8A5501H instead of 8A15H

- Because the MOD field contains a 11, the R/M field also indicates a register.
- R/M = 100(SP); therefore, this instruction moves data from SP into BP.
 - written in symbolic form as a MOV BP,SP instruction
- The assembler program keeps track of the register- and address-size prefixes and the mode of operation.

Data Types

- In order to tell the assembler about data type, these prefixes should be used:
 - **BYTE PTR** - for byte.
 - **WORD PTR** - for word (two bytes)
- Examples:
 - **MOV AL, BYTE PTR [BX]** ; byte access
 - **MOV CX, WORD PTR [BX]** ; word access
- Assembler supports shorter prefixes as well:
 - **B.** - for **BYTE PTR**
 - **W.** - for **WORD PTR**
- In certain cases the assembler can calculate the data type automatically.

MOV Instruction

- Copies the **second operand** (source) to the **first operand** (destination).
- The source operand can be an immediate value, general-purpose register or memory location.
- The destination register can be a general-purpose register, or memory location.
- Both operands must be the **same size**, which can be a **byte** or a **word**.
- the **MOV** instruction **cannot** be used to set the value of the **CS** and **IP** registers.

Operands of MOV

- These types of operands are supported:
 - MOV REG, memory
 - MOV memory, REG
 - MOV REG, REG
 - MOV memory, immediate
 - MOV REG, immediate
- **REG:** AX, BX, CX, DX, AH, AL, BL, BH, CH, CL, DH, DL, DI, SI, BP, SP.
- **memory:** [BX], [BX+SI+7], variable, etc.
- **immediate:** 5, -24, 3Fh, 10001101b, etc.

Segment Register Operands

- For segment registers only these types of **MOV** are supported:
 - MOV SREG, memory
 - MOV memory, SREG
 - MOV REG, SREG
 - MOV SREG, REG
- **SREG**: DS, ES, SS, and only as second operand: CS.
- **REG**: AX, BX, CX, DX, AH, AL, BL, BH, CH, CL, DH, DL, DI, SI, BP, SP.
- **memory**: [BX], [BX+SI+7], variable, etc.

MOV Example

ORG 100h	; this directive required
MOV AX, 0B800h	; set AX to hexadecimal value of B800h.
MOV DS, AX	; copy value of AX to DS.
MOV CL, 'A'	; set CL to ASCII code of 'A', it is 41h.
MOV CH, 11011111b	; set CH to binary value.
MOV BX, 15Eh	; set BX to 15Eh.
MOV [BX], CX	; copy contents of CX to memory at B800:015E
RET	; returns to operating system.

Variables

- Syntax for a variable declaration:
 - name **DB** value
 - name **DW** value
- **DB** - stays for Define Byte.
- **DW** - stays for Define Word.
- name - can be any letter or digit combination, though it should start with a letter. It's possible to declare unnamed variables by not specifying the name (this variable will have an address but no name).
- value - can be any numeric value in any supported numbering system (hexadecimal, binary, or decimal), or "?" symbol for variables that are not initialized.

Example

ORG 100h

MOV AL, var1

MOV BX, var2

RET ; stops the program.

var1 DB 7

var2 DW 1234H

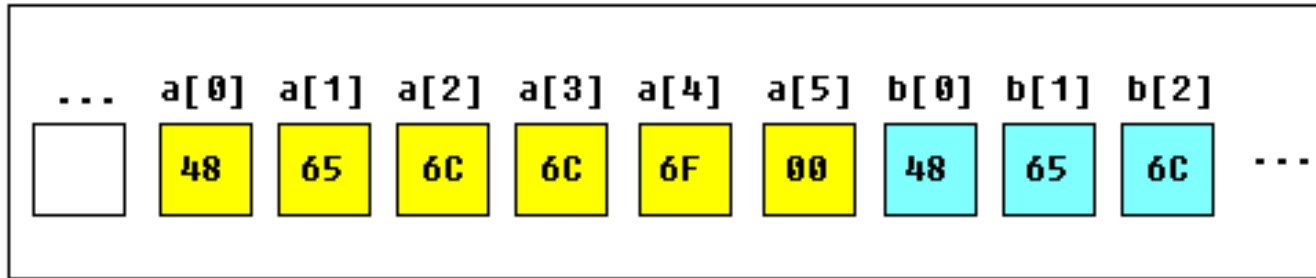
ORG Directive

- **ORG 100h** is an assembler directive (it tells assembler how to handle the source code).
- It tells assembler that the executable file will be loaded at the **offset** of 100h (256 bytes), so assembler should calculate the correct address for all variables when it replaces the variable names with their **offsets**.
- Directives are never converted to any real **machine code**.
- Why executable file is loaded at **offset** of **100h**?
- Operating system keeps some data about the program in the first 256 bytes of the **CS** (code segment), such as command line parameters and etc.

Arrays

- Arrays can be seen as chains of variables.
- A text string is an example of a byte array, each character is represented as an ASCII code value (0..255).
- Examples:
 - **a DB 48h, 65h, 6Ch, 6Ch, 6Fh, 00h**
 - **b DB 'Hello', 0**
- **b** is an exact copy of the **a** array, when assembler sees a string inside quotes it automatically converts it to set of bytes.

Accessing Array Elements



- You can access the value of any element in array using square brackets, for example:
 - MOV AL, a[3]
- You can also use any of the memory index registers **BX, SI, DI, BP**, for example:
 - MOV SI, 3
 - MOV AL, a[SI]

Declaring Large Arrays

- If you need to declare a large array you can use **DUP** operator.
- The syntax for **DUP**:
 - number DUP (value(s))
 - number - number of duplicate to make (any constant value).
 - value - expression that DUP will duplicate.
- Example:
c DB 5 DUP(9)
is an alternative way of declaring:
c DB 9, 9, 9, 9, 9

Declaring Large Arrays

- One more example:
d DB 5 DUP(1, 2)
is an alternative way of declaring:
d DB 1, 2, 1, 2, 1, 2, 1, 2, 1, 2
- Of course, you can use **DW** instead of **DB** if it's required to keep values larger than 255, or smaller than -128.
- **DW** cannot be used to declare strings.

Getting the Address of a Variable

- The **LEA** instruction can be used to get the offset address of a variable.
- Getting the address of the variable can be very useful in some situations, for example when you need to pass parameters to a procedure.

Example

ORG 100h

MOV AL, VAR1 ; check value of VAR1 by moving it to AL.

LEA BX, VAR1 ; get address of VAR1 in BX.

MOV BYTE PTR [BX], 44h ; modify the contents of VAR1.

MOV AL, VAR1 ; check value of VAR1 by moving it to AL.

RET

VAR1 DB 22h

END

Constants

- Constants are just like variables, but they exist only until your program is compiled (assembled).
- After definition of a constant its value cannot be changed.
- To define constants **EQU** directive is used:
name EQU < any expression >
- Example:
k EQU 5
MOV AX, k
- The above example is functionally identical to code:
MOV AX, 5

PUSH

- Always transfers 2 bytes of data to the stack;
 - 80386 and above transfer 2 or 4 bytes
- PUSH instruction copies contents of the internal register set, except the segment registers, to the stack.
- PUSH (push all) instruction copies the registers to the stack in the following order: AX, CX, DX, BX, SP, BP, SI, and DI.

- PUSHF (**push flags**) instruction copies the contents of the flag register to the stack.
- PUSHAD and POPAD instructions push and pop the contents of the 32-bit register set in 80386 - Pentium 4.
 - PUSHA and POPA instructions do not function in the 64-bit mode of operation for the Pentium 4

POP

- Performs the inverse operation of PUSH.
- POP removes data from the stack and places it in a target 16-bit register, segment register, or a 16-bit memory location.
 - not available as an immediate POP
- POPF (pop flags) removes a 16-bit number from the stack and places it in the flag register;
 - POPFD removes a 32-bit number from the stack and places it into the extended flag register

- POPA (pop all) removes 16 bytes of data from the stack and places them into the following registers, in the order shown: DI, SI, BP, SP, BX, DX, CX, and AX.
 - reverse order from placement on the stack by PUSHHA instruction, causing the same data to return to the same registers

4-4 STRING DATA TRANSFERS

- Before the string instructions are presented, the operation of the D flag-bit (direction), DI, and SI must be understood as they apply to the string instructions.

The Direction Flag

- The direction flag (D, located in the flag register) selects auto-increment or auto-decrement operation for the DI and SI registers during string operations.
 - used only with the string instructions
- The CLD instruction clears the D flag and the STD instruction sets it .
 - CLD instruction selects the auto-increment mode and STD selects the auto-decrement mode

DI and SI

- During execution of string instruction, memory accesses occur through DI and SI registers.
 - DI offset address accesses data in the extra segment for all string instructions that use it
 - SI offset address accesses data by default in the data segment

LODSB Instruction

- Load byte at DS:[SI] into AL. Update SI.

Algorithm:

AL = DS:[SI]

- if DF = 0 then
 - SI = SI + 1
- else
 - SI = SI - 1

Example

ORG 100h

LEA SI, a1

MOV CX, 5

MOV AH, 0Eh

m: LODSB

INT 10h

LOOP m

RET

a1 DB 'H', 'e', 'l', 'l', 'o'

STOSB Instruction

- Store byte in AL into ES:[DI]. Update DI.

Algorithm:

ES:[DI] = AL

- if DF = 0 then
 - DI = DI + 1
- else
 - DI = DI - 1

Example

ORG 100h

LEA DI, a1

MOV AL, 12h

MOV CX, 5

REP

STOSB

RET

a1 DB 5 dup(0)

MOVSb Instruction

- Copy byte at DS:[SI] to ES:[DI]. Update SI and DI.

Algorithm:

$ES:[DI] = DS:[SI]$

- if DF = 0 then
 - $SI = SI + 1$
 - $DI = DI + 1$
- else
 - $SI = SI - 1$
 - $DI = DI - 1$

Example

ORG 100h

CLD

LEA SI, a1

LEA DI, a2

MOV CX, 5

REP

MOVSB

RET

a1 DB 1,2,3,4,5

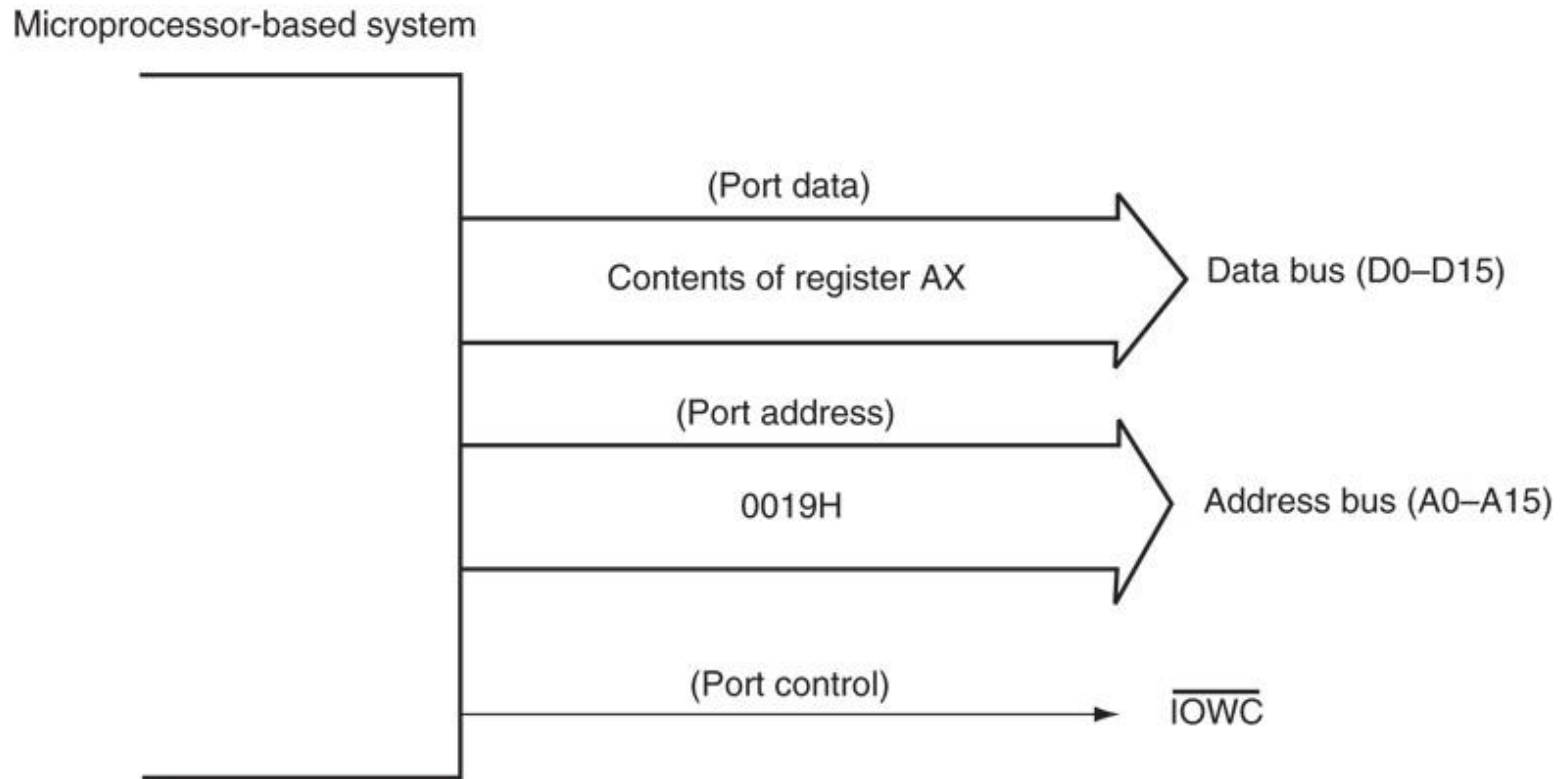
a2 DB 5 DUP(0)

IN and OUT

- IN & OUT instructions perform I/O operations.
- Contents of AL, AX, or EAX are transferred only between I/O device and microprocessor.
 - an IN instruction transfers data from an external I/O device into AL, AX, or EAX
 - an OUT transfers data from AL, AX, or EAX to an external I/O device

- Two forms of I/O device (port) addressing:
- *Fixed-port addressing* allows data transfer between AL, AX, or EAX using an 8-bit I/O port address.
 - port number follows the instruction's opcode
- *Variable-port addressing* allows data transfers between AL, AX, or EAX and a 16-bit port address.
 - the I/O port number is stored in register DX, which can be changed (varied) during the execution of a program.

Figure 4–20 The signals found in the microprocessor-based system for an OUT 19H,AX instruction.





The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Arithmetic and Logic Instructions

Introduction

- We examine the arithmetic and logic instructions. The arithmetic instructions include addition, subtraction, multiplication, division, comparison, negation, increment, and decrement.
- The logic instructions include AND, OR, Exclusive-OR, NOT, shifts, rotates, and the logical compare (TEST).

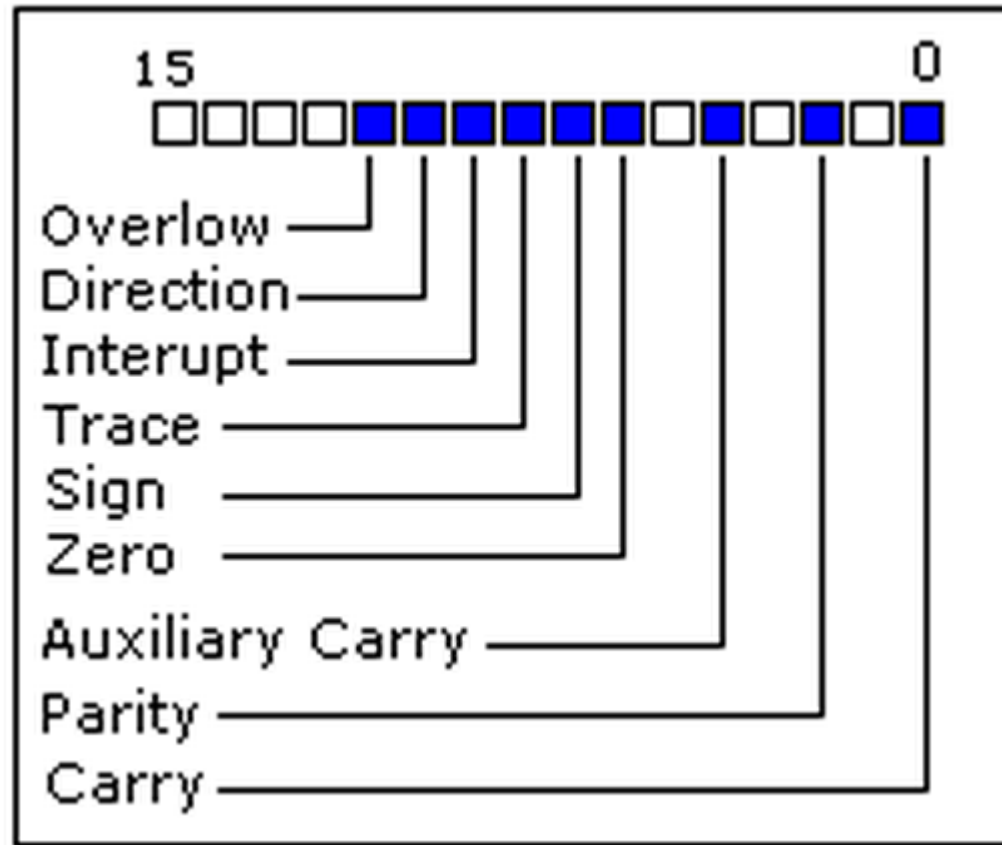
Chapter Objectives

Upon completion of this chapter, you will be able to:

- Use arithmetic instructions to accomplish simple binary, BCD, and ASCII arithmetic.
- Use logic instructions to accomplish binary bit manipulation.
- Use the shift and rotate instructions.

Arithmetic and Logic Instructions

- Most arithmetic instructions affect the **FLAGS** register.



Arithmetic and Logic Instructions

- **Carry Flag (C)** - this flag is set to **1** when there is an **unsigned overflow**. For example when you add bytes **255 + 1** (result is not in range 0...255). When there is no overflow this flag is set to **0**.
- **Zero Flag (Z)** - set to **1** when result is **zero**. For none-zero result this flag is set to **0**.
- **Sign Flag (S)** - set to **1** when result is **negative**. When result is **positive** it is set to **0**. Actually this flag take the value of the most significant bit.
- **Overflow Flag (O)** - set to **1** when there is a **signed overflow**. For example, when you add bytes **100 + 50** (result is not in range -128...127).

Arithmetic and Logic Instructions

- **Parity Flag (P)** - this flag is set to **1** when there is even number of one bits in result, and to **0** when there is odd number of one bits. Even if result is a word only 8 low bits are analyzed!
- **Auxiliary Flag (A)** - set to **1** when there is an **unsigned overflow** for low nibble (4 bits).
- **Interrupt enable Flag (I)** - when this flag is set to **1** CPU reacts to interrupts from external devices.
- **Direction Flag (D)** - this flag is used by some instructions to process data chains, when this flag is set to **0** - the processing is done forward, when this flag is set to **1** the processing is done backward.

ADDITION, SUBTRACTION AND COMPARISON

- The bulk of the arithmetic instructions found in any microprocessor include addition, subtraction, and comparison.
- Addition, subtraction, and comparison instructions are illustrated.
- Also shown are their uses in manipulating register and memory data.

Addition

- Addition (ADD) appears in many forms in the microprocessor.
- A second form of addition, called **add-with-carry**, is introduced with the ADC instruction.
- The only types of addition *not* allowed are memory-to-memory and segment register.
 - segment registers can only be moved, pushed, or popped.
- Increment instruction (INC) is a special type of addition that adds 1 to a number.

- Any ADD instruction modifies the contents of the sign, zero, carry, auxiliary carry, parity, and overflow flags.

ADD

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Add.

Algorithm:

$\text{operand1} = \text{operand1} + \text{operand2}$

Example:

MOV AL, 5 ; AL = 5

ADD AL, -3 ; AL = 2

RET

C	Z	S	O	P	A
r	r	r	r	r	r

INC

REG
memory

Increment.

Algorithm:

$\text{operand} = \text{operand} + 1$

Example:

MOV AL, 4

INC AL ; AL = 5

RET

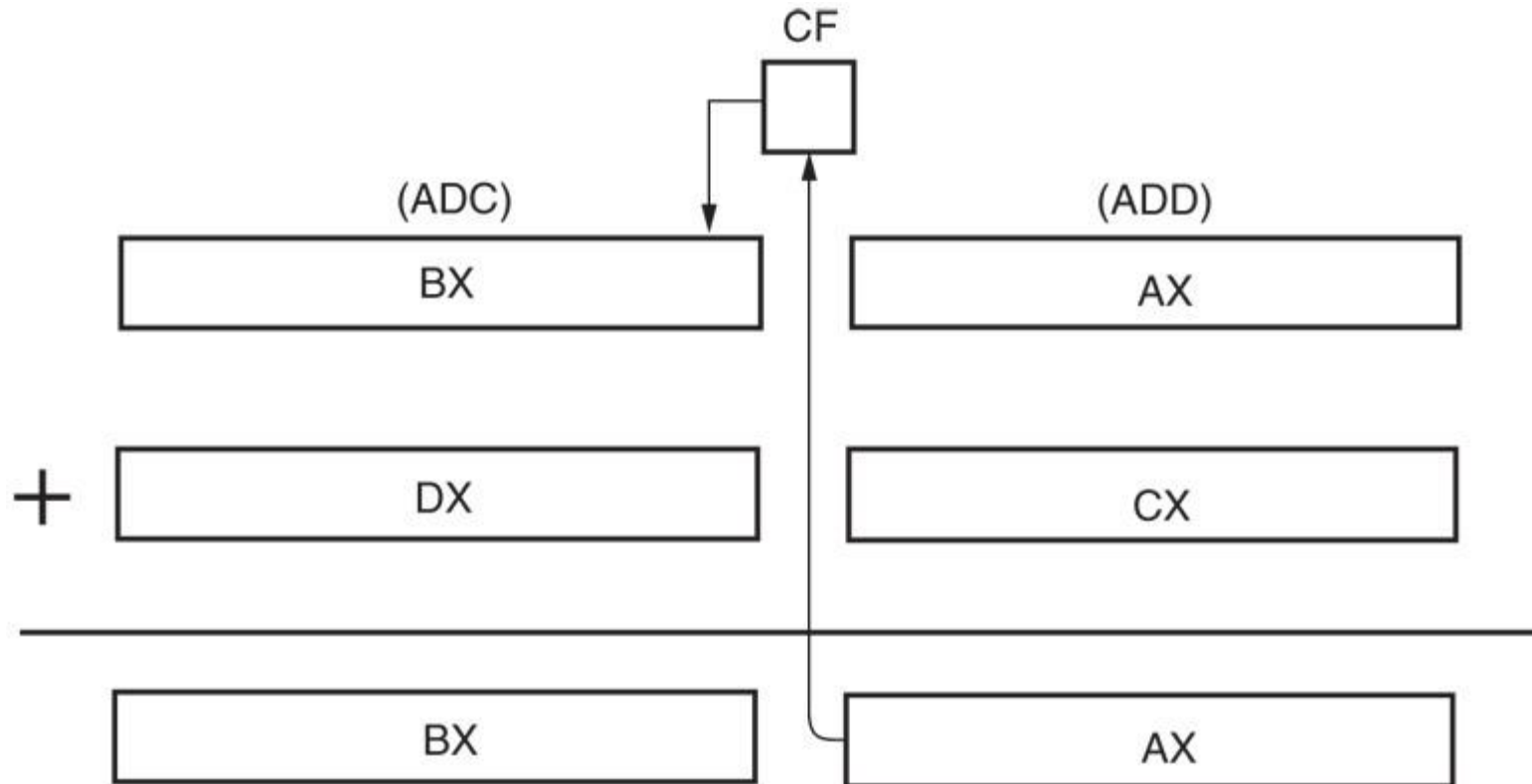
Z	S	O	P	A
r	r	r	r	r

CF - unchanged!

Addition-with-Carry

- ADC) adds the bit in the carry flag (C) to the operand data.
 - mainly appears in software that adds numbers wider than 16 or 32 bits.
 - like ADD, ADC affects the flags after the addition
- Figure 5–1 illustrates this so placement and function of the carry flag can be understood.
 - cannot be easily performed without adding the carry flag bit because the 8086–80286 only adds 8- or 16-bit numbers

Figure 5–1 Addition-with-carry showing how the carry flag (C) links the two 16-bit additions into one 32-bit addition.



ADC

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

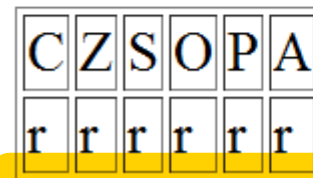
Add with Carry.

Algorithm:

$\text{operand1} = \text{operand1} + \text{operand2} + \text{CF}$

Example:

STC ; set CF = 1
MOV AL, 5 ; AL = 5
ADC AL, 1 ; AL = 7
RET



Subtraction

- Many forms of subtraction (SUB) appear in the instruction set.
 - these use any addressing mode with 8-, 16-, or 32-bit data
 - a special form of subtraction (decrement, or DEC) subtracts 1 from any register or memory location
- After each subtraction, the microprocessor modifies the contents of the flag register.
 - flags change for most arithmetic/logic operations

SUB

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Subtract.

Algorithm:

$\text{operand1} = \text{operand1} - \text{operand2}$

Example:

MOV AL, 5

SUB AL, 1 ; AL = 4

RET

C	Z	S	O	P	A
r	r	r	r	r	r

DEC

REG
memory

Decrement.

Algorithm:

$\text{operand} = \text{operand} - 1$

Example:

MOV AL, 255 ; AL = 0FFh (255 or -1)

DEC AL ; AL = 0FEh (254 or -2)

RET

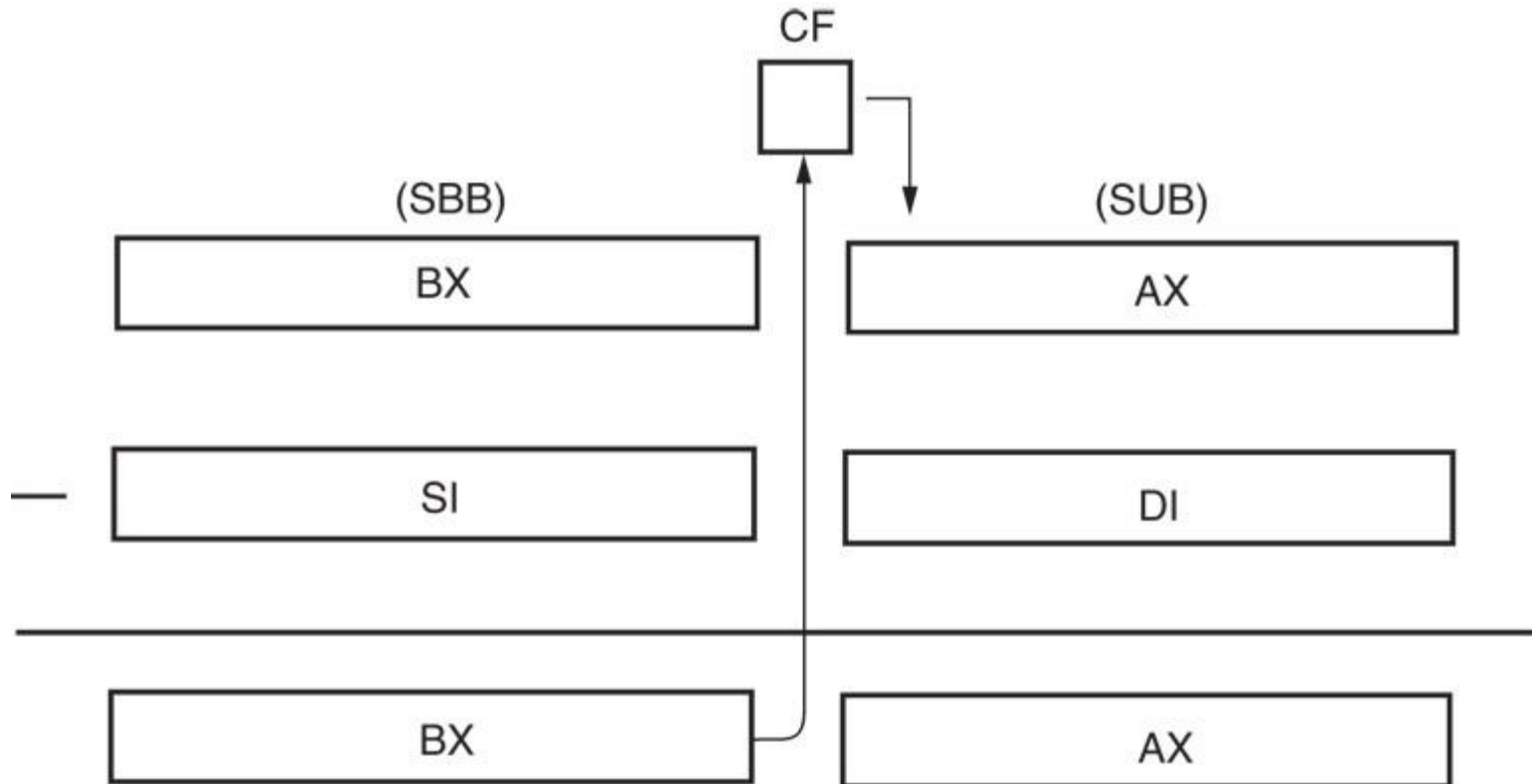
Z	S	O	P	A
r	r	r	r	r

CF - unchanged!

Subtraction-with-Borrow

- A subtraction-with-borrow (SBB) instruction functions as a regular subtraction, except that the carry flag (C), which holds the borrow, also subtracts from the difference.
 - most common use is subtractions wider than 16 bits in the 8086–80286 microprocessors or wider than 32 bits in the 80386–Core2.
 - wide subtractions require borrows to propagate through the subtraction, just as wide additions propagate the carry

Figure 5–2 Subtraction-with-borrow showing how the carry flag (C) propagates the borrow.



SBB

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Subtract with Borrow.

Algorithm:

$\text{operand1} = \text{operand1} - \text{operand2} - \text{CF}$

Example:

STC

MOV AL, 5

SBB AL, 3 ; $\text{AL} = 5 - 3 - 1 = 1$

RET

C	Z	S	O	P	A
r	r	r	r	r	r

Comparison

- The comparison instruction (CMP) is a subtraction that changes only the flag bits.
 - destination operand never changes
- Useful for checking the contents of a register or a memory location against another value.
- A CMP is normally followed by a conditional jump instruction, which tests the condition of the flag bits.

CMP

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Compare.

Algorithm:

operand1 - operand2

result is not stored anywhere, flags are set
(OF, SF, ZF, AF, PF, CF) according to result.

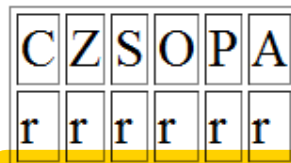
Example:

MOV AL, 5

MOV BL, 5

CMP AL, BL ; AL = 5, ZF = 1 (so equal!)

RET



MULTIPLICATION AND DIVISION

- Earlier 8-bit microprocessors could not multiply or divide without the use of a program that multiplied or divided by using a series of shifts and additions or subtractions.
 - manufacturers were aware of this inadequacy, they incorporated multiplication and division into the instruction sets of newer microprocessors.
- Pentium–Core2 contains special circuitry to do multiplication in as few as one clocking period.
 - over 40 clocking periods in earlier processors

Multiplication

- The multiplication instruction contains one operand because it always multiplies the operand times the contents of register AL/AX.
- Performed on bytes, words, or doublewords,
 - can be signed (IMUL) or unsigned integer (MUL)
- Product after a multiplication always a double-width product.
 - two 8-bit numbers multiplied generate a 16-bit product; two 16-bit numbers generate a 32-bit; two 32-bit numbers generate a 64-bit product

MUL

REG
memory

Unsigned multiply.

Algorithm:

when operand is a **byte**:

$AX = AL * \text{operand}$.

when operand is a **word**:

$(DX\ AX) = AX * \text{operand}$.

Example:

MOV AL, 200 ; AL = 0C8h

MOV BL, 4

MUL BL ; AX = 0320h (800)

RET

C	Z	S	O	P	A
r	?	?	r	?	?

CF=OF=0 when high section of the result is zero.

IMUL

REG
memory

Signed multiply.

Algorithm:

when operand is a **byte**:

$AX = AL * \text{operand}$.

when operand is a **word**:

$(DX\ AX) = AX * \text{operand}$.

Example:

MOV AL, -2

MOV BL, -4

IMUL BL ; AX = 8

RET

C	Z	S	O	P	A
r	?	?	r	?	?

CF=OF=0 when result fits into operand of IMUL.

Division

- Occurs on 8- or 16-bit and 32-bit numbers depending on microprocessor.
 - signed (IDIV) or unsigned (DIV) integers
- Dividend is always a double-width dividend, divided by the operand.
- There is no immediate division instruction available to any microprocessor.

- A division can result in two types of errors:
 - attempt to divide by zero
 - other is a divide overflow, which occurs when a small number divides into a large number
- In either case, the microprocessor generates an interrupt if a divide error occurs.
- In most systems, a divide error interrupt displays an error message on the video screen.

The Remainder

- Could be used to round the quotient or dropped to truncate the quotient.
- If division is unsigned, rounding requires the remainder be compared with half the divisor to decide whether to round up the quotient
- The remainder could also be converted to a fractional remainder.

DIV

REG
memory

Unsigned divide.

Algorithm:

when operand is a **byte**:

$AL = AX / \text{operand}$

$AH = \text{remainder (modulus)}$

when operand is a **word**:

$AX = (DX\ AX) / \text{operand}$

$DX = \text{remainder (modulus)}$

Example:

MOV AX, 203 ; AX = 00CBh

MOV BL, 4

DIV BL ; AL = 50 (32h), AH = 3

RET

C	Z	S	O	P	A
?	?	?	?	?	?

IDIV

REG
memory

Signed divide.

Algorithm:

when operand is a **byte**:

$AL = AX / \text{operand}$

$AH = \text{remainder (modulus)}$

when operand is a **word**:

$AX = (DX\ AX) / \text{operand}$

$DX = \text{remainder (modulus)}$

Example:

MOV AX, -203 ; $AX = 0FF35h$

MOV BL, 4

IDIV BL ; $AL = -50 (0CEh)$, $AH = -3 (0FDh)$

RET

C	Z	S	O	P	A
?	?	?	?	?	?

BCD and ASCII Arithmetic

- The microprocessor allows arithmetic manipulation of both BCD (**binary-coded decimal**) and ASCII (**American Standard Code for Information Interchange**) data.
- BCD operations occur in systems such as point-of-sales terminals (e.g., cash registers) and others that seldom require complex arithmetic.

BCD Arithmetic

- Two arithmetic techniques operate with BCD data: addition and subtraction.
- DAA (**decimal adjust after addition**) instruction follows BCD addition,
- DAS (**decimal adjust after subtraction**) follows BCD subtraction.
 - both correct the result of addition or subtraction so it is a BCD number

DAA Instruction

- DAA follows the ADD or ADC instruction to adjust the result into a BCD result.
- After adding the BL and DL registers, the result is adjusted with a DAA instruction before being stored in CL.

DAS Instruction

- Functions as does DAA instruction, except it follows a subtraction instead of an addition.

DAA

No operands

Decimal adjust After Addition.

Corrects the result of addition of two packed BCD values.

Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL + 6
- AF = 1

if AL > 9Fh or CF = 1 then:

- AL = AL + 60h
- CF = 1

Example:

MOV AL, 0Fh ; AL = 0Fh (15)

DAA ; AL = 15h

RET

C	Z	S	O	P	A
r	r	r	r	r	r

DAS

No operands

Decimal adjust After Subtraction.
Corrects the result of subtraction of two packed BCD values.

Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL - 6
- AF = 1

if AL > 9Fh or CF = 1 then:

- AL = AL - 60h
- CF = 1

Example:

```
MOV AL, 0FFh ; AL = 0FFh (-1)
DAS          ; AL = 99h, CF = 1
RET
```

C	Z	S	O	P	A
r	r	r	r	r	r

ASCII Arithmetic

- ASCII arithmetic instructions function with coded numbers, value 30H to 39H for 0–9.
- Four instructions in ASCII arithmetic operations:
 - AAA (**ASCII adjust after addition**)
 - AAD (**ASCII adjust before division**)
 - AAM (**ASCII adjust after multiplication**)
 - AAS (**ASCII adjust after subtraction**)
- These instructions use register AX as the source and as the destination.

AAA Instruction

- Addition of two one-digit ASCII-coded numbers will not result in any useful data.

AAD Instruction

- Appears before a division.
- The AAD instruction requires the AX register contain a two-digit unpacked BCD number (not ASCII) before executing.

AAM Instruction

- Follows multiplication instruction after multiplying two one-digit unpacked BCD numbers.
- AAM converts from binary to unpacked BCD.
- If a binary number between 0000H and 0063H appears in AX, AAM converts it to BCD.

AAS Instruction

- AAS adjusts the AX register after an ASCII subtraction.

BASIC LOGIC INSTRUCTIONS

- Include AND, OR, Exclusive-OR, and NOT.
 - also TEST, a special form of the AND instruction
 - NEG, similar to the NOT instruction
- Logic operations provide binary bit control in low-level software.
 - allow bits to be set, cleared, or complemented
- Low-level software appears in machine language or assembly language form and often controls the I/O devices in a system.

- All logic instructions affect the flag bits.
- Logic operations always clear the carry and overflow flags
 - other flags change to reflect the result
- When binary data are manipulated in a register or a memory location, the rightmost bit position is always numbered bit 0.
 - position numbers increase from bit 0 to the left, to bit 7 for a byte, and to bit 15 for a word
 - a doubleword (32 bits) uses bit position 31 as its leftmost bit and a quadword (64-bits) position 63

AND

- Performs logical multiplication, illustrated by a truth table.
- AND can replace discrete AND gates if the speed required is not too great
 - normally reserved for embedded control applications
- In 8086, the AND instruction often executes in about a microsecond.
 - with newer versions, the execution speed is greatly increased

Figure 5–3 (a) The truth table for the AND operation and (b) the logic symbol of an AND gate.

A	B	T
0	0	0
0	1	0
1	0	0
1	1	1

(a)



(b)

- AND clears bits of a binary number.
 - called **masking**
- AND uses any mode except memory-to-memory and segment register addressing.
- An ASCII number can be converted to BCD by using AND to mask off the leftmost four binary bit positions.

Figure 5–4 The operation of the AND function showing how bits of a number are cleared to zero.

x x x x	x x x x	Unknown number
• 0 0 0 0	1 1 1 1	Mask
0 0 0 0	x x x x	Result
Her Türlü sıfır gelicek high nibble maskladık		

```
MOV  BX, 3135H      ;load ASCII
AND   BX, 0F0FH      ;mask BX
```

AND

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Logical AND between all bits of two operands.
Result is stored in operand1.

These rules apply:

1 AND 1 = 1

1 AND 0 = 0

0 AND 1 = 0

0 AND 0 = 0

Example:

MOV AL, 'a' ; AL = 01100001b

AND AL, 11011111b ; AL = 01000001b ('A')

RET

C	Z	S	O	P
0	r	r	0	r

Examples

TABLE 5–16 Example AND instructions.

<i>Assembly Language</i>	<i>Operation</i>
AND AL,BL	AL = AL and BL
AND CX,DX	CX = CX and DX
AND ECX,EDI	ECX = ECX and EDI
AND RDX,RBP	RDX = RDX and RBP (64-bit mode)
AND CL,33H	CL = CL and 33H
AND DI,4FFFH	DI = DI and 4FFFH
AND ESI,34H	ESI = ESI and 34H
AND RAX,1	RAX = RAX and 1 (64-bit mode)
AND AX,[DI]	The word contents of the data segment memory location addressed by DI are ANDed with AX
AND ARRAY[SI],AL	The byte contents of the data segment memory location addressed by ARRAY plus SI are ANDed with AL
AND [EAX],CL	CL is ANDed with the byte contents of the data segment memory location addressed by ECX

OR

- Performs logical addition
 - often called the *Inclusive-OR* function
- The OR function generates a logic 1 output if any inputs are 1.
 - a 0 appears at output only when all inputs are 0
- Figure 5–6 shows how the OR gate sets (1) any bit of a binary number.
- The OR instruction uses any addressing mode except segment register addressing.

Figure 5–5 (a) The truth table for the OR operation and (b) the logic symbol of an OR gate.

A	B	T
0	0	0
0	1	1
1	0	1
1	1	1

(a)



(b)

Figure 5–6 The operation of the OR function showing how bits of a number are set to one.

x x x x	x x x x	Unknown number
+ 0 0 0 0	1 1 1 1	Mask
x x x x	1 1 1 1	Result

1 ler gereksiz o nedenle 1 ile maskledik

```
MOV    AL, 5           ;load data
MOV    BL, 7
MUL    BL
AAM                    ;adjust
OR     AX, 3030H       ;convert to ASCII
```

OR

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Logical OR between all bits of two operands.
Result is stored in first operand.

These rules apply:

1 OR 1 = 1

1 OR 0 = 1

0 OR 1 = 1

0 OR 0 = 0

Example:

MOV AL, 'A' ; AL = 01000001b

OR AL, 00100000b ; AL = 01100001b ('a')

RET

C	Z	S	O	P	A
0	r	r	0	r	?

Examples

TABLE 5–17 Example OR instructions.

<i>Assembly Language</i>	<i>Operation</i>
OR AH,BL	AL = AL or BL
OR SI,DX	SI = SI or DX
OR EAX,EBX	EAX = EAX or EBX
OR R9,R10	R9 = R9 or R10 (64-bit mode)
OR DH,0A3H	DH = DH or 0A3H
OR SP,990DH	SP = SP or 990DH
OR EBP,10	EBP = EBP or 10
OR RBP,1000H	RBP = RBP or 1000H (64-bit mode)
OR DX,[BX]	DX is ORed with the word contents of data segment memory location addressed by BX
OR DATES[DI + 2],AL	The byte contents of the data segment memory location addressed by DI plus 2 are ORed with AL

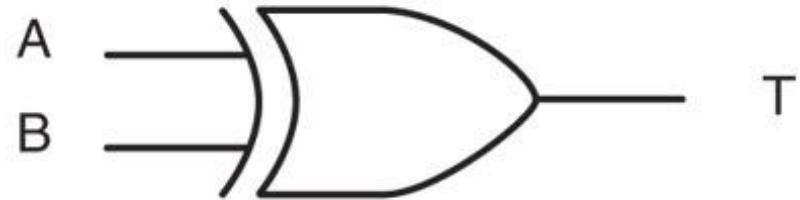
Exclusive-OR

- Differs from Inclusive-OR (OR) in that the 1,1 condition of Exclusive-OR produces a 0.
 - a 1,1 condition of the OR function produces a 1
- The Exclusive-OR operation *excludes* this condition; the Inclusive-OR *includes* it.
- If inputs of the Exclusive-OR function are both 0 or both 1, the output is 0; if the inputs are different, the output is 1.
- Exclusive-OR is sometimes called a comparator.

Figure 5–7 (a) The truth table for the Exclusive-OR operation and (b) the logic symbol of an Exclusive-OR gate.

A	B	T
0	0	0
0	1	1
1	0	1
1	1	0

(a)



(b)

- XOR uses any addressing mode except segment register addressing.
- Exclusive-OR is useful if some bits of a register or memory location must be inverted.
- Figure 5–8 shows how just part of an unknown quantity can be inverted by XOR.
 - when a 1 Exclusive-ORs with X, the result is X
 - if a 0 Exclusive-ORs with X, the result is X
- A common use for the Exclusive-OR instruction is to clear a register to zero

Figure 5–8 The operation of the Exclusive-OR function showing how bits of a number are inverted.

x x x x	x x x x	Unknown number
⊕ 0 0 0 0	1 1 1 1	Mask
x x x x	$\bar{x} \bar{x} \bar{x} \bar{x}$	Result

Sıfırlama Yöntemi

OR	CX, 0600H	;set bits 9 and 10
AND	CX, 0FFFCH	;clear bits 0 and 1
XOR	CX, 1000H	;invert bit 12

XOR

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Logical XOR (Exclusive OR) between all bits of two operands. Result is stored in first operand.

These rules apply:

1 XOR 1 = 0

1 XOR 0 = 1

0 XOR 1 = 1

0 XOR 0 = 0

Example:

MOV AL, 00000111b

XOR AL, 00000010b ; AL = 00000101b

RET

C	Z	S	O	P	A
0	r	r	0	r	?

Examples

TABLE 5–18 Example Exclusive-OR instructions.

<i>Assembly Language</i>	<i>Operation</i>
XOR CH,DL	CH = CH xor DL
XOR SI,BX	SI = SI xor BX
XOR EBX,EDI	EBX = EBX xor EDI
XOR RAX,RBX	RAX = RAX xor RBX (64-bit mode)
XOR AH,0EEH	AH = AH xor 0EEH
XOR DI,00DDH	DI = DI xor 00DDH
XOR ESI,100	ESI = ESI xor 100
XOR R12,20	R12 = R12 xor 20 (64-bit mode)
XOR DX,[SI]	DX is Exclusive-ORed with the word contents of the data segment memory location addressed by SI
XOR DEAL[BP+2],AH	AH is Exclusive-ORed with the byte contents of the stack segment memory location addressed by BP plus 2

Test and Bit Test Instructions

- **TEST** performs the AND operation.
 - only affects the condition of the flag register, which indicates the result of the test
 - functions the same manner as a CMP
- Usually followed by either the **JZ** (jump if zero) or **JNZ** (jump if not zero) instruction.
- The destination operand is normally tested against immediate data.

```
TEST AL,1           ;test right bit
JNZ  RIGHT          ;if set
TEST AL,128         ;test left bit
JNZ  LEFT           ;if set
```

TEST

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Logical AND between all bits of two operands for flags only. These flags are effected: **ZF, SF, PF**. Result is not stored anywhere.

These rules apply:

1 AND 1 = 1

1 AND 0 = 0

0 AND 1 = 0

0 AND 0 = 0

Example:

MOV AL, 00000101b

TEST AL, 1 ; ZF = 0.

TEST AL, 10b ; ZF = 1.

RET

C	Z	S	O	P
0	r	r	0	r

Examples

TABLE 5–19 Example
TEST instructions.

<i>Assembly Language</i>	<i>Operation</i>
TEST DL,DH	DL is ANDed with DH
TEST CX,BX	CX is ANDed with BX
TEST EDX,ECX	EDX is ANDed with ECX
TEST RDX,R15	RDX is ANDed with R15 (64-bit mode)
TEST AH,4	AH is ANDed with 4
TEST EAX,256	EAX is ANDed with 256

NOT and NEG

- NOT and NEG can use any addressing mode except segment register addressing.
- The NOT instruction inverts all bits of a byte, word, or doubleword.
- NEG two's complements a number.
 - the arithmetic sign of a signed number changes from positive to negative or negative to positive
- The NOT function is considered logical, NEG function is considered an arithmetic operation.

NOT

REG
memory

Invert each bit of the operand.

Algorithm:

- if bit is 1 turn it to 0.
- if bit is 0 turn it to 1.

Example:

MOV AL, 00011011b

NOT AL ; AL = 11100100b

RET

C	Z	S	O	P	A
unchanged					

NEG

REG
memory

Negate. Makes operand negative (two's complement).

Algorithm:

- Invert all bits of the operand
- Add 1 to inverted operand

Example:

MOV AL, 5 ; AL = 05h

NEG AL ; AL = 0FBh (-5)

NEG AL ; AL = 05h (5)

RET

C	Z	S	O	P	A
r	r	r	r	r	r

Examples

TABLE 5–21 Example NOT and NEG instructions.

<i>Assembly Language</i>	<i>Operation</i>
NOT CH	CH is one's complemented
NEG CH	CH is two's complemented
NEG AX	AX is two's complemented
NOT EBX	EBX is one's complemented
NEG ECX	ECX is two's complemented
NOT RAX	RAX is one's complemented (64-bit mode)
NOT TEMP	The contents of data segment memory location TEMP is one's complemented
NOT BYTE PTR[BX]	The byte contents of the data segment memory location addressed by BX are one's complemented

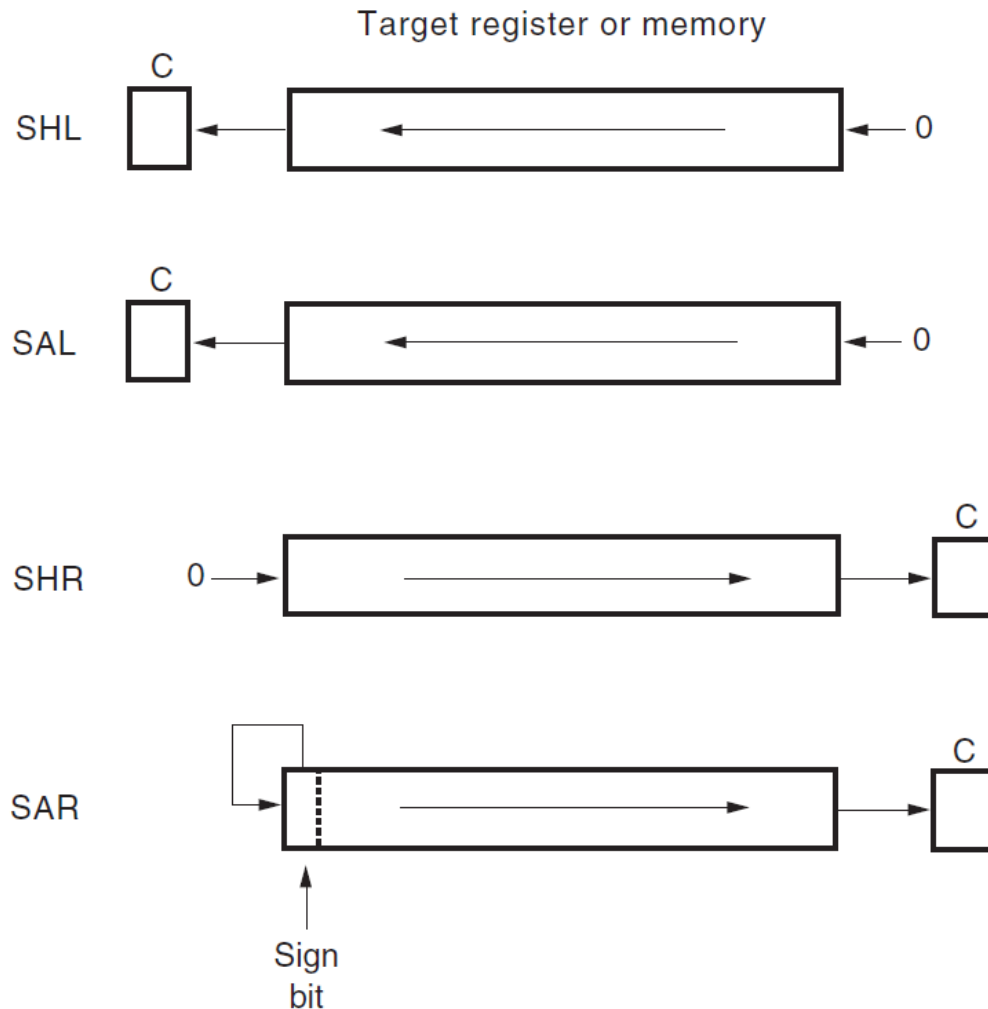
Shift and Rotate

- Shift and rotate instructions manipulate binary numbers at the binary bit level.
 - as did AND, OR, Exclusive-OR, and NOT
- Common applications in low-level software used to control I/O devices.
- The microprocessor contains a complete complement of shift and rotate instructions that are used to shift or rotate any memory data or register.

Shift

- Position or move numbers to the left or right within a register or memory location.
 - also perform simple arithmetic as multiplication by powers of 2^{+n} (left shift) and division by powers of 2^{-n} (right shift).
- The microprocessor's instruction set contains four different shift instructions:
 - two are logical; two are arithmetic shifts
- All four shift operations appear in Figure 5–9.

Figure 5–9 The shift instructions showing the operation and direction of the shift.



- logical shifts move 0 in the rightmost bit for a logical left shift;
- 0 to the leftmost bit position for a logical right shift
- arithmetic right shift copies the sign-bit through the number
- logical right shift copies a 0 through the number.

Logical shifts multiply or divide unsigned data; arithmetic shifts multiply or divide signed data.

- a shift left always multiplies by 2 for each bit position shifted
- a shift right always divides by 2 for each position
- shifting a two places, multiplies or divides by 4

```
SHL DX,14
```

or

```
MOV CL,14  
SHL DX,CL
```

```
;Multiply AX by 10 (1010)  
;  
    SHL  AX,1           ;AX times 2  
    MOV  BX,AX  
    SHL  AX,2           ;AX times 8  
    ADD  AX,BX          ;AX times 10  
;  
;Multiply AX by 18 (10010)  
;  
    SHL  AX,1           ;AX times 2  
    MOV  BX,AX  
    SHL  AX,3           ;AX times 16  
    ADD  AX,BX          ;AX times 18  
;  
;Multiply AX by 5 (101)  
;  
    MOV  BX,AX  
    SHL  AX,2           ;AX times 4  
    ADD  AX,BX          ;AX times 5
```

SHL

memory, immediate
REG, immediate

memory, CL
REG, CL

Shift operand1 Left. The number of shifts is set by operand2.

Algorithm:

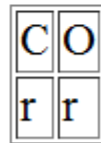
- Shift all bits left, the bit that goes off is set to CF.
- Zero bit is inserted to the right-most position.

Example:

MOV AL, 11100000b

SHL AL, 1 ; AL = 11000000b, CF=1.

RET



OF=0 if first operand keeps original sign.

Examples

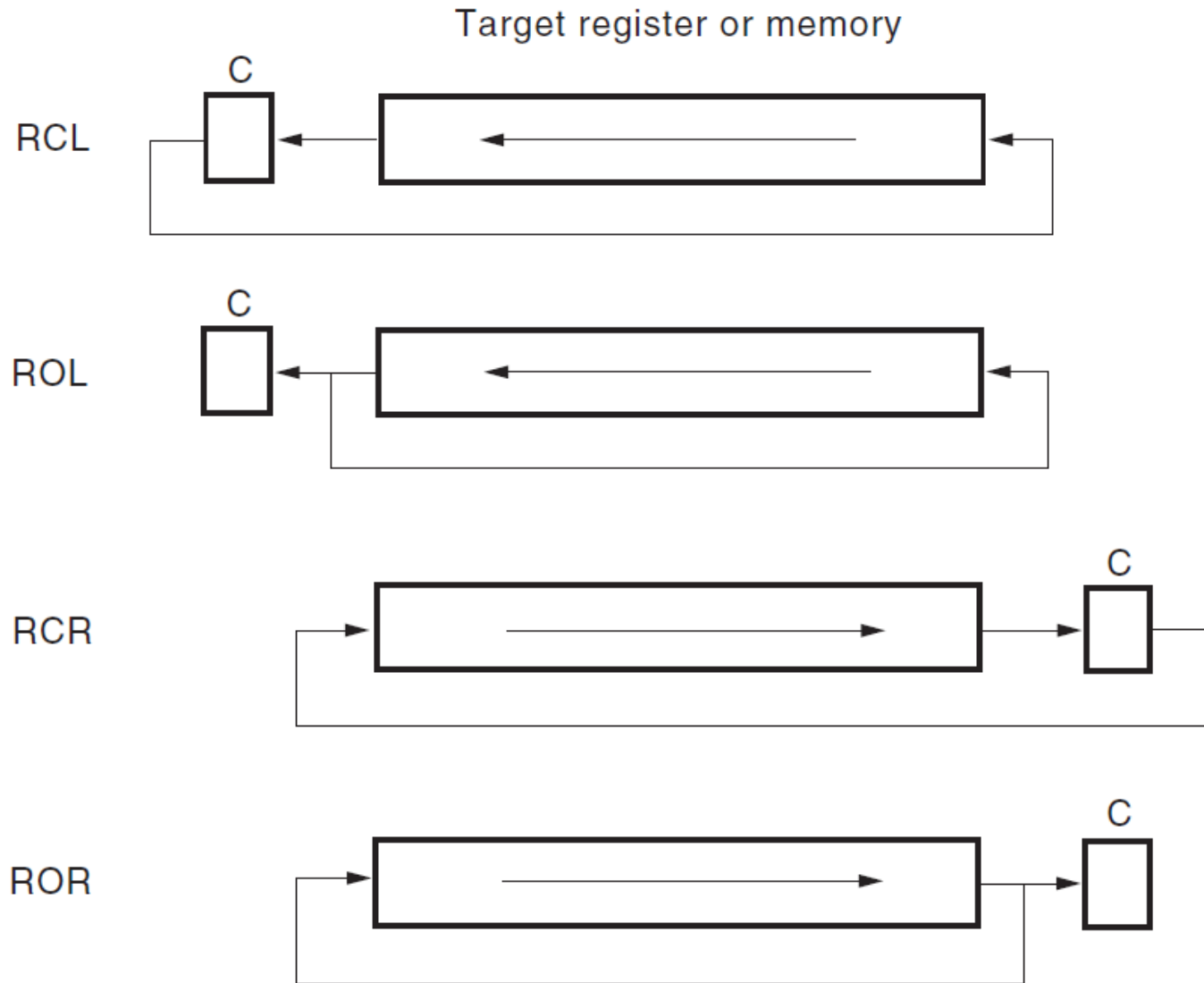
TABLE 5–22 Example shift instructions.

<i>Assembly Language</i>	<i>Operation</i>
SHL AX,1	AX is logically shifted left 1 place
SHR BX,12	BX is logically shifted right 12 places
SHR ECX,10	ECX is logically shifted right 10 places
SHL RAX,50	RAX is logically shifted left 50 places (64-bit mode)
SAL DATA1,CL	The contents of data segment memory location DATA1 are arithmetically shifted left the number of spaces specified by CL
SHR RAX,CL	RAX is logically shifted right the number of spaces specified by CL (64-bit mode)
SAR SI,2	SI is arithmetically shifted right 2 places
SAR EDX,14	EDX is arithmetically shifted right 14 places

Rotate

- Positions binary data by rotating information in a register or memory location, either from one end to another or through the carry flag.
 - used to shift/position numbers wider than 16 bits
- With either type of instruction, the programmer can select either a left or a right rotate.
- Addressing modes used with rotate are the same as those used with shifts.
- Rotate instructions appear in Figure 5–10.

Figure 5–10 The rotate instructions showing the direction and operation of each rotate.



- A rotate count can be immediate or located in register CL.
 - if CL is used for a rotate count, it does not change
 - Rotate instructions are often used to shift wide numbers to the left or right.
-

```
SHL AX, 1  
RCL BX, 1  
RCL DX, 1
```

- This program shifts the 48-bit number in registers DX, BX, and AX left one binary place.
- Notice that the least significant 16 bits (AX) shift left first.
- This moves the leftmost bit of AX into the carry flag bit.
- Next, the rotate BX instruction rotates carry into BX, and its leftmost bit moves into carry.
- The last instruction rotates carry into DX, and the shift is complete

ROL

memory, immediate
REG, immediate

memory, CL
REG, CL

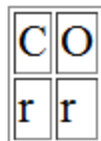
Rotate operand1 left. The number of rotates is set by operand2.

Algorithm:

shift all bits left, the bit that goes off is set to CF and the same bit is inserted to the right-most position.

Example:

```
MOV AL, 1Ch    ; AL = 00011100b
ROL AL, 1      ; AL = 00111000b, CF=0.
RET
```



OF=0 if first operand keeps original sign.

Examples

TABLE 5–23 Example rotate instructions.

<i>Assembly Language</i>	<i>Operation</i>
ROL SI,14	SI rotates left 14 places
RCL BL,6	BL rotates left through carry 6 places
ROL ECX,18	ECX rotates left 18 places
ROL RDX,40	RDX rotates left 40 places
RCR AH,CL	AH rotates right through carry the number of places specified by CL
ROR WORD PTR[BP],2	The word contents of the stack segment memory location addressed by BP rotate right 2 places

SUMMARY

- Flags
- Addition, Subtraction and Comparison
- Multiplication and Division
- BCD and ASCII Arithmetic
- Basic Logic Instructions
- Shift and Rotate



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Program Control Instructions

Introduction

- This chapter explains the program control instructions, including the jumps, loops, calls, and returns.

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Use both conditional and unconditional jump instructions to control the flow of a program.
- Use conditional and unconditional loops to control the flow of a program.
- Use the call and return instructions to include procedures in the program structure.

Program Flow Control

- Controlling the program flow is very important, where your program can make decisions based on particular conditions.
- Remember the **if** statement in C programming language.
- In the Assembly language, flow control is handled by several **Jump** instructions including conditional and unconditional jumps.

Unconditional Jumps

- The basic instruction that transfers control to another point in the program is **JMP**:
JMP label
- To declare a **label** in your program, just type its name and add ":" to the end.
- A label can be any character combination but it cannot start with a number. Examples:
label1:
label2:
a:

More About Labels

- Label can be declared on a separate line or before any other instruction

- Examples:

x1:

MOV AX, 1

x2: MOV AX, 2

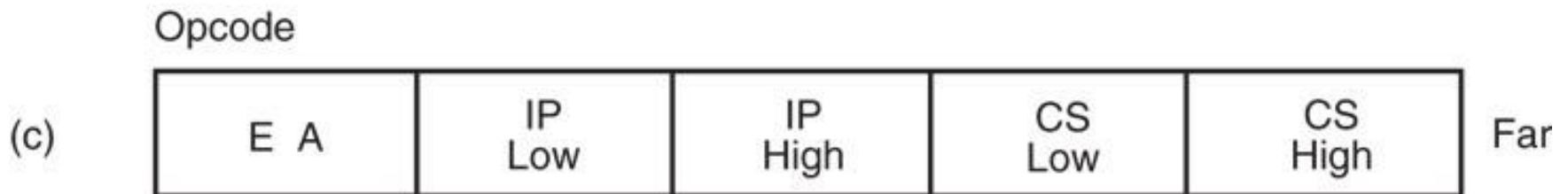
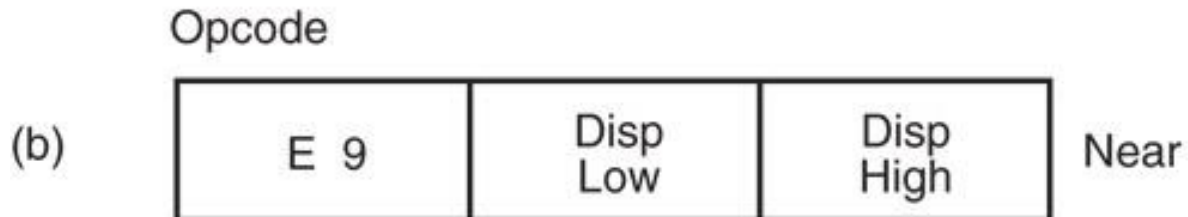
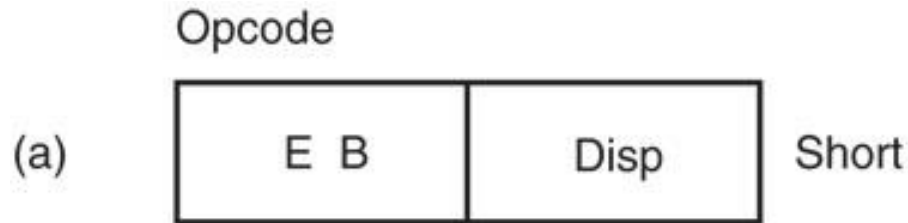
JMP Example

```
ORG 100H
MOV AX, 5      ; set AX to 5.
MOV BX, 2      ; set BX to 2.
JMP calc       ; go to 'calc'.
back:
JMP stop       ; go to 'stop'.
calc:
ADD AX, BX     ; add BX to AX.
JMP back       ; go 'back'.
stop:
RET            ; return to operating system.
```

JMP in Detail

- There are three types of the JMP instruction:
 1. **Short Jump**
 2. **Near Jump**
 3. **Far Jump**
- **Short** and **near** jumps are intra-segment jumps and **far** jump is inter-segment jump.

Jump Types

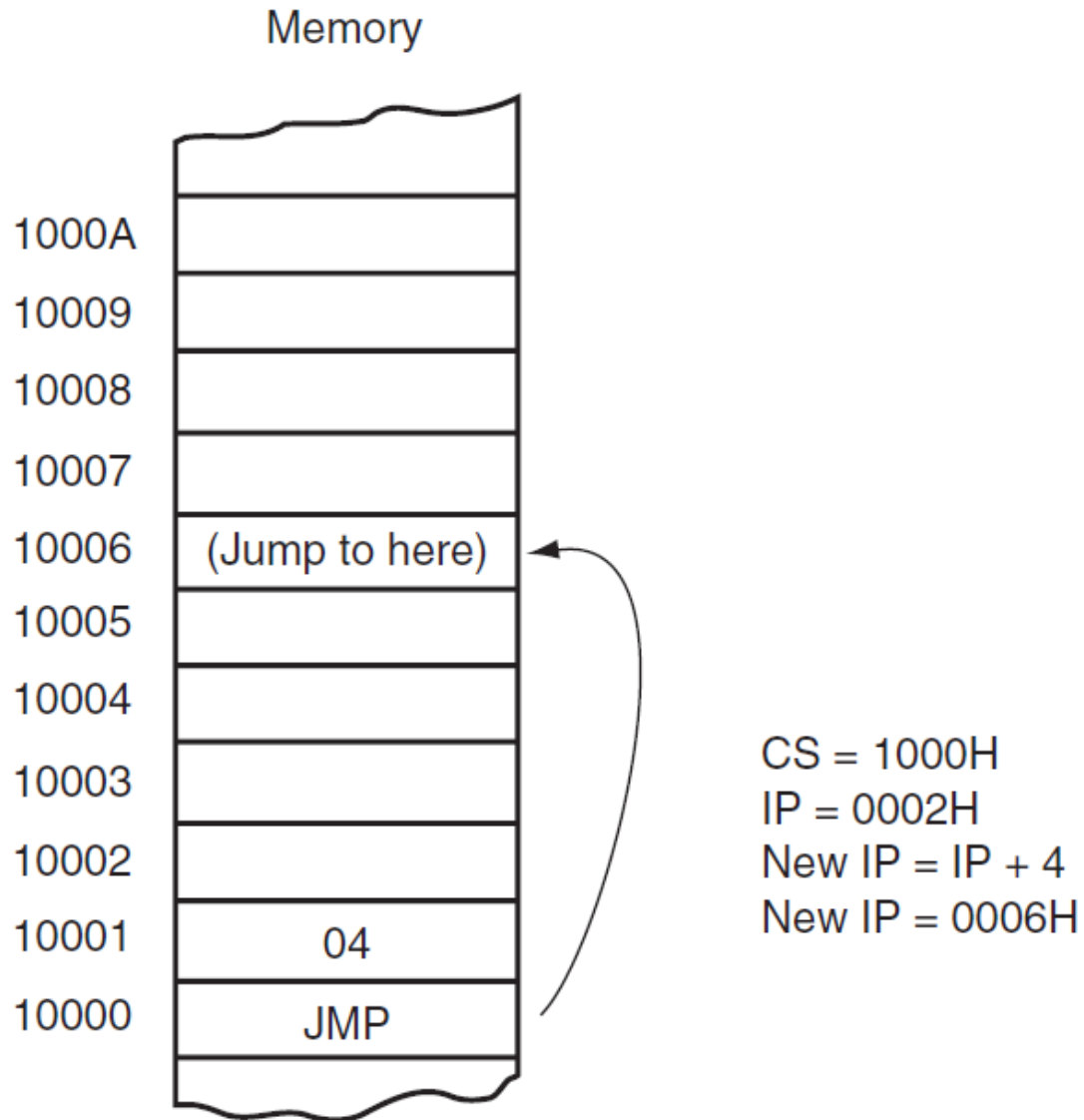


Short Jump

- The jump address is not stored in the opcode.
- Instead, a **distance**, or **displacement**, follows the opcode.
- Therefore, short jumps are also called relative jumps.
- The displacement takes a value in the range between **-128** and **+127**.

Short Jump Example

FIGURE 6–2 A short jump to four memory locations beyond the address of the next instruction.

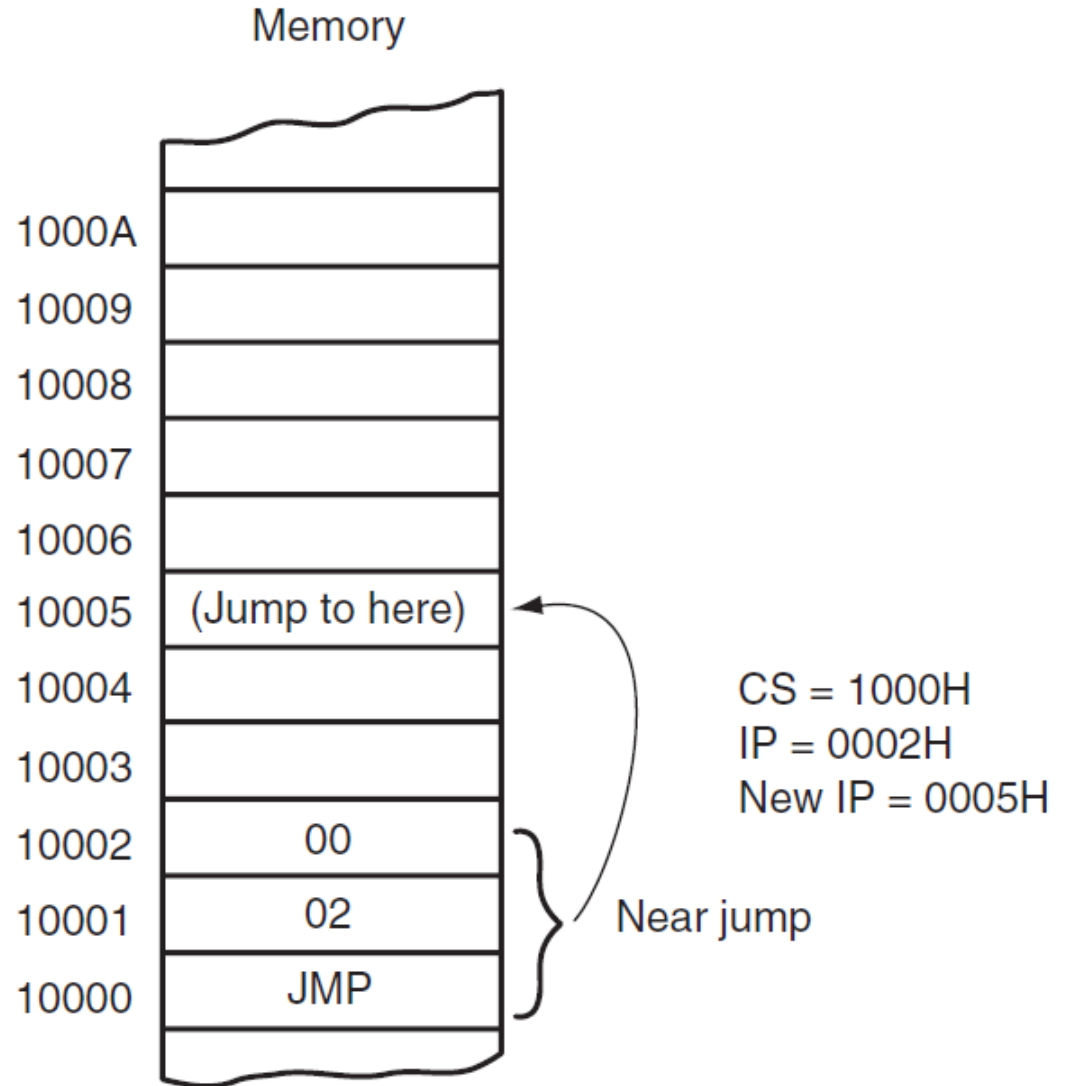


Near Jump

- The near jump is **similar** to the short jump, except that the **distance is farther**.
- A **near jump** passes control to an instruction in the current code segment located within **$\pm 32\text{K}$ bytes** from the near jump instruction.
- Near jump is also a **relative** jump.

Near Jump Example

FIGURE 6–3 A near jump that adds the displacement (0002H) to the contents of IP.

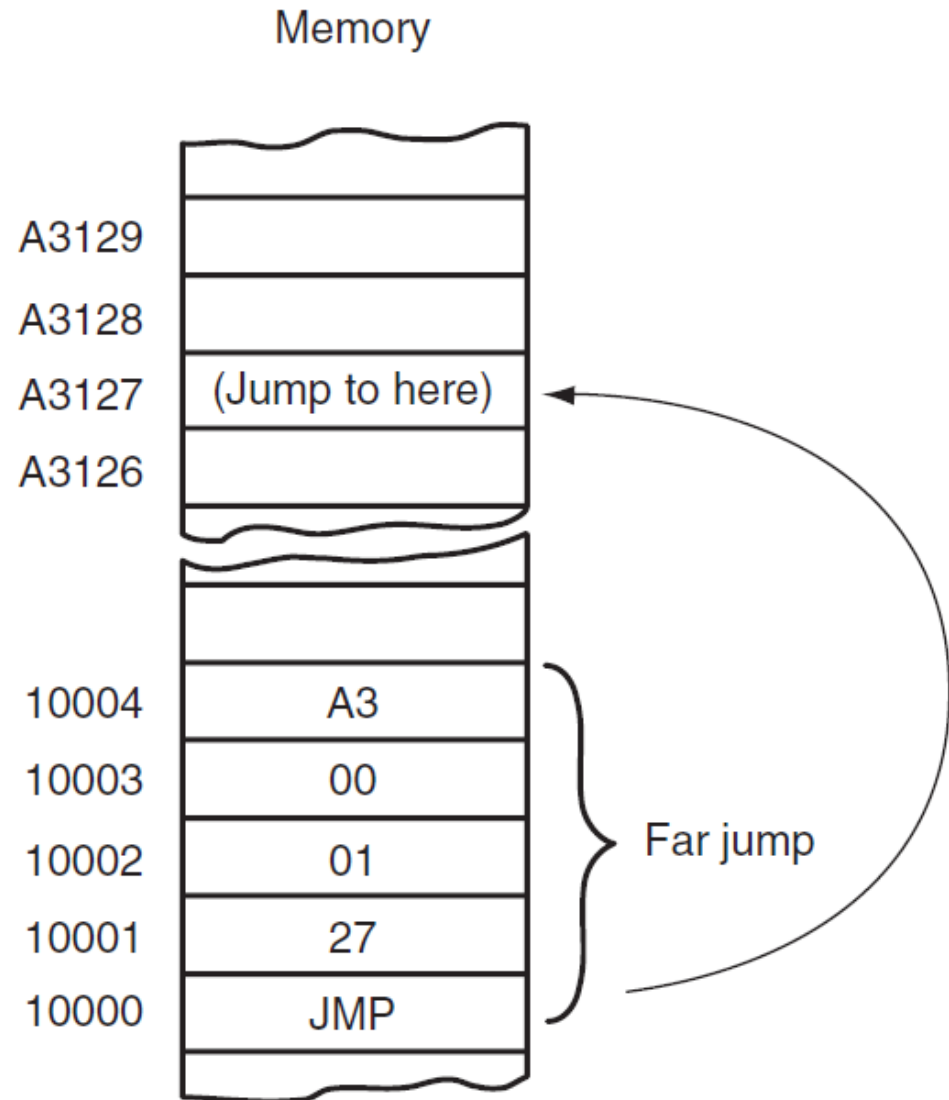


Far Jump

- A **far jump** instruction obtains a new segment and offset address to accomplish the jump.
- The far jump instruction sometimes appears with the **FAR PTR** directive.
- Another way to obtain a far jump is to define a label as a **far label**. A label is far only if it is external to the current code segment or procedure (i.e. the compiler sets it automatically).

Far Jump Example

FIGURE 6–4 A far jump instruction replaces the contents of both CS and IP with 4 bytes following the opcode.



Conditional Jumps

- Unlike **JMP** instruction that does an unconditional jump, there are instructions that perform conditional jumps (jump only when some conditions are true)
- Conditional jumps are always short jumps in 8086 - 80286.
- These instructions are divided in three groups:
 - First group just tests single flag
 - Second group compares numbers as signed
 - Third group compares numbers as unsigned.

General Syntax

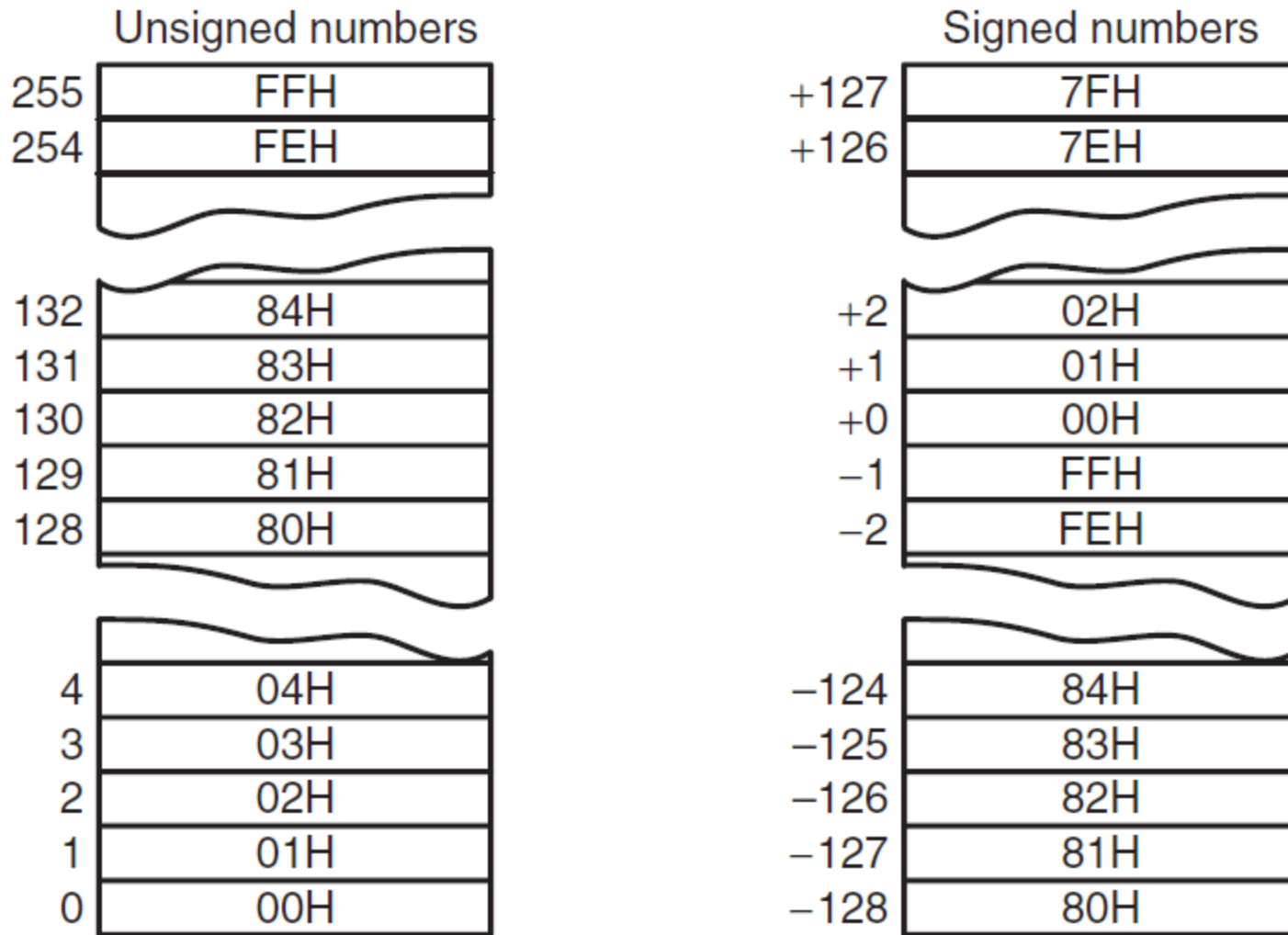
- The general syntax for conditional jumps:
<Conditional jump instruction> <label>
Example: **JC label1**
- If the condition under the test is true, a branch to the label occurs.
- If the condition is false, the next sequential step in the program executes.

Jump Instructions That Test Single Flag

Instruction	Description	Condition	Opposite
JZ, JE	Jump if Zero (Equal)	$Z = 1$	JNZ, JNE
JC, JB, JNAE	Jump if Carry (Below, Not Above Equal)	$C = 1$	JNC, JNB, JAE
JS	Jump if Sign	$S = 1$	JNS
JO	Jump if Overflow	$O = 1$	JNO
JPE, JP	Jump if Parity Even	$P = 1$	JPO, JNP
JNZ, JNE	Jump if Not Zero (Not Equal)	$Z = 0$	JZ, JE
JNC, JNB, JAE	Jump if Not Carry (Not Below, Above Equal)	$C = 0$	JC, JB, JNAE
JNS	Jump if Not Sign	$S = 0$	JS
JNO	Jump if Not Overflow	$O = 0$	JO
JPO, JNP	Jump if Parity Odd (No Parity)	$P = 0$	JPE, JP

* Different names are used to make programs easier to understand.

Figure 6–5 Signed and unsigned numbers follow different orders.



Jump Instructions for Signed Numbers

Instruction	Description	Condition	Opposite
JE, JZ	Jump if Equal (=) Jump if Zero	<u>Z = 1</u>	JNE, JNZ
JNE, JNZ	Jump if Not Equal (≠) Jump if Not Zero	<u>Z = 0</u>	JE, JZ
JG, JNLE	Jump if Greater (>) Jump if Not Less or Equal (not <=)	<u>Z = 0</u> and <u>S = 0</u>	JNG, JLE
JL, JNGE	Jump if Less (<) Jump if Not Greater or Equal	<u>S ≠ 0</u>	JNL, JGE
JGE, JNL	Jump if Greater or Equal (>=) Jump if Not Less	<u>S = 0</u>	JNGE, JL
JLE, JNG	Jump if Less or Equal (<=) Jump if Not Greater	<u>Z = 1</u> or <u>S ≠ 0</u>	JNLE, JG

Jump Instructions for Unsigned Numbers

Instruction	Description	Condition	Opposite
JE, JZ	Jump if Equal (=) Jump if Zero	<u>Z = 1</u>	JNE, JNZ
JNE, JNZ	Jump if Not Equal (≠) Jump if Not Zero	<u>Z = 0</u>	JE, JZ
JA, JNBE	Jump if Above (>) Jump if Not Below or Equal	<u>C = 0</u> and <u>Z = 0</u>	JNA, JBE
JBE, JNA	Jump if Below or Equal (<=) Jump if Not Above	<u>C = 1</u> or <u>Z = 1</u>	JNBE, JA
JB, JNAE, JC	Jump if Below (<) Jump if Not Above or Equal Jump if Carry	<u>C = 1</u>	JNB, JAE, JNC
JAE, JNB, JNC	Jump if Above or Equal (>=) Jump if Not Below Jump if Not Carry	<u>C = 0</u>	JB, JNAE, JC

Signed or Unsigned?

- The numbers in a comparison may be signed or unsigned, according to your selection.
- When signed numbers are compared, use the instructions with the terms **“greater than”**, **“less than”**, etc.
- When unsigned numbers are compared, use the instructions with the terms **“above”** and **“below”**.

Conditional Jump Example

```
ORG 100H
MOV AL, 25 ; set AL to 25.
MOV BL, 10 ; set BL to 10.
CMP AL, BL ; compare AL - BL.
JE equal   ; jump if AL = BL (Z = 1).
MOV CX, 1  ; if it gets here, then AL <> BL
JMP stop   ; so set CX, and jump to stop.
equal:     ; if gets here,
MOV CX, 0  ; then AL = BL, so clear CX.
stop:
RET        ; gets here no matter what.
```

Limitation

- All conditional jumps have one big limitation, unlike **JMP** instruction they can only jump **127 bytes forward and 128 bytes backward**.
- We can easily avoid this limitation using a cute trick:
 - Get an opposite conditional jump instruction from the table above, make it jump to *label_x*.
 - Use **JMP** instruction to jump to desired location.
 - Define *label_x*: just after the **JMP** instruction.
 - *label_x*: - can be any valid label name, but there must not be two or more labels with the same name.

Example

```
include "emu8086.inc"

org    100h

mov     al, 5
mov     bl, 5

cmp     al, bl      ; compare al - bl.

; je equal          ; there is only 1 byte

jne     not_equal   ; jump if al <> bl (zf = 0).
jmp     equal
not_equal:

add     bl, al
sub     al, 10
xor     al, bl

jmp     skip_data
db 256 dup(0)      ; 256 bytes
skip_data:

putc    'n'         ; if it gets here, then al <> bl,
jmp     stop        ; so print 'n', and jump to stop.

equal:
putc    'y'         ; if gets here,
                ; then al = bl, so print 'y'.

stop:

ret
```

LOOP

- LOOP is similar to JMP.
- It is a combination of a decrement CX and the JNZ conditional jump.
- LOOP decrements CX.
 - if $CX \neq 0$, it jumps to the address indicated by the label
 - If CX becomes 0, the next sequential instruction executes

Conditional LOOPS

- LOOP instruction also has conditional forms:
 LOOPE and LOOPNE
- LOOPE (**loop while equal**) instruction jumps if CX != 0 while an equal condition exists.
 - will exit loop if the condition is not equal or the CX register decrements to 0
- LOOPNE (**loop while not equal**) jumps if CX != 0 while a not-equal condition exists.
 - will exit loop if the condition is equal or the CX register decrements to 0

Loops

Instruction	Operation and Jump Condition	Opposite Direction
LOOP	Decrease CX, jump to label if CX not zero	DEC CX and JCXZ
LOOPE	Decrease CX, jump to label if CX not zero and equal (Z = 1)	LOOPNE
LOOPNE	Decrease CX, jump to label if CX not zero and not equal (Z = 0)	LOOPE
LOOPNZ	Decrease CX, jump to label if CX not zero and Z = 0	LOOPZ
LOOPZ	Decrease CX, jump to label if CX not zero and Z = 1	LOOPNZ
JCXZ	Jump to label if CX is zero	OR CX, CX and JNZ

Loop Example: C Code

- Write a program that sums the contents of two arrays and stores the result over the second array.

```
short arr1[100];  
short arr2[100];  
int count = 100;  
int idx = 0;  
while (count > 0)  
{  
    arr2[idx] = arr1[idx] + arr2[idx];  
    idx++;  
    count--;  
}
```

Loop Example: Assembly Code

```
org 100h
mov cx, 100 ; Number of elements in the blocks
mov bx, 0   ; Start index
L1:
mov ax, BLOCK1[bx] ; read next number from BLOCK1
add ax, BLOCK2[bx] ; add next number from BLOCK2
mov BLOCK2[bx], ax ; store the result
add bx, 2          ; skip to next element
loop L1
Ret
BLOCK1 DW 100 DUP (1)
BLOCK2 DW 100 DUP (2)
```

Nested Loops

- All loop instructions use **CX** register to count steps, as you know CX register has 16 bits and the maximum value it can hold is 65535 or FFFF
- However with some agility it is possible to put one loop into another, and receive a nice value of 65535 x 65535 x 65535till infinity.... or the end of ram or stack memory.
- It is possible store original value of CX register using **PUSH CX** instruction and return it to original when the internal loop ends with **POP CX**

Nested Loop Example

```
org 100h
mov bx, 0 ; total step counter.
mov cx, 5
k1: add bx, 1
    mov al, '1'
    mov ah, 0eh
    int 10h
    push cx
    mov cx, 5
    k2: add bx, 1
        mov al, '2'
        mov ah, 0eh
        int 10h
        push cx
        mov cx, 5
        k3: add bx, 1
            mov al, '3'
            mov ah, 0eh
            int 10h
            loop k3 ; internal in internal loop.
        pop cx
    loop k2 ; internal loop.
    pop cx
loop k1 ; external loop.
ret
```

5

5

5

$O(n^3)$

PROCEDURES

- A procedure is a group of instructions that usually performs one task.
 - subroutine, method, or **function** is an important part of any system's architecture
- A procedure is a reusable section of the software stored in memory once, used as often as necessary.
 - saves memory space and makes it easier to develop software

Procedure Syntax

- name PROC
 ; here goes the code
 ; of the procedure ...
RET
name ENDP
- name - is the procedure name
- the same name should be in the **top** and the **bottom**, this is used to check correct closing of procedures.

Procedure Example

```
SUMS    PROC  
        ADD AX, BX  
        ADD AX, CX  
        ADD AX, DX  
        RET  
SUMS    ENDP
```


Calling a Procedure

- **CALL** instruction is used to call a procedure.
- Example:

ORG 100H

CALL m1

MOV AX, 2

RET ; return to operating system.

m1 PROC

MOV BX, 5

RET ; return to caller.

m1 ENDP

END

CALL

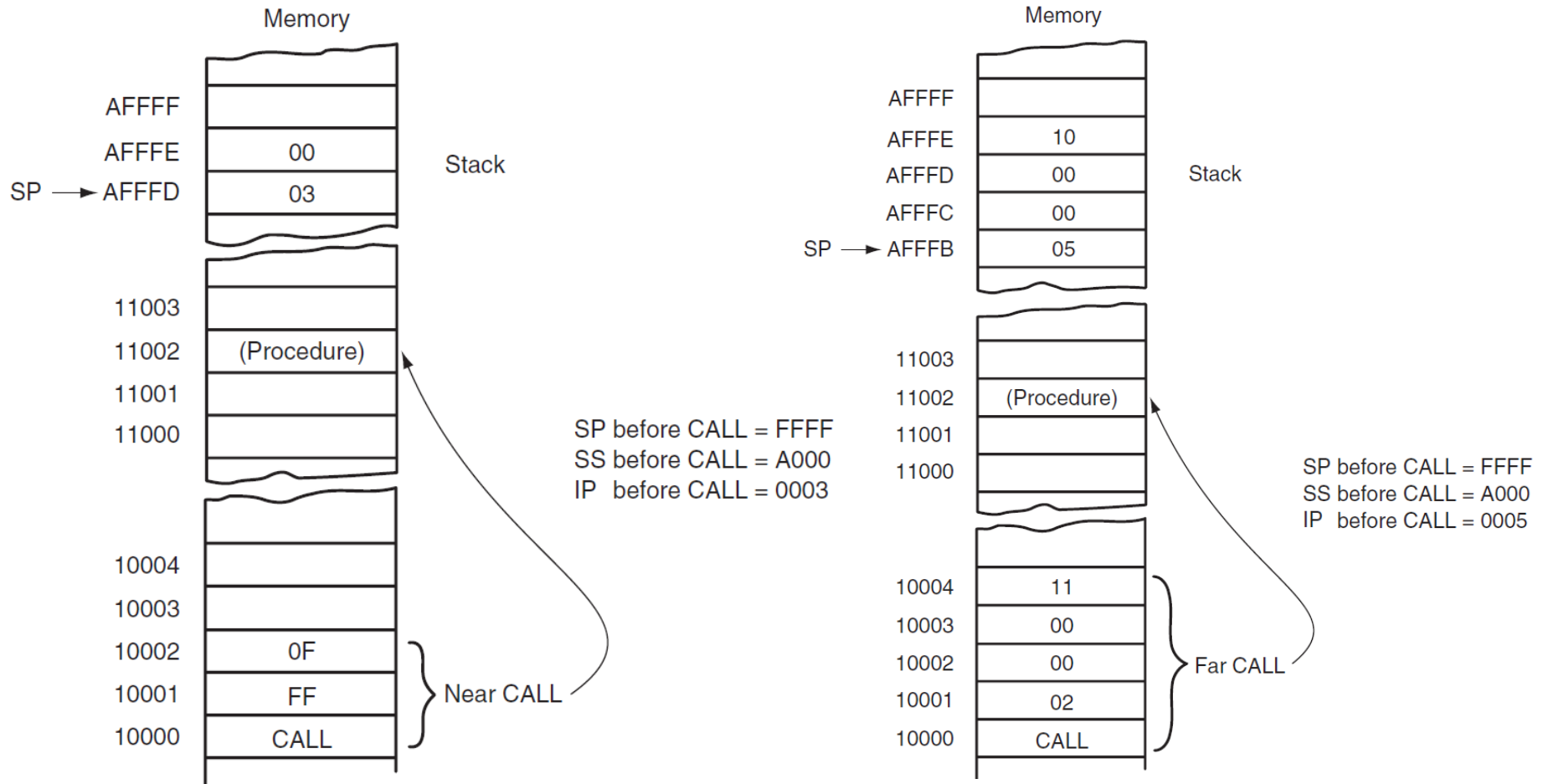
- Transfers the flow of the program to the procedure.
- CALL instruction differs from the jump instruction because a CALL saves a return address on the stack.
- The return address returns control to the instruction that immediately follows the CALL in a program when a RET instruction executes at the end of procedure.

CALL

(cont.)

- The CALL construction is a combination of a PUSH and a JMP instruction.
- When CALL executes, it pushes the return address on the stack and then jumps to the procedure.
- A near CALL places the contents of IP on the stack, and a far CALL places both IP and CS on the stack.

Near and Far CALL Examples



RET

- The RET instruction returns from a procedure by removing the return address from the stack and placing it into IP (near return), or IP and CS (far return).

Passing Parameters to a Procedure

- There are several ways to pass parameters to a procedure
- The easiest way is by using registers
 - Put parameters to the registers and call the procedure
 - The procedure should be designed so that it uses those registers
- Another way is by using stack
 - Push the parameters to the stack and call the procedure
 - The procedure should be designed so that it receives parameters from the stack
- In either way, the programmer should know the structure of the procedure

Example

```
ORG 100h
MOV AL, 1
MOV BL, 2
CALL m2
CALL m2
CALL m2
CALL m2
RET ; return to operating system.
```

```
m2 PROC
    MUL BL ; AX = AL * BL.
    RET ; return to caller.
m2 ENDP
END
```


About the Example

- The program calculates 2^4
- The procedure takes its parameters from **AL** and **BL**; multiplies them and stores the result into **AX**
- The C equivalent can be thought as the following:

```
short m2(char a1, char b1)
{
    short ax = a1 * b1;
    return ax;
}
```



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Memory Interface

Introduction

- Simple or complex, every microprocessor-based system has a memory system.
- Almost all systems contain two main types of memory: **read-only memory** (ROM) and **random access memory** (RAM) or read/write memory.
- This chapter explains how to interface both memory types to the Intel family of microprocessors.

Chapter Objectives

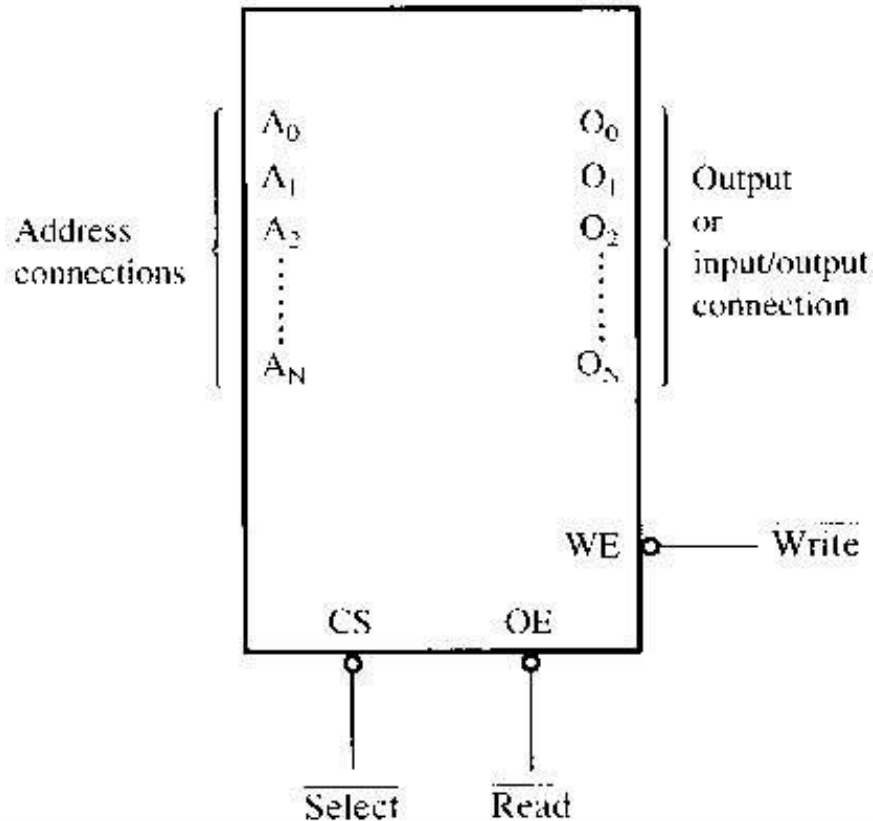
Upon completion of this chapter, you will be able to:

- Decode the memory address and use the outputs of the decoder to select various memory components.
- Use programmable logic devices (PLDs) to decode memory addresses.
- Explain how error correction code (ECC) is used with memory.

MEMORY DEVICES

- Before attempting to interface memory to the microprocessor, it is essential to understand the operation of memory components.
- In this section, we explain functions of the four common types of memory:
 - Read-Only Memory (ROM)
 - Flash Memory (EEPROM)
 - Static Random Access Memory (SRAM)
 - Dynamic Random Access Memory (DRAM)

Memory Pin Connections



- address inputs
- data outputs or input/outputs
- some type of selection input
- at least one control input to select a read or write operation

Figure. A pseudomemory component illustrating the address, data, and control connections.

Address Connections

- Memory devices have address inputs to select a memory location within the device.
- Almost always labeled from A_0 , the least significant address input, to A_n
 - where subscript n can be any value
 - always labeled as one less than total number of address pins
- A memory device with 10 address pins has its address pins labeled from A_0 to A_9 .

- The number of address pins on a memory device is determined by the number of memory locations found within it.
- Today, memory devices have memory locations starting from 1K to multiple G.
- A 1K memory device has 10 address pins.
 - therefore, 10 address inputs are required to select any of its 1024 memory locations

- It takes a 10-bit binary number to select any single location on a 1024-location device.
 - 1024 different combinations
 - if a device has 11 address connections, it has 2048 (2K) internal memory locations
- The number of memory locations can be extrapolated from the number of pins.

Data Connections

- All memory devices have a set of data outputs or input/outputs.
 - today, many devices have bidirectional common I/O pins
 - data connections are points at which data are entered for storage or extracted for reading
- Data pins on memory devices are labeled D_0 through D_7 for an 8-bit-wide memory device.

- An 8-bit-wide memory device is often called a **byte-wide** memory.
 - most devices are currently 8 bits wide,
 - some are 16 bits, 4 bits, or just 1 bit wide
- Catalog listings of memory devices often refer to memory locations times bits per location.
 - a memory device with 1K memory locations and 8 bits in each location is often listed as a **1K × 8** by the manufacturer
- Memory devices are often classified according to total bit capacity.

Selection Connections

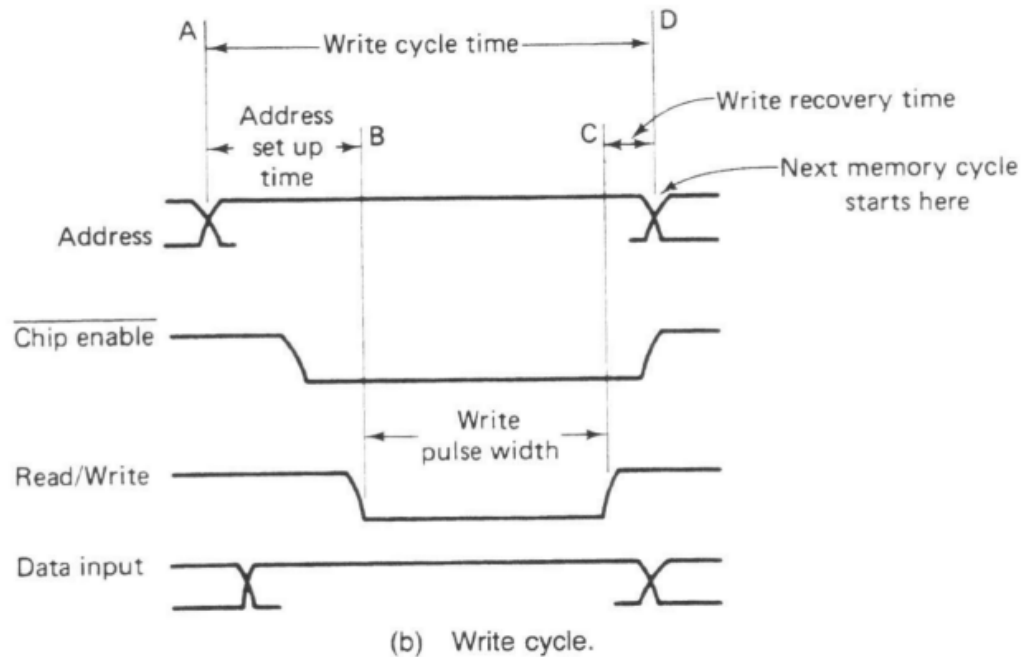
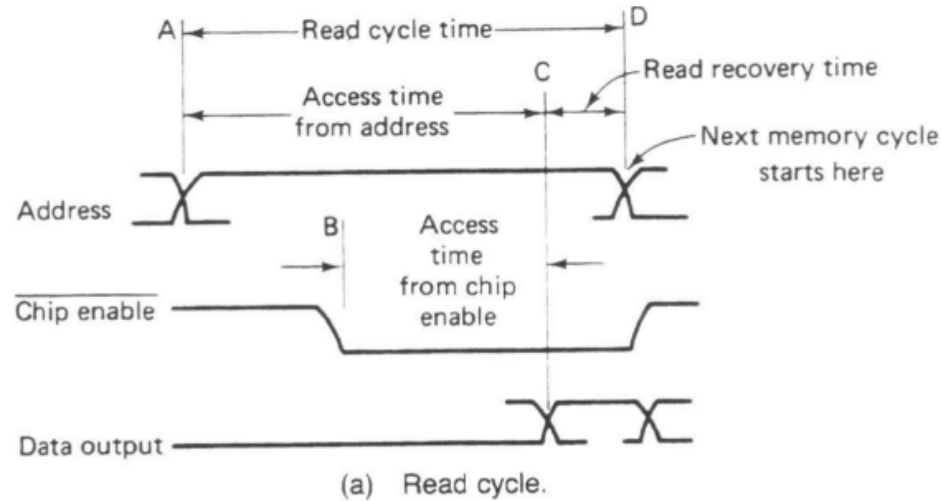
- Each memory device has an input that selects or enables the memory device.
 - sometimes more than one
- This type of input is most often called a **chip select** ($\overline{CS}$) **chip enable** ($\overline{CE}$) or simply **select** ($\overline{S}$) input.
- RAM memory generally has at least one $\overline{CS}$ or $\overline{S}$ input, and ROM has at least one $\overline{CE}$
- If more than one $\overline{CE}$ connection is present, all must be activated to read or write data.

Control Connections

- All memory devices have some form of control input or inputs.
 - ROM usually has one control input, while RAM often has one or two control inputs
- Control input often found on ROM is the **output enable** or **gate** connection, which allows data flow from output data pins.
- The $\overline{\text{OE}}$ connection enables and disables a set of three-state buffers located in the device and must be active to read data.

- RAM has either one or two control inputs.
 - if one control input, it is often called $R/\overline{W}$
- If the RAM has two control inputs, they are usually labeled $\overline{WE}$ (or $\overline{W}$), and $\overline{OE}$ (or $\overline{G}$).
 - **write enable** must be active to perform memory write, and $\overline{OE}$ active to perform a memory read
 - when the two controls are present, they must never both be active at the same time
- If both inputs are inactive, data are neither written nor read.
 - the connections are at their high-impedance state

Sample Memory Read/Write Timing Cycles



Datasheets

- You can get detailed information on ICs including memory units by just looking at their datasheets.
- Every manufacturer releases datasheets of their products which includes operational information.

ROM Memory

- **Read-only memory (ROM)** permanently stores programs/data resident to the system.
 - and must not change when power disconnected
- Often called **nonvolatile memory**, because its contents *do not* change even if power is disconnected.
- A device we call a ROM is purchased in mass quantities from a manufacturer.
 - programmed during fabrication at the factory

- The EPROM (**erasable programmable read-only memory**) is commonly used when software must be changed often.
 - or when low demand makes ROM uneconomical
 - for ROM to be practical at least 10,000 devices must be sold to recoup factory charges
- An EPROM is programmed in the field on a device called an EPROM programmer.
- Also erasable if exposed to high-intensity ultraviolet light.
 - depending on the type of EPROM

- PROM memory devices are also available, although they are not as common today.
- The PROM (**programmable read-only memory**) is also programmed in the field by burning open tiny Ni-chrome or silicon oxide fuses.
- Once it is programmed, it cannot be erased.

- A newer type of **read-mostly memory** (RMM) is called the **flash memory**.
 - also often called an EEPROM (**electrically erasable programmable ROM**)
 - EAROM (**electrically alterable ROM**)
 - or a NOVRAM (**nonvolatile RAM**)
- Electrically erasable in the system, but they require more time to erase than normal RAM.
- The flash memory device is used to store setup information for systems such as the video card in the computer.

- Flash has all but replaced the EPROM in most computer systems for the BIOS.
 - some systems contain a password stored in the flash memory device
- Flash memory has its biggest impact in memory cards for digital cameras and memory in MP3 audio players.

Static RAM (SRAM) Devices

- Static RAM memory devices retain data as long as DC power is applied.
- Because no special action is required to retain data, these devices are called **static memory**.
 - also called **volatile memory** because they will not retain data without power
- The main difference between ROM and RAM is that RAM is written under normal operation, whereas ROM is programmed outside the computer and normally is only read.

- Access time on a typical 4016 SRAM is 250 ns, fast enough to connect directly to an 8088/8086 at 5 MHz, without wait states.
- Access time must always be checked to determine compatibility of memory components
- Access times can be as low as 1.0 ns for SRAM used in computer cache memory

Dynamic RAM (DRAM) Memory

- Available up to $256\text{M} \times 8$ (2G bits).
- DRAM is essentially the same as SRAM, except that it retains data for only 2 or 4 ms on an integrated capacitor.
- After 2 or 4 ms, the contents of the DRAM must be completely rewritten (*refreshed*).
 - because the capacitors, which store a logic 1 or logic 0, lose their charges

- In DRAM, the entire contents are refreshed with 256 reads in a 2- or 4-ms interval.
 - also occurs during a write, a read, or during a special refresh cycle
- DRAM requires so many address pins that manufacturers multiplexed address inputs.
- DRAM is often placed on small boards called SIMMs (Single In-Line Memory Modules) or DIMMs (Dual In-Line Memory Modules).

- Another type is the RIMM memory module from RAMBUS Corporation,
 - this memory type has faded from the market
- The latest DRAM is the DDR (**double-data rate**) memory device and DDR2.
 - DDR transfers data at each edge of the clock, making it operate at twice the speed of SDRAM.



Common DRAM packages

From top to bottom:
DIP, SIPP, SIMM
(30-pin), SIMM (72-
pin), DIMM (168-
pin), DDR DIMM
(184-pin).

SRAM vs DRAM

- SRAM is static while DRAM is dynamic
- SRAM is faster compared to DRAM
- SRAM consumes less power than DRAM
- SRAM uses more transistors per bit of memory compared to DRAM
- SRAM is more expensive than DRAM
- Cheaper DRAM is used in main memory while SRAM is commonly used in cache memory

ADDRESS DECODING

- In order to attach a memory device to the microprocessor, it is necessary to decode the address sent from the microprocessor.
- Decoding makes the memory function at a unique section or partition of the memory map.
- Without an address decoder, only one memory device can be connected to a microprocessor, which would make it virtually useless.

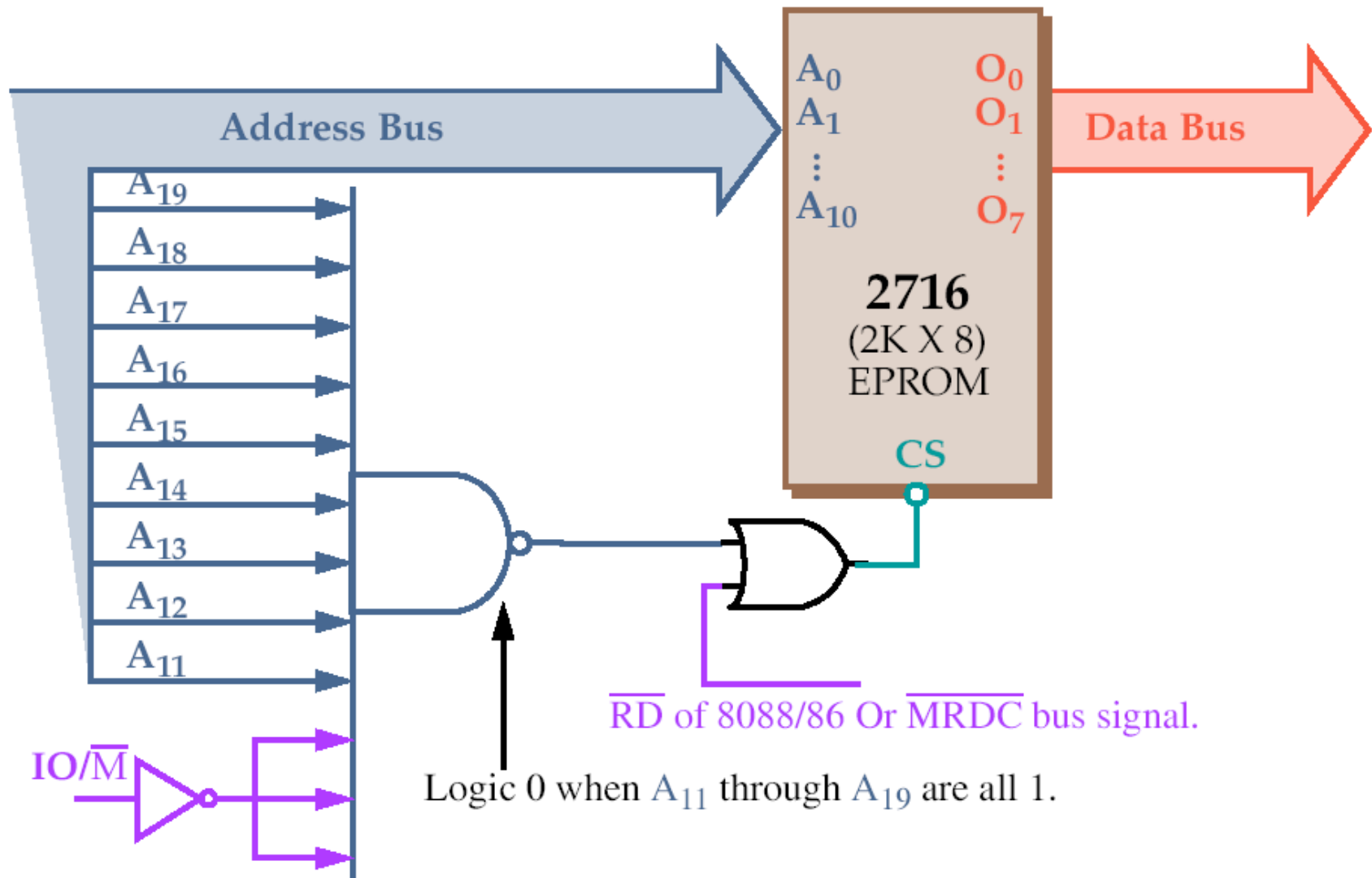
Why Decode Memory?

- The 8088 has 20 address connections and the 2716 EPROM has 11 connections.
- The 8088 sends out a 20-bit memory address whenever it reads or writes data.
 - because the 2716 has only 11 address pins, there is a mismatch that must be corrected
- The decoder corrects the mismatch by decoding address pins that do not connect to the memory component.

Simple NAND Gate Decoder

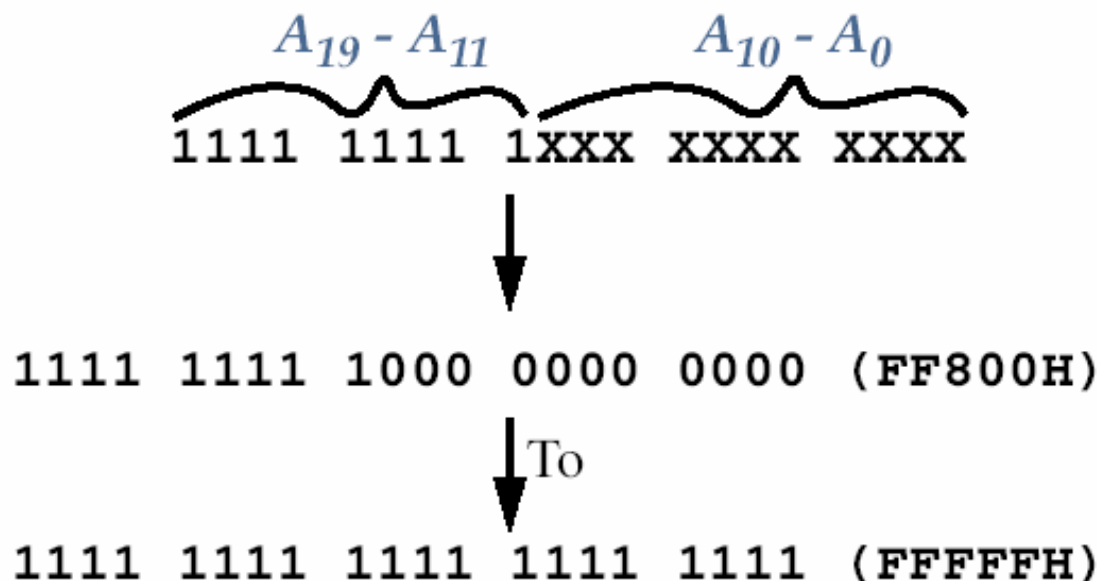
- When the $2K \times 8$ EPROM is used, address connections $A_{10}-A_0$ of 8088 are connected to address inputs $A_{10}-A_0$ of the EPROM.
 - the remaining nine address pins ($A_{19}-A_{11}$) are connected to a NAND gate decoder
- The decoder selects the EPROM from one of the 2K-byte sections of the 1M-byte memory system in the 8088 microprocessor.
- In this circuit a NAND gate decodes the memory address, as seen in the figure.

Figure. A simple NAND gate decoder that selects a 2716 EPROM for memory location FF800H–FFFFFH.



- If the 20-bit binary address, decoded by the NAND gate, is written so that the leftmost nine bits are 1s and the rightmost 11 bits are don't cares (X), the actual address range of the EPROM can be determined.
 - a *don't care* is a logic 1 or a logic 0, whichever is appropriate

To determine the address range that a device is mapped into:



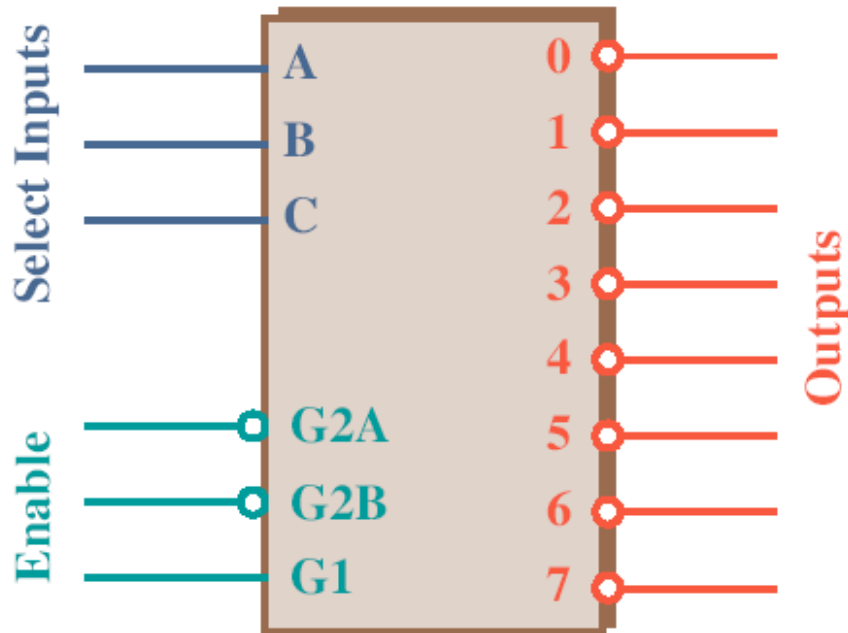
NAND gate decoders are not often used

Large fan-in NAND gates are not efficient

Multiple NAND gate IC's might be required to perform such decoding

Rather the 3-to-8 Line Decoder (74LS138) is more common.

The 3-to-8 Line Decoder (74LS138)

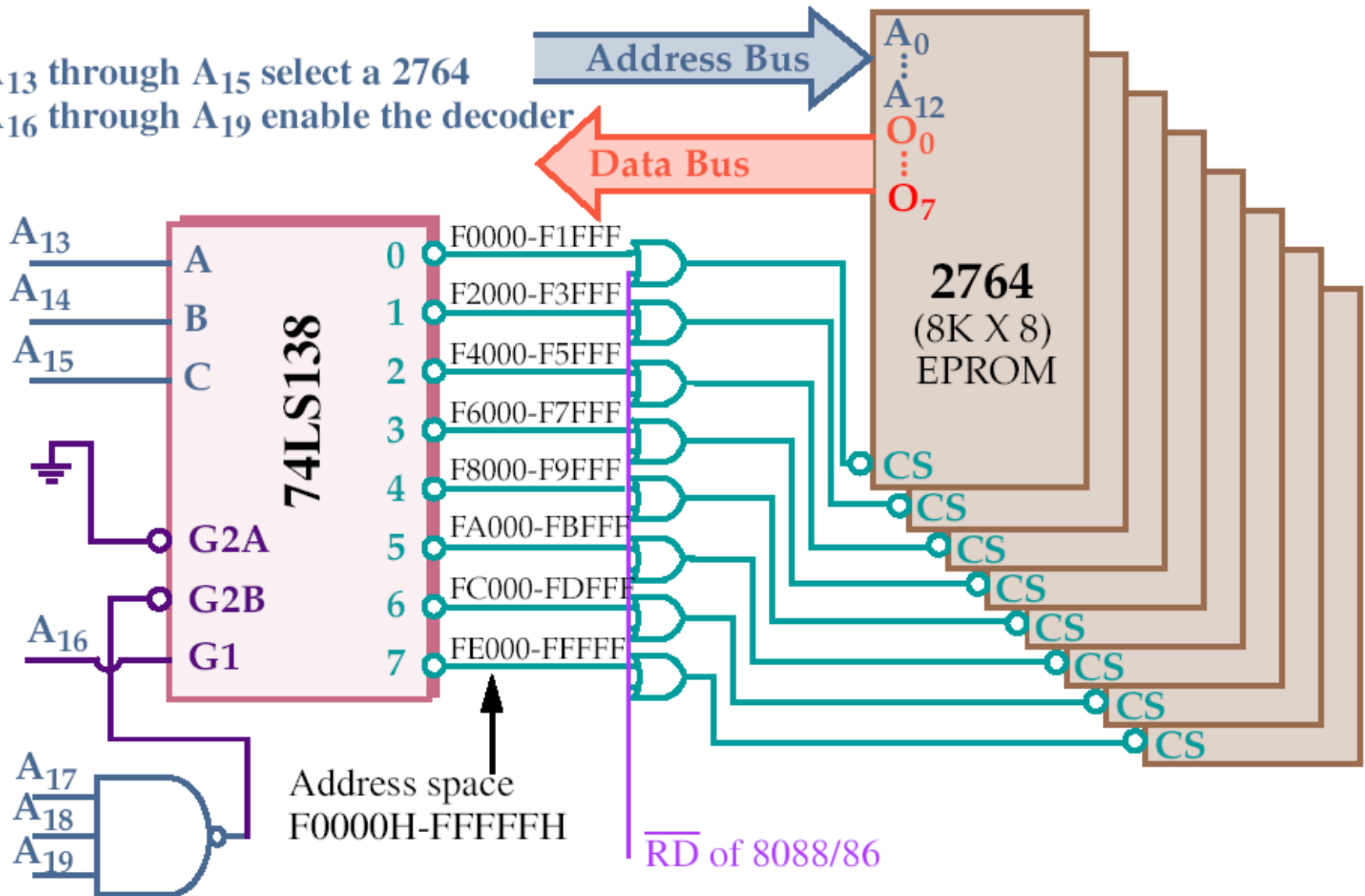


Inputs						Output							
Enable			Select										
G2A	G2B	G1	C	B	A	0	1	2	3	4	5	6	7
1	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	0	X	X	X	1	1	1	1	1	1	1	1
0	0	1	0	0	0	0	1	1	1	1	1	1	1
0	0	1	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	0	1	1	0	1	1	1	1	1
0	0	1	0	1	1	1	1	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	1	0	1	1	1
0	0	1	1	0	1	1	1	1	1	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	1	0

Note that all *three* Enables (G2A, G2B, and G1) must be active, e.g. low, low and high, respectively.

Each output of the decoder can be attached to an 2764 EPROM (8K X 8).

A_{13} through A_{15} select a 2764
 A_{16} through A_{19} enable the decoder



The EPROMs cover a 64KB section of memory.

PLD Programmable Decoders

- Three SPLD (**simple PLD**) devices function in the same manner but have different names:
 - PLA (**programmable logic array**)
 - PAL (**programmable array logic**)
 - GAL (**gated array logic**)
- In existence since the mid-70s, they have appeared in memory system and digital designs since the early 1990s.

- PAL and PLA are fuse-programmed, and some PLD devices are erasable devices.
 - all are arrays of programmable logic elements
- Other PLDs available:
 - CPLDs (**complex programmable logic devices**)
 - FPGAs (**field programmable gate arrays**)
 - FPICs (**field programmable interconnect**)
- These PLDs are more complex than the SPLDs used more commonly in designing a complete system.

- If the concentration is on decoding addresses, the SPLD is used.
- If the concentration is on a complete system, then the CPLD, FPGA, or FPIC is used to implement the design.
- These devices are also referred to as an ASIC (**application-specific integrated circuit**).

Error Detection and Correction

- ***Parity, BCC (Block-Check Character) and CRC (Cyclic Redundancy Check)*** are only mechanisms for error detection.
- The system is halted if an error is found in memory.
- Error *correction* is starting to show up in new systems.
- ***SDRAM*** has ***ECC (Error Correction Code)***.
- Correction will allow the system can continue operating.
- If *two* errors occur, they can be *detected* but not *corrected*.
- Error correction will of course cost more in terms of extra bits.

Error Correction

- Error correction is based on Hamming codes
- Error detection and correction
- Presumes one bit error most likely
- Detects error and position
therefore allows for correction

Hamming Code Example

Data is on bits 3, 5, 6, 7, 9, 10, 11, 12

Bit position

1	2	3	4	5	6	7	8	9	10	11	12
P_1	P_2	1	P_4	1	0	0	P_8	0	1	0	0

$$P_1 = \text{XOR of bits (3, 5, 7, 9, 11)} = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$P_2 = \text{XOR of bits (3, 6, 7, 10, 11)} = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$P_4 = \text{XOR of bits (5, 6, 7, 12)} = 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

$$P_8 = \text{XOR of bits (9, 10, 11, 12)} = 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

Bit position

1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	1	0	0	1	0	1	0	0

This is what is transmitted or stored

Note that parity bits are in power of 2 positions

Hamming Code Example

If data received or read from memory is correct

Bit position	1	2	3	4	5	6	7	8	9	10	11	12
	0	0	1	1	1	0	0	1	0	1	0	0

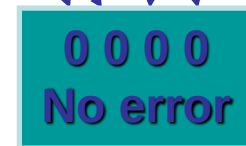
4 check bits are evaluated as follows:

$$C_1 = \text{XOR of bits (1, 3, 5, 7, 9, 11)} = 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$C_2 = \text{XOR of bits (2, 3, 6, 7, 10, 11)} = 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$C_4 = \text{XOR of bits (4, 5, 6, 7, 12)} = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$C_8 = \text{XOR of bits (8, 9, 10, 11, 12)} = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$



0000
No error

Hamming Code Example

If data received or read from memory is not correct

Bit position	1	2	3	4	5	6	7	8	9	10	11	12
	0	0	1	1	1	0	0	1	0	1	0	0

If read incorrectly (change bit 3 from a 1 to 0)

Bit position	1	2	3	4	5	6	7	8	9	10	11	12
	0	0	0	1	1	0	0	1	0	1	0	0

4 check bits are evaluated as follows:

$$C_1 = \text{XOR of bits (1, 3, 5, 7, 9, 11)} = 0 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

$$C_2 = \text{XOR of bits (2, 3, 6, 7, 10, 11)} = 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$C_4 = \text{XOR of bits (4, 5, 6, 7, 12)} = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$C_8 = \text{XOR of bits (8, 9, 10, 11, 12)} = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

0 0 1 1
Error
in bit 3



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

I/O Interface

Introduction

- Basic I/O interface
- Handshaking process
- Serial and Parallel communication
- I/O interface examples

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Explain the operation of the basic input and output interfaces.
- Use I/O instructions.
- Define handshaking and explain how to use it with I/O devices.

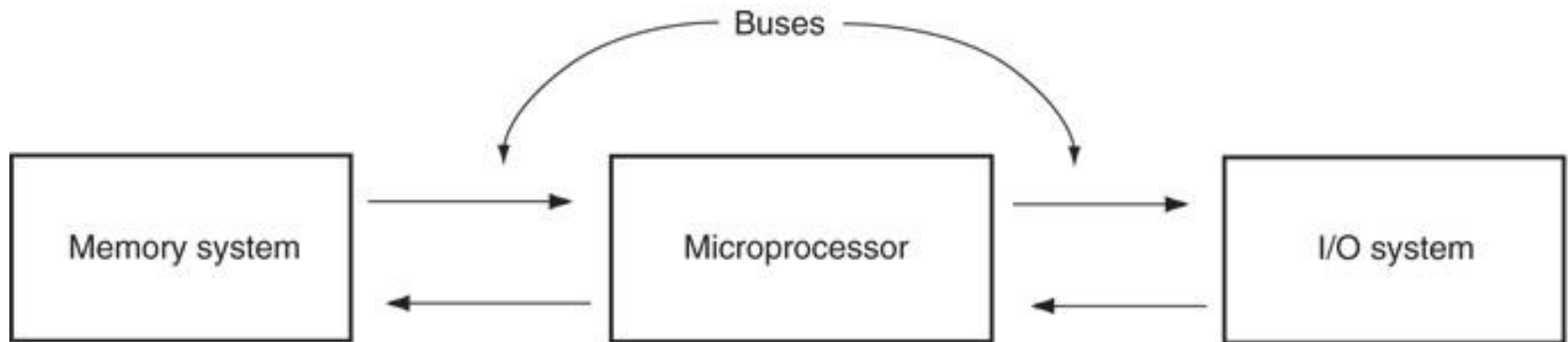
Chapter Objectives

Upon completion of this chapter, you will be able to:

- Learn the differences between serial and parallel communication.
- Understand how to interface external devices to the microprocessor via I/O ports.
- Use virtual devices provided with the emulator.

INTRO TO I/O INTERFACE

- A microprocessor is great at solving problems.
- But, if it can not communicate with the outside world, it is of little worth.
- Therefore, I/O interface is of great importance.



INTRO TO I/O INTERFACE

- I/O instructions (IN, INS, OUT, and OUTS) are explained in this lecture.
- Also isolated (direct or I/O mapped I/O) and memory-mapped I/O, the basic input and output interfaces, and handshaking.
- Knowledge of these topics makes it easier to understand the connection and operation of the programmable interface components and I/O techniques.

The I/O Instructions

- One type of instruction transfers information to an I/O device (OUT).
- Another reads from an I/O device (IN).
- Instructions are also provided to transfer strings of data between memory and I/O.
 - INS and OUTS, found except the 8086/8088

- Instructions that transfer data between an I/O device and the microprocessor's accumulator (**AL**, **AX**, or EAX) are called **IN** and **OUT**.
- The I/O address is stored in register DX as a 16-bit address or in the byte (p8) immediately following the opcode as an 8-bit address.
- The 16-bit address is called a **variable address** because it is stored in DX, and then used to address the I/O device.

- Other instructions that use DX to address I/O are the INS and OUTS instructions.
- I/O ports are 8 bits in width.
 - a 16-bit port is actually two consecutive 8-bit ports being addressed
 - a 32-bit I/O port is actually four 8-bit ports

- When data are transferred using IN or OUT, the I/O address, (**port number** or simply port), appears on the address bus.
- External I/O interface decodes the port number in the same manner as a memory address.
 - the 8-bit fixed port number (p8) appears on address bus connections A_7 – A_0 with bits A_{15} – A_8 equal to 00000000_2
 - connections above A_{15} are undefined for I/O instruction

- The 16-bit variable port number (DX) appears on address connections A_{15} – A_0 .
- The first 256 I/O port addresses (00H–FFH) are accessed by both fixed and variable I/O instructions.
 - any I/O address from 0100H to FFFFH is only accessed by the variable I/O address (via DX)
- In a PC computer, all 16 address bus bits are decoded with locations 0000H–03FFH.
 - used for I/O inside the PC on the ISA (**industry standard architecture**) bus

- INS and OUTS instructions address an I/O device using the DX register.
 - but do not transfer data between accumulator and I/O device as do the IN/OUT instructions
 - Instead, they transfer data between memory and the I/O device
- Pentium 4 and Core2 operating in the 64-bit mode have the same I/O instructions.
- There are no 64-bit I/O instructions in the 64-bit mode.
 - most I/O is still 8 bits and likely will remain so

Summary of I/O instructions

- IN AL, p8
- IN AX, p8
- IN AL, DX
- IN AX, DX
- OUT p8, AL
- OUT p8, AX
- OUT DX, AL
- OUT DX, AX

p8: A port number between 0 and 255
For other port numbers, use DX register.

Isolated (direct) and Memory-Mapped I/O

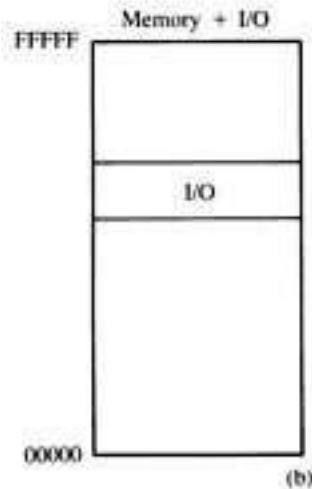
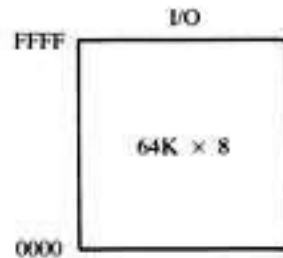
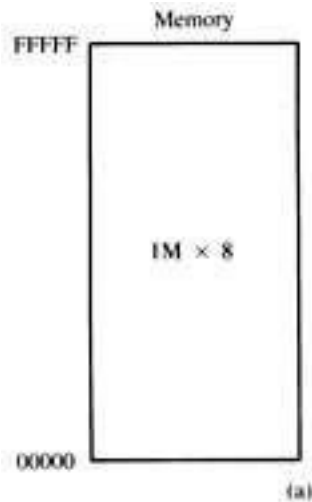
- Two different methods of interfacing I/O: **isolated I/O** and **memory-mapped I/O**.
- In isolated I/O, the IN, INS, OUT, and OUTS transfer data between the microprocessor's accumulator or memory and the I/O device.
- In memory-mapped I/O, any instruction that references memory can accomplish the transfer.
- The PC does not use memory-mapped I/O.

Isolated I/O

- The most common I/O transfer technique used in the Intel-based system is isolated I/O.
 - *isolated* describes how I/O locations are isolated from memory in a separate I/O address space
- Addresses for isolated I/O devices, called ports, are separate from memory.
- Because the ports are separate, the user can expand the memory to its full size without using any of memory space for I/O devices.

- A disadvantage of isolated I/O is that data transferred between I/O and microprocessor must be accessed by the IN, INS, OUT, and OUTS instructions.
- Separate control signals for the I/O space are developed (using $M/\overline{IO}$ and $W/\overline{R}$), which indicate an I/O read ($\overline{IORC}$) or an I/O write ($\overline{RD}$) operation.
- These signals indicate an I/O port address, which appears on the address bus, is used to select the I/O device.

Figure 11–1 The memory and I/O maps for the 8086/8088 microprocessors. (a) Isolated I/O. (b) Memory-mapped I/O.

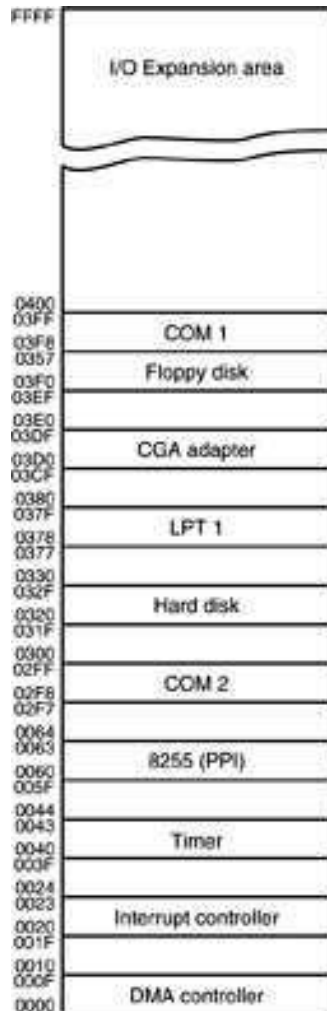


- in the PC, isolated I/O ports are used to control peripheral devices
- an 8-bit port address is used to access devices located on the system board, such as the timer and keyboard interface
- a 16-bit port is used to access serial and parallel ports, video and disk drive systems

Memory-Mapped I/O

- Memory-mapped I/O does not use the IN, INS, OUT, or OUTS instructions.
- It uses any instruction that transfers data between the microprocessor and memory.
 - treated as a memory location in memory map
- Advantage is any memory transfer instruction can access the I/O device.
- Disadvantage is a portion of memory system is used as the I/O map.
 - reduces memory available to applications

Personal Computer I/O Map



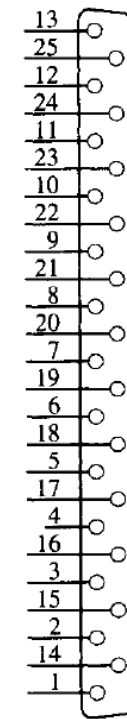
- the PC uses part of I/O map for dedicated functions, as shown here
- I/O space between ports 0000H and 03FFH is normally reserved for the system and ISA bus
- ports at 0400H–FFFFH are generally available for user applications, main-board functions, and the PCI bus
- 80287 coprocessor uses 00F8H–00FFH, so Intel reserves I/O ports 00F0H–00FFH

Figure 11–2 I/O map of a personal computer illustrating many of the fixed I/O areas.

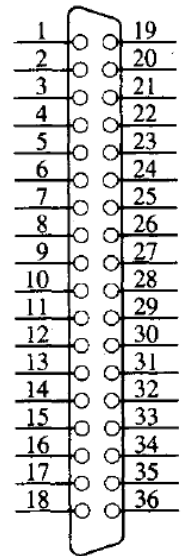
Handshaking

- Many I/O devices accept or release information slower than the microprocessor.
- A method of I/O control called **handshaking** or **polling**, synchronizes the I/O device with the microprocessor.
- An example is a parallel printer that prints a few hundred characters per second (CPS).
- The processor can send data much faster.
 - a way to slow the microprocessor down to match speeds with the printer must be developed

- Fig 11–5 illustrates typical input output connections found printer.
 - data transfers via data connections
- ASCII data are placed on D_7 – D_0 , pulse is then applied to STB connection.
 - BUSY indicates the printer is busy
 - STB is a clock pulse used to send data to printer
- The strobe signal sends or clocks the data into the printer so that they can be printed.
 - as the printer receives data, it places logic 1 on the BUSY pin, indicating it is printing data

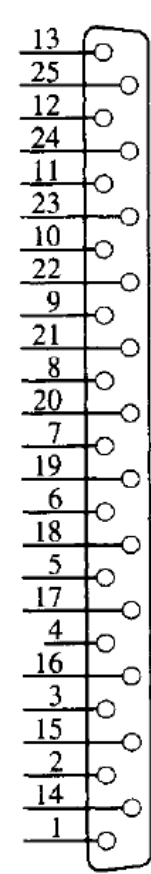


Connector DB25

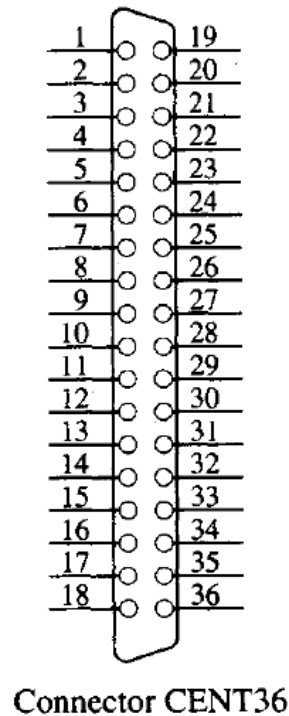


Connector CENT36

Figure 11-5 The DB25 connector found on computers and the Centronics 36-pin connector found on printers for the Centronics parallel printer interface.



Connector DB25



Connector CENT36

DB25 Pin number	CENT36 Pin number	Function	DB25 Pin number	CENT36 Pin number	Function
1	1	Data Strobe	12	12	Paper empty
2	2	Data 0 (D0)	13	13	Select
3	3	Data 1 (D1)	14	14	Afd
4	4	Data 2 (D2)	15	32	Error
5	5	Data 3 (D3)	16	—	RESET
6	6	Data 4 (D4)	17	31	Select in
7	7	Data 5 (D5)	18—25	19—30	Ground
8	8	Data 6 (D6)	—	17	Frame ground
9	9	Data 7 (D7)	—	16	Ground
10	10	Ack	—	33	Ground
11	11	Busy			

- The software polls or tests the BUSY pin to decide whether the printer is busy.
 - If the printer is busy, the processor waits
 - if not, the next ASCII character goes to the printer
- This process of interrogating the printer, or any asynchronous device like a printer, is called **handshaking** or **polling**.

I/O COMMUNICATION METHODS

1. Serial communication
 2. Parallel communication
- Serial communication is the process of sending data one bit at one time, sequentially, over a communication channel or computer bus.
 - In parallel communication, on the other hand, several bits are sent together on a link comprising of several wired channels in parallel.

Examples of serial communication architectures

- 
- RS-232
 - I²C (Inter Integrated Circuit)
 - SPI (Serial Peripheral Interface)
 - USB (Universal Serial Bus)
 - FireWire
 - Ethernet
 - Fibre Channel
 - SATA (Serial Advanced Technology Attachment)
 - PCI (Peripheral Component Interconnect) Express

Examples of parallel communication architectures



- Printer port
- ISA (Industry Standard Architecture)
- ATA (Advanced Technology Attachment)
- SCSI (Small Computer System Interface)
- PCI

Serial vs Parallel

- Serial circuitry is simple and cheap.
- Parallel circuitry is complex and expensive.
- In parallel communication, you can send many bits at once; however, receive 'at once' is not a true statement.
- Each bit travels a slightly different path down the cable and to a different pin on the chip .
- The small differences in path length that each bit travels becomes more and more significant as speed increases.
- Therefore, some sort of buffering and synchronization is necessary.
- In serial communication, no such a problem exists.
- So, after a certain speed and distance threshold, serial communication is superior.

Interfacing processors to serial and parallel devices

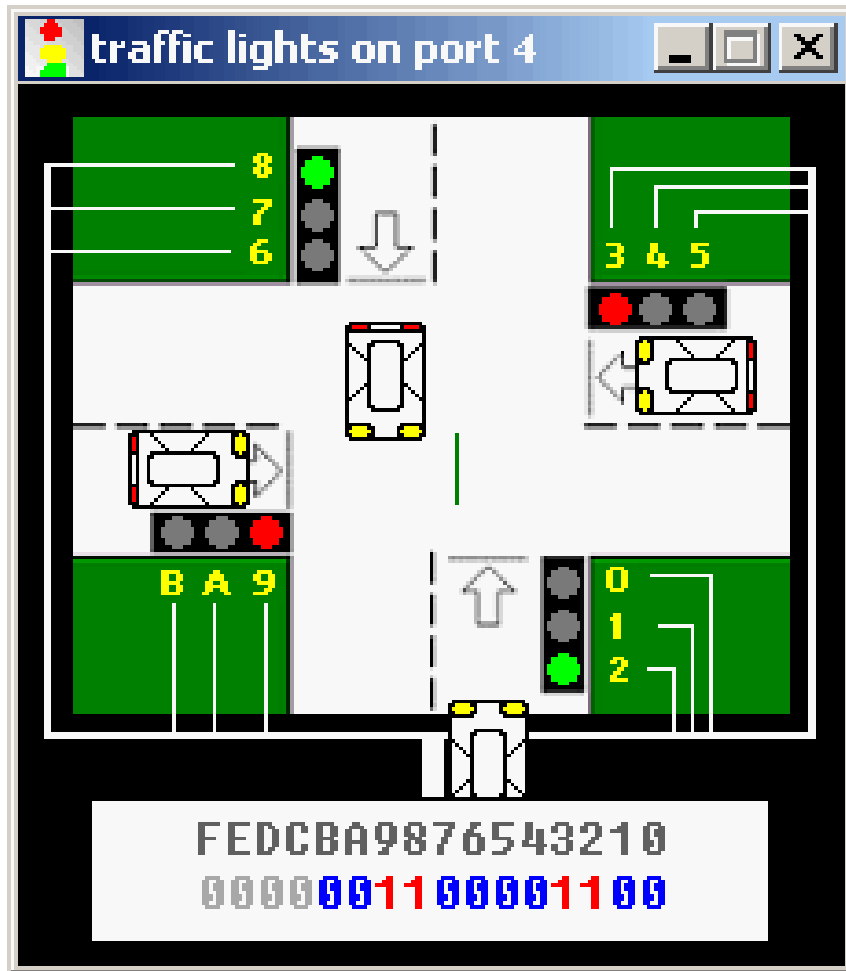
- Processors use peripheral interface circuits to interface to serial and parallel devices.
- These circuits are either provided as separate integrated circuits or built inside the processor.

Examples:

- A UART (Universal asynchronous receiver/transmitter) is used for **serial** communications over a computer or peripheral device serial port.
- Intel 8255 chip is used to give the CPU access to programmable **parallel** I/O.

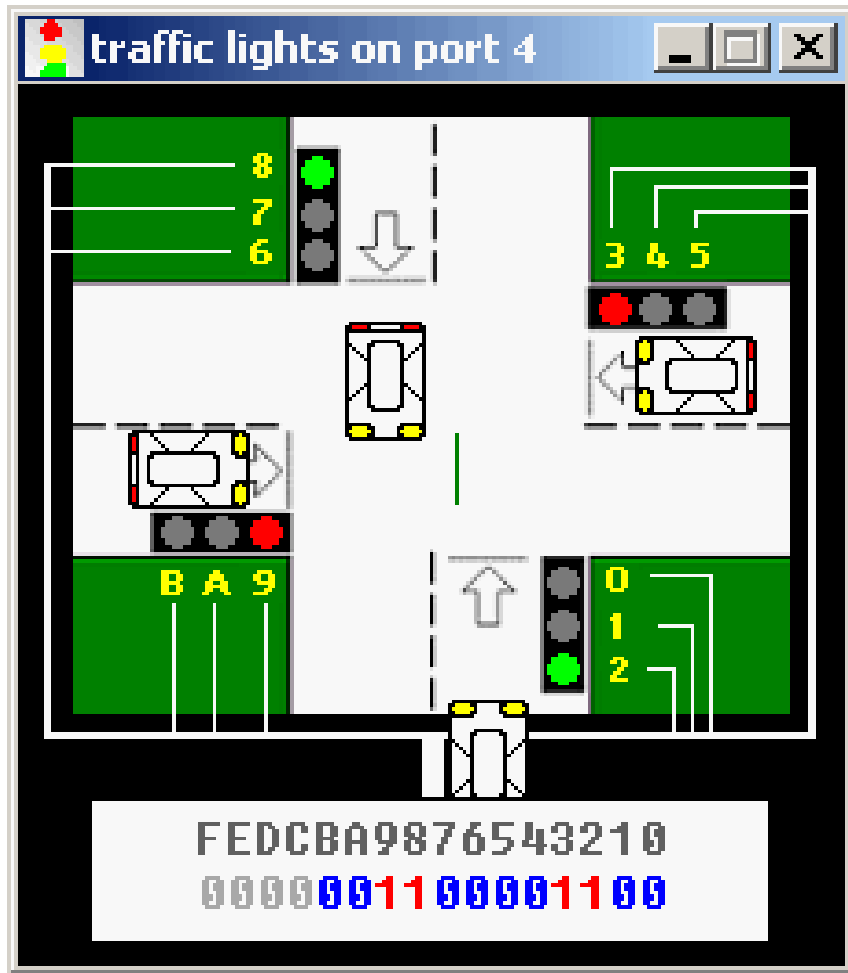
I/O INTERFACE EXAMPLES

Traffic Lights Emulation



- Traffic lights are controlled by sending data to I/O port 4.
- There are 12 lamps:
 - 4 green
 - 4 yellow
 - 4 red.

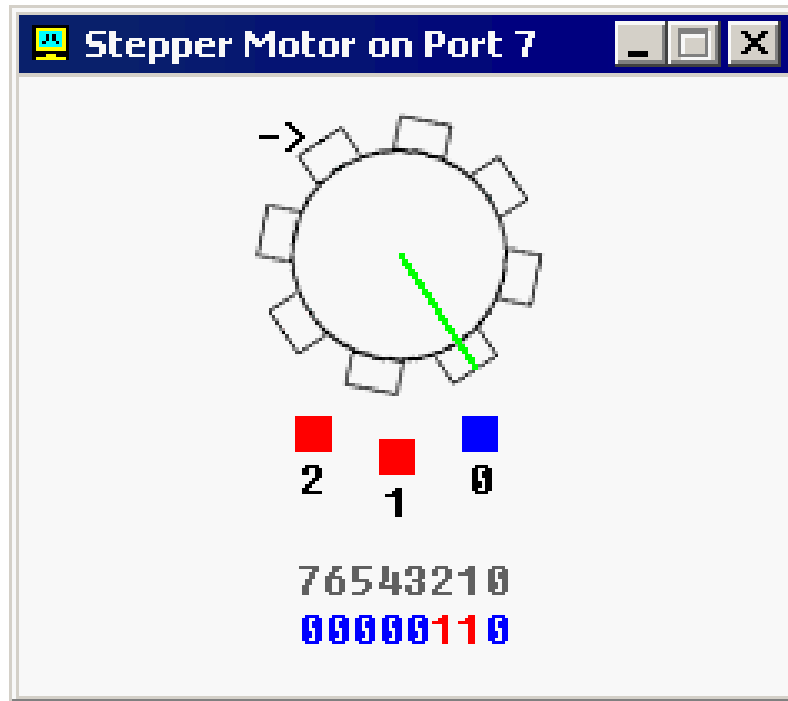
Traffic Lights Emulation



- You can set the state of each lamp by setting its bit:
1 - the lamp is turned on.
0 - the lamp is turned off.
- Bits (12 to 15) are unused.
- For example, all lights will be red when following code is executed:

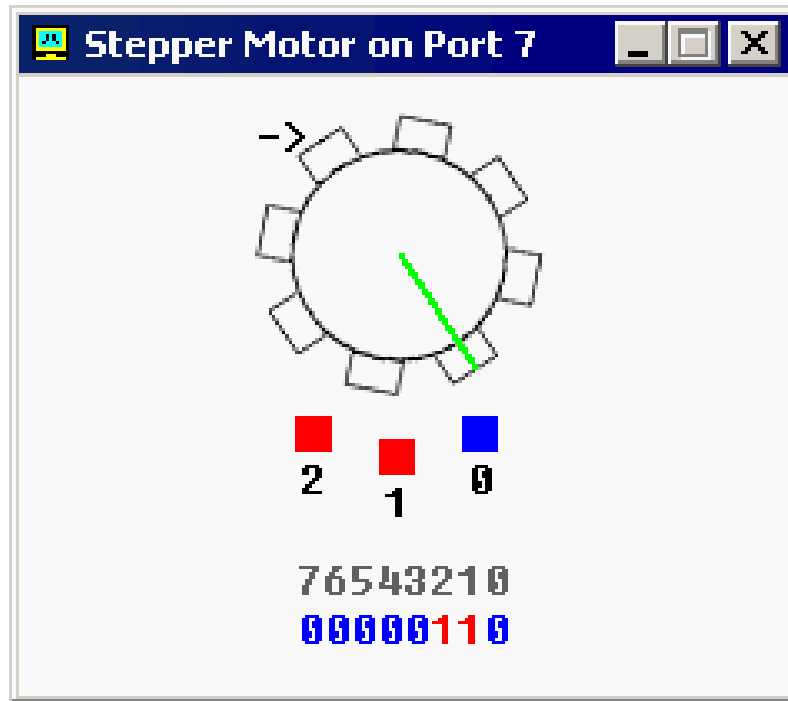
```
MOV AX, 0000001001001001b  
OUT 4, AX
```

Stepper Motor



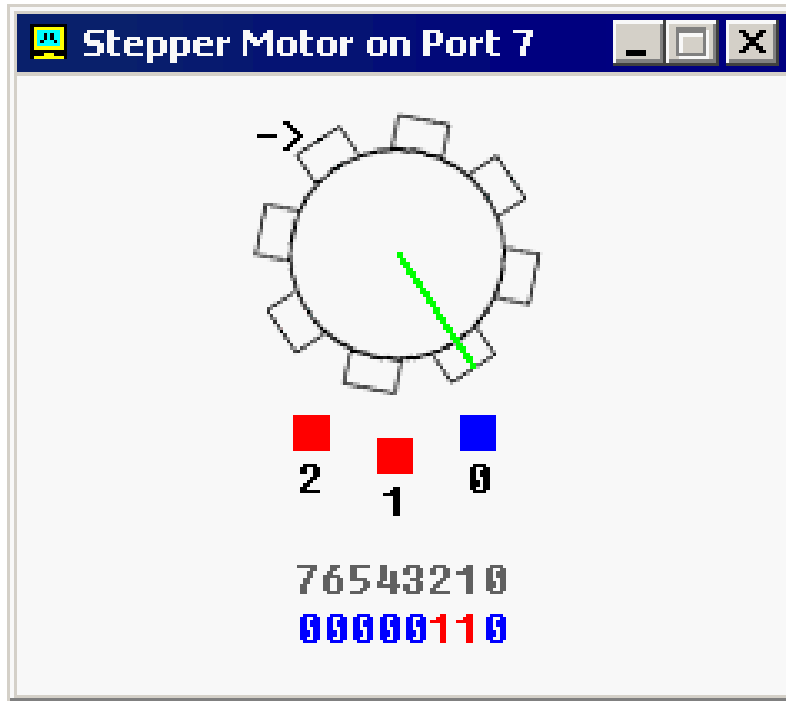
- The stepper motor is controlled by sending data to I/O port 7.
- Stepper motor is electric motor that can be precisely controlled by signals from a computer.
- The motor turns through a precise angle each time it receives a signal.

Stepper Motor



- By varying the rate at which signal pulses are produced, the motor can be run at different speeds or turned through an exact angle and then stopped.
- This is a basic 3-phase stepper motor, it has 3 magnets controlled by bits **0, 1 and 2**. other bits (3..7) are unused.

Stepper Motor



- When magnet is working it becomes red.
- The arrow in the left upper corner shows the direction of the last motor move.
- Green line is here just to see that it is really rotating.

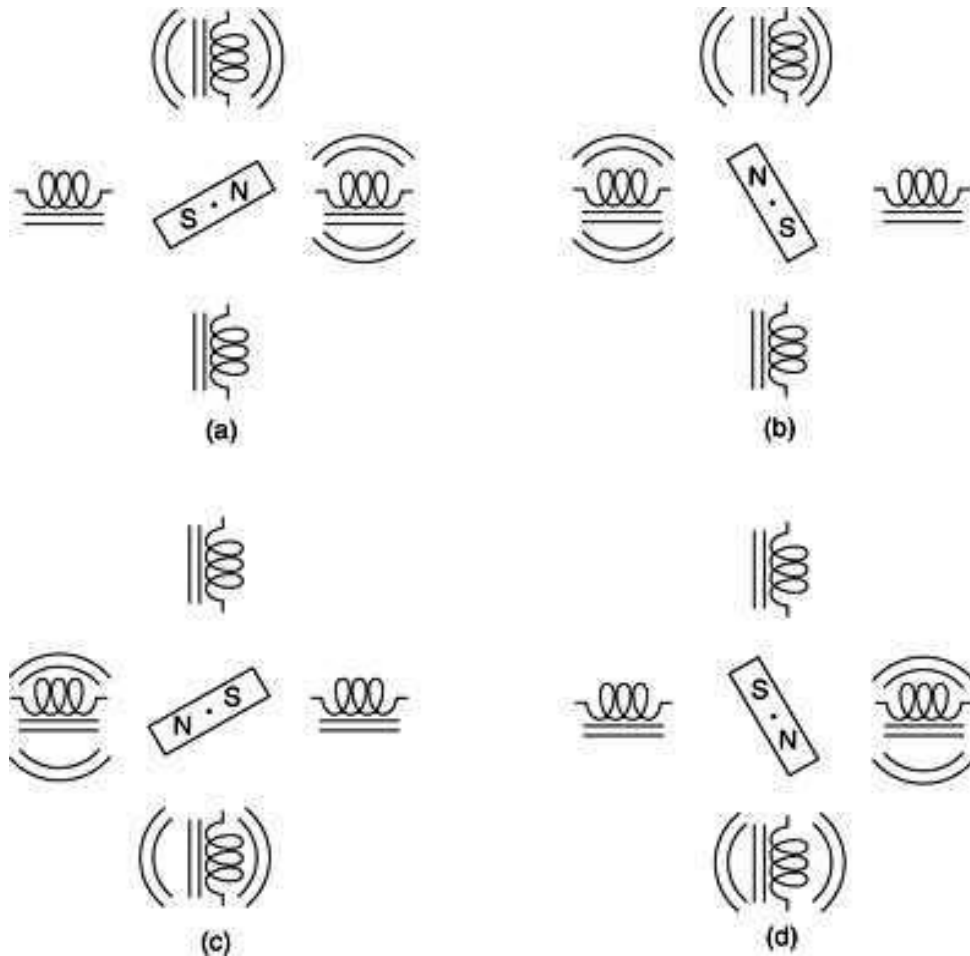
Stepper Motor

- If you ever played with magnets you will understand how it works.
- Just activate the magnets in a precise order.
- For example, the code below will do three clock-wise half-steps:

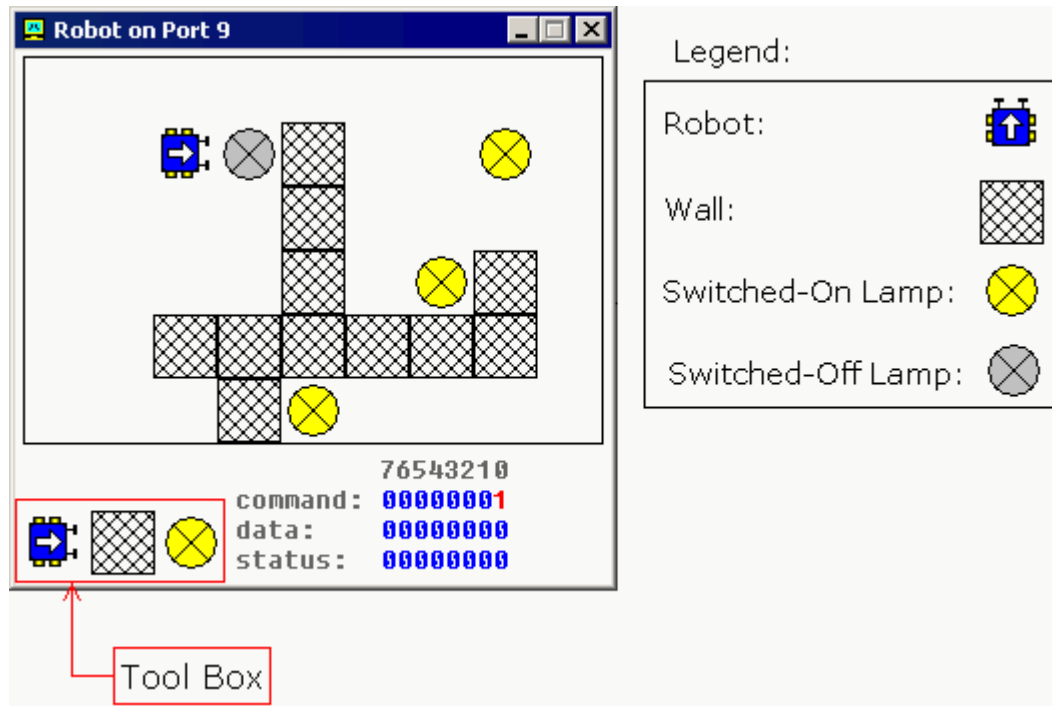
`MOV AL, 001b ; initialize`
`OUT 7, AL`
`MOV AL, 011b ; half step 1`
`OUT 7, AL`
`MOV AL, 010b ; half step 2`
`OUT 7, AL`
`MOV AL, 110b ; half step 3`
`OUT 7, AL`

Stepper Motor

Four-Coil
Stepper Motor
that uses an
Armature with
a Single Pole



Robot



The robot is controlled by sending data to I/O port 9.

Robot

- The first byte (port 9) is a **command register**.
- Send data to this port to make the robot do something.

decimal value	binary value	action
0	00000000	do nothing.
1	00000001	move forward.
2	00000010	turn left.
3	00000011	turn right.
4	00000100	examine. examines an object in front using sensor. when robot completes the task, result is set to data register and bit #0 of status register is set to 1 .
5	00000101	switch on a lamp.
6	00000110	switch off a lamp.

Robot

- The second byte (port 10) is a **data register**.
- This register is set after robot completes the **examine** command:

decimal value	binary value	meaning
255	11111111	wall
0	00000000	nothing
7	00000111	switched-on lamp
8	00001000	switched-off lamp

Robot

- The third byte (port 11) is a **status register**.
- Read values from this port to determine the state of the robot.
- Each bit has a specific property:

bit number	description
bit #0	zero when there is no new data in data register , one when there is new data in data register .
bit #1	zero when robot is ready for next command, one when robot is busy doing some task.
bit #2	zero when there is no error on last command execution, one when there is an error on command execution (when robot cannot complete the task: move, turn, examine, switch on/off lamp).

Robot

- Example:

```
MOV AL, 1 ; move forward.  
OUT 9, AL ;  
MOV AL, 3 ; turn right.  
OUT 9, AL ;  
MOV AL, 1 ; move forward.  
OUT 9, AL ;  
MOV AL, 2 ; turn left.  
OUT 9, AL ;  
MOV AL, 1 ; move forward.  
OUT 9, AL ;
```

Robot

- Keep in mind that robot is a mechanical creature and it takes some time for it to complete a task.
- You should always check bit#1 of **status register** before sending data to port 9.
- Otherwise the robot will reject your command and "**busy!**" will be shown.
- Remember handshaking process!



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Interrupts

Introduction

- In this chapter, the coverage of basic I/O and programmable peripheral interfaces is expanded by examining a technique called interrupt-processed I/O.
- An interrupt is a hardware-initiated procedure that interrupts whatever program is currently executing.
- This chapter provides examples and a detailed explanation of the interrupt structure of the Intel family of microprocessors.

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Explain the interrupt structure of the Intel family of microprocessors.
- Explain the operation of software interrupt instructions INT, INTO, IRET
- Explain how the interrupt enable flag bit (IF) modifies the interrupt structure.
- Learn the interrupts supported by the emulator.

INTERRUPT CONCEPT

Example 1: Man in office

- Imagine a man working at his office. During his work, he may be subject to different interrupts.
- He may choose to deal with, postpone or ignore the interrupt.
- For example:
 - A friend comes in and asks for a favor. Since he is busy, tells the friend to come back later.
 - Telephone rings, but he ignores the phone.
 - Some interrupts should not be ignored - fire alarm!
- We need to be able to respond to interrupts properly.

Example 2: Bus driver

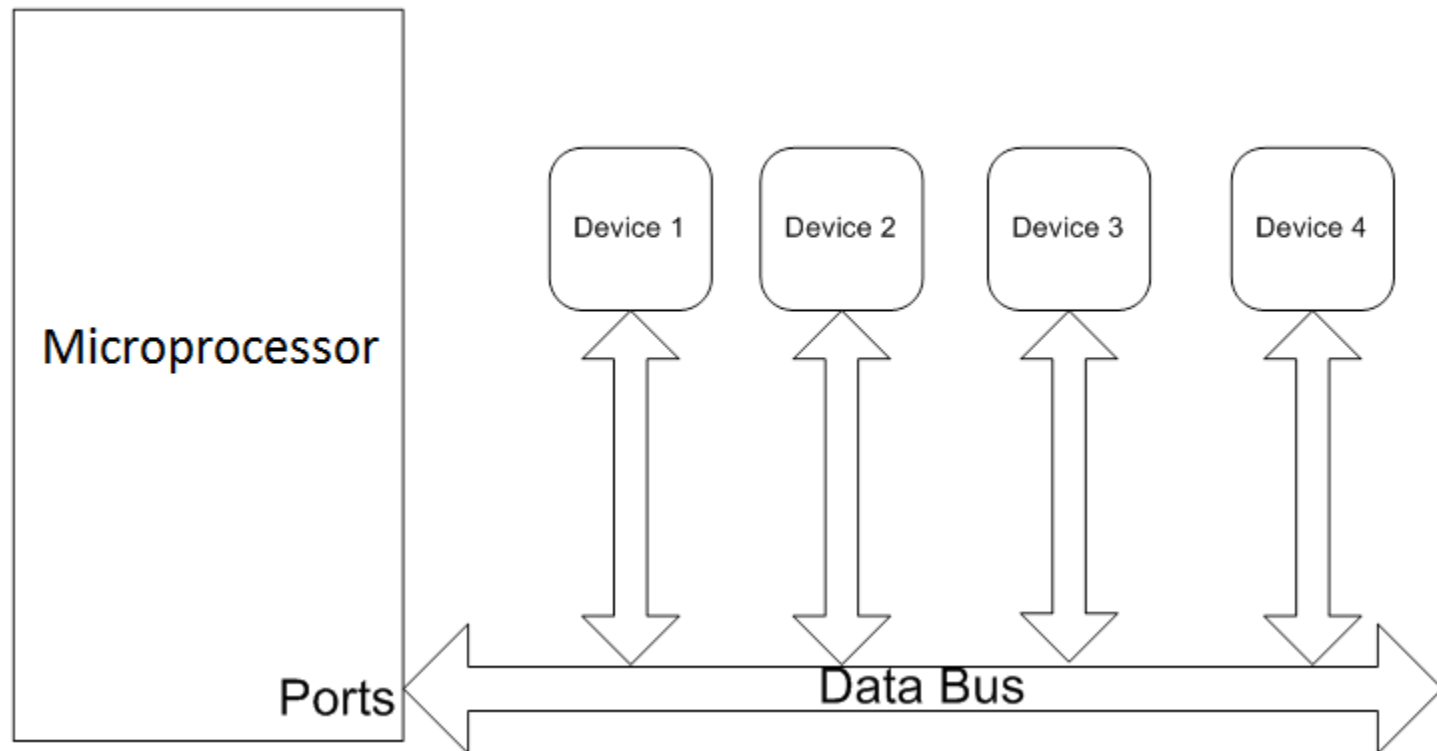
- The bus driver is normally driving the bus.
- If interrupted by any of the passengers who presses the “STOP” button, he will;
 - a) Pull over the bus at the next bus stop
 - b) Open door
 - c) Wait for passenger(s) to leave
 - d) Close door
 - e) Continue driving
- As a summary, he will initiate a fixed sequence of responses when interrupted, then continue to drive.
- One of the major tasks of a processor is to handle **interrupts**.

INTERRUPTS and I/O

- Synchronisation of tasks is a very important function on a computer.
- Visualise all the components in a typical computer trying to communicate with the processor and the processor trying to communicate with them.
- A fast computer normally has to interact with relatively slow I/O devices, slow because they are mechanical, or because they have to work at a human friendly speeds.
- Two ways of interaction
 - Polling (Handshaking)
 - Interrupt

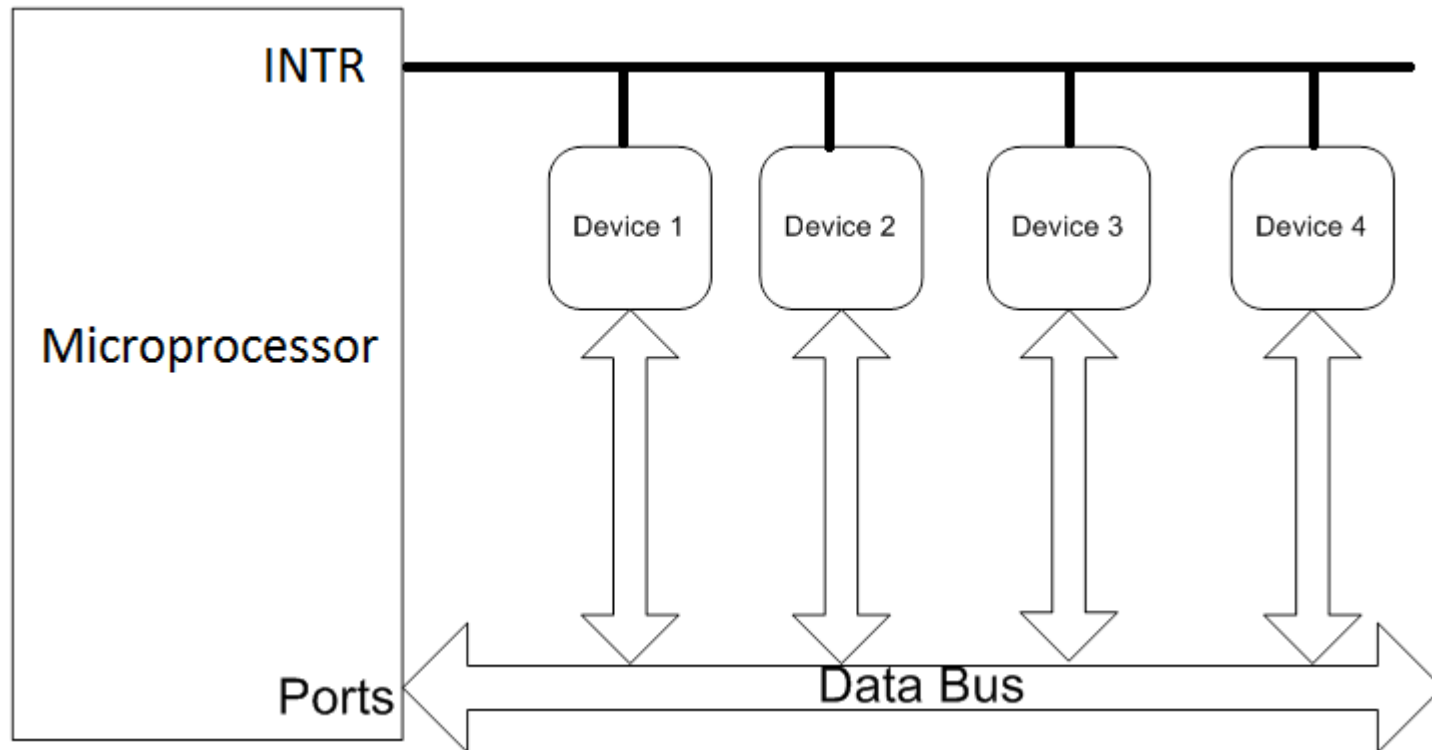
Polling

- “Ask” each device sequentially if it needs service. Note, no devices may need servicing during the poll.



Interrupt

- Device “interrupts” CPU to indicate that it needs service.



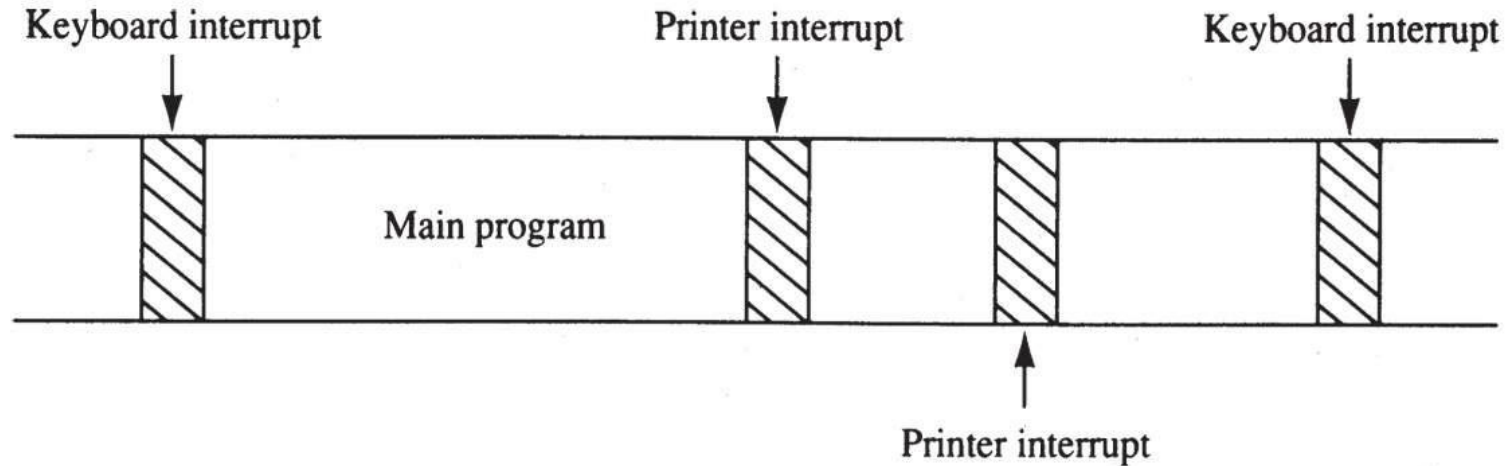
Which method is good?

- So, the computer need not to “freeze” while waiting for an input. It can go off and execute other programs, etc.
- In fact a program that is waiting for an input or output to complete requires very little attention from the CPU. The CPU’s attitude can be summed up as “Get back to me when you have something”.
- This is very important for multi-tasking systems. While one task is waiting for I/O to complete, another can use CPU cycles. And most programs spend most of their time waiting on I/O.
- For example, if you print a file from a Word Processor, transfer of characters to the printer takes place in the background using interrupts. You can still type away (and run other programs).

What needs to interrupt?

- External Devices
 - Keyboard, Mouse, Sensors, etc.
- Internal Events
 - Reset, Illegal Operation, User Interrupt, etc.

Figure 12–1 A time line that indicates interrupt usage in a typical system.



- a time line shows typing on a keyboard, a printer removing data from memory, and a program executing
- the keyboard interrupt service procedure, called by the keyboard interrupt, and the printer interrupt service procedure each take little time to execute

Types of Interrupts

- **Software interrupt** is an interrupt generated within a processor by executing an instruction. Software interrupts are often used to implement system calls.
- **Hardware interrupt** is an interrupt generated by hardware such as I/O devices or peripherals
- **Maskable interrupt**: is a hardware interrupt that may be ignored.
- **Non-maskable interrupt (NMI)** is a hardware interrupt that can never be ignored. NMIs are used for the highest priority tasks such as timers, reset, etc.

Interrupts on Intel

- Intel processors include two hardware pins (INTR and NMI) that request interrupts.
- And one hardware pin ($\overline{\text{INTA}}$) to acknowledge the interrupt requested through INTR.
- The processor also has software interrupts such as INT, INTO
- Flag bit IF (interrupt flag) is also used with the interrupt structure and special return instruction IRET
 - IRETD in the 80386, 80486, or Pentium

Interrupt Vectors

- Interrupt vectors and the vector table are crucial to an understanding of hardware and software interrupts.
- The **interrupt vector table** is located in the first 1024 bytes of memory at addresses 000000H–0003FFH.
 - contains 256 different four-byte interrupt vectors
- An interrupt vector contains the address (segment and offset) of the interrupt service routine/procedure (ISR).

Interrupt Service Routine (ISR)

- ISR is a special subroutine that is designed to “service” the interrupt
 - Also called an “interrupt handler”
- In other words, ISR contains the subroutine that is to be executed when a particular interrupt occurs.

Figure 12–2 (a) The interrupt vector table for the microprocessor and (b) the contents of an interrupt vector.

	Type 32 — 255 User interrupt vectors
080H	Type 14 — 31 Reserved
040H	Type 16 Coprocessor error
03CH	Type 15 Unassigned
038H	Type 14 Page fault
034H	Type 13 General protection
030H	Type 12 Stack segment overrun
02CH	Type 11 Segment not present
028H	Type 10 Invalid task state segment
024H	Type 9 Coprocessor segment overrun
020H	Type 8 Double fault
01CH	Type 7 Coprocessor not available
018H	Type 6 Undefined opcode
014H	Type 5 BOUND
010H	Type 4 Overflow (INTO)
00CH	Type 3 1-byte breakpoint
008H	Type 2 NMI pin
004H	Type 1 Single-step
000H	Type 0 Divide error

(a)

- the first five interrupt vectors are identical in all Intel processors
- Intel reserves the first 32 interrupt vectors
- the last 224 vectors are user-available
- each is four bytes long in real mode and contains the starting address of the interrupt service procedure.
- the first two bytes contain the offset address
- the last two contain the segment address

	Any interrupt vector
3	Segment (high)
2	Segment (low)
1	Offset (high)
0	Offset (low)

(b)

Software Interrupt Instructions

- Common software interrupt instructions are available to the microprocessor:
- INT is unconditional
- INTO is conditional.
- IRET is a special interrupt return instruction.

- INTO checks or tests the overflow flag (O).
 - If $O = 1$, INTO calls the procedure whose address is stored in interrupt vector type 4
 - If $O = 0$, INTO performs no operation and the next sequential program instruction executes
- The INT n instruction calls the interrupt service procedure at the address represented in vector number n .
- The IRET instruction is a special return instruction used to return for both software and hardware interrupts.
 - much like a far RET, it retrieves the return address from the stack

Operation of a Real Mode Interrupt

- When the processor completes executing the current instruction, it determines whether an interrupt is active by checking:
 - (1) instruction executions
 - (2) single-step
 - (3) NMI
 - (4) coprocessor segment overrun
 - (5) INTR
 - (6) INT instructions in the order presented

- If one or more are present:
 - 1. Flag register contents are pushed on the stack.
 - 2. Interrupt flag is cleared, disabling the INTR pin.
 - 3. Contents of the code segment register (CS) are pushed onto the stack.
 - 4. Contents of the instruction pointer (IP) are pushed onto the stack.
 - 5. Interrupt vector contents are fetched and placed into IP and CS so the next instruction executes at the interrupt service procedure addressed by the vector.
 - 6. When ISR is done, previous status is restored from the stack.

Interrupt Flag Bits

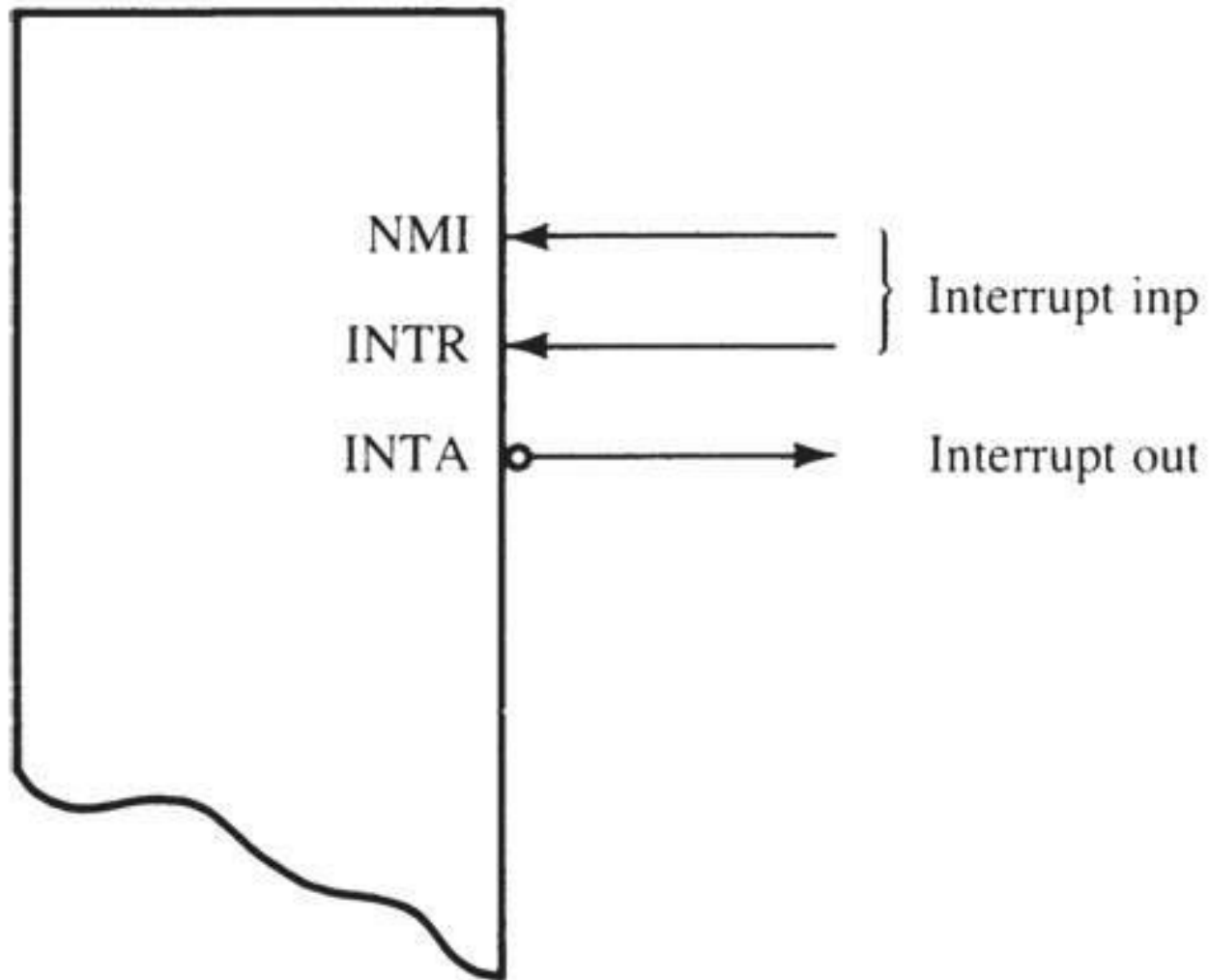
- The interrupt flag (IF) is cleared after the contents of the flag register are stacked during an interrupt.
- When IF is set, it *allows* the INTR pin to cause an interrupt
- When IF is cleared, it *prevents* the INTR pin from causing an interrupt

Hardware Interrupts

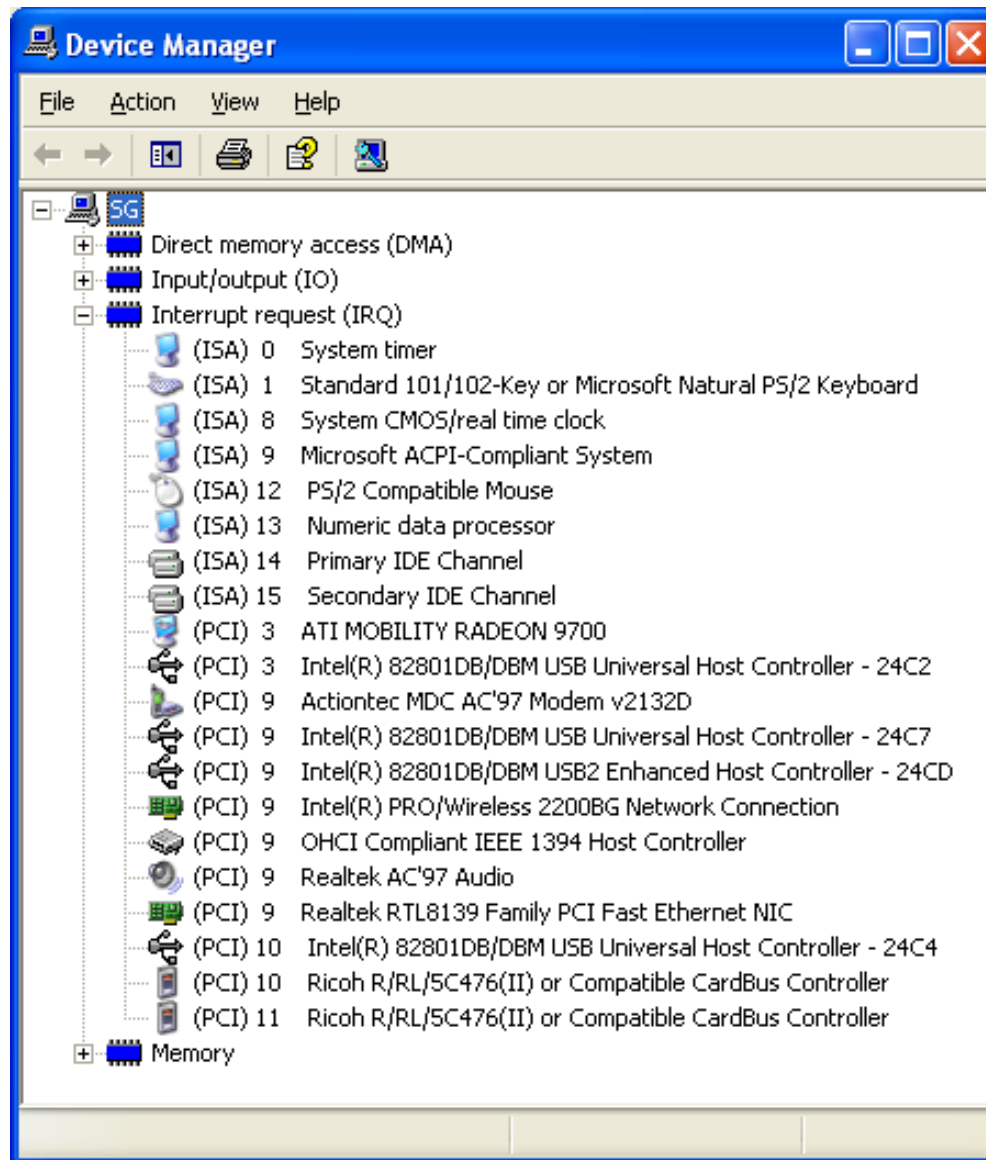
- The two hardware interrupt inputs:
 - non-maskable interrupt (NMI)
 - interrupt request (INTR)
- When NMI input is activated, a type 2 interrupt occurs
 - because NMI is internally decoded
- The INTR input must be externally decoded to select a vector.

- Any interrupt vector can be chosen for the INTR pin, but we usually use an interrupt type number between 20H and FFH.
- Intel has reserved interrupts 00H - 1FH for internal and future expansion.
- INTA is also an interrupt pin on the processor.
 - it is an output used in response to INTR input to apply a vector type number to the data bus connections D_7-D_0
- Figure 12–5 shows the three user interrupt connections on the microprocessor.

Figure 12-5 The interrupt pins on all versions of the Intel microprocessor.



Interrupts on a Personal Computer



Interrupts on Emulator

- There are many useful software interrupts that can be used with emulator.
- **The instruction `INT value`**, where **value** can be a number between 0 to 255 (or 0 to 0FFh) is used.
- List of supported interrupts and their descriptions are available within the emulator documentation.

Example: INT 10h

- **INT 10h** sub-function **0Eh** is used to print a single character to the screen.
- It takes the character to be printed from the **AL** register.
- It doesn't change the value of the **AH** register.

```

ORG    100h    ; instruct compiler to make simple single segment .com file.

; The sub-function that we are using
; does not modify the AH register on
; return, so we may set it only once.

MOV    AH, 0Eh    ; select sub-function.

; INT 10h / 0Eh sub-function
; receives an ASCII code of the
; character that will be printed
; in AL register.

MOV    AL, 'H'    ; ASCII code: 72
INT    10h        ; print it!

MOV    AL, 'e'    ; ASCII code: 101
INT    10h        ; print it!

MOV    AL, 'l'    ; ASCII code: 108
INT    10h        ; print it!

MOV    AL, 'l'    ; ASCII code: 108
INT    10h        ; print it!

MOV    AL, 'o'    ; ASCII code: 111
INT    10h        ; print it!

MOV    AL, '?'    ; ASCII code: 33
INT    10h        ; print it!

RET                                ; returns to operating system.

```

Set Cursor Position

- **INT 10h / AH = 2**
- Inputs:
 - DH = row
 - DL = column
 - BH = page number (0..7)
- Example:
mov dh, 10
mov dl, 20
mov bh, 0
mov ah, 2
int 10h

Get Cursor Position and Size

- **INT 10h / AH = 03h**
- Inputs:
 - BH = page number.
- Returns:
 - DH = row.
 - DL = column.
 - CH = cursor start line.
 - CL = cursor bottom line.

Read Character and Attribute

- **INT 10h / AH = 08h** - read character and attribute at cursor position.
- Input:
 - **BH** = page number.
- Returns:
 - **AH** = attribute.
 - **AL** = character.

Write Character and Attribute

- **INT 10h / AH = 09h** - write character and attribute at cursor position.
- Inputs:
 - **AL** = character to display.
 - **BH** = page number.
 - **BL** = attribute.
 - **CX** = number of times to write character.

Attributes

- Character attribute is 8 bit value, low 4 bits set fore color, high 4 bits set background color.

HEX	BIN	Color	HEX	BIN	Color
0	0000	Black	8	1000	Dark Gray
1	0001	Blue	9	1001	Light Blue
2	0010	Green	A	1010	Light Green
3	0011	Cyan	B	1011	Light Cyan
4	0100	Red	C	1100	Light Red
5	0101	Magenta	D	1101	Light Magenta
6	0110	Brown	E	1110	Yellow
7	0111	Light Gray	F	1111	White

Write Character

- **INT 10h / AH = 0Ah** - write character only at cursor position.
- Inputs:
 - **AL** = character to display.
 - **BH** = page number.
 - **CX** = number of times to write character.



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Floating Point Numbers

Introduction

- In this chapter, representation of real numbers in processors is explained.
- The floating point (FP) representation is introduced.
- Widely preferred floating point number standard, IEEE 754, is described.
- FP arithmetic is briefly explained.

Chapter Objectives

Upon completion of this chapter, you will be able to:

- Explain the reason of why floating point numbers are needed.
- Convert between floating point and real representation of numbers.
- Perform FP arithmetic.

Floating Point

- In high level languages, you can define the type of variables such as:
 - int
 - float
 - double
 - etc.
- However, this is not directly possible in assembly language.
- Binary numbers may have different meaning based on the representation you use.

Floating Point

- The floating-point representation is one way to represent real numbers in binary form.
- Numbers are in general represented approximately to a fixed number of significant digits and scaled using an exponent.
- The base for the scaling is normally 2, 10 or 16.
- The typical number that can be represented exactly is of the form:
$$\text{significant digits} \times \text{base}^{\text{exponent}}$$
- The term floating point refers to the fact that the radix point (decimal point, or, more commonly in computers, binary point) can "float"; that is, it can be placed anywhere relative to the significant digits of the number.

Floating Point

- Though there are different floating point representations, the most commonly encountered representation is IEEE 754 Standard.
- The advantage of floating-point representation over fixed-point (and integer) representation is that it can support a much wider range of values.
- The speed of floating-point operations is an important measure of performance for computers in many application domains. It is measured in FLOPS.

Exponential Notation

- The following are equivalent representations of 1,234

$$123,400.0 \times 10^{-2}$$

$$12,340.0 \times 10^{-1}$$

$$1,234.0 \times 10^0$$

$$123.4 \times 10^1$$

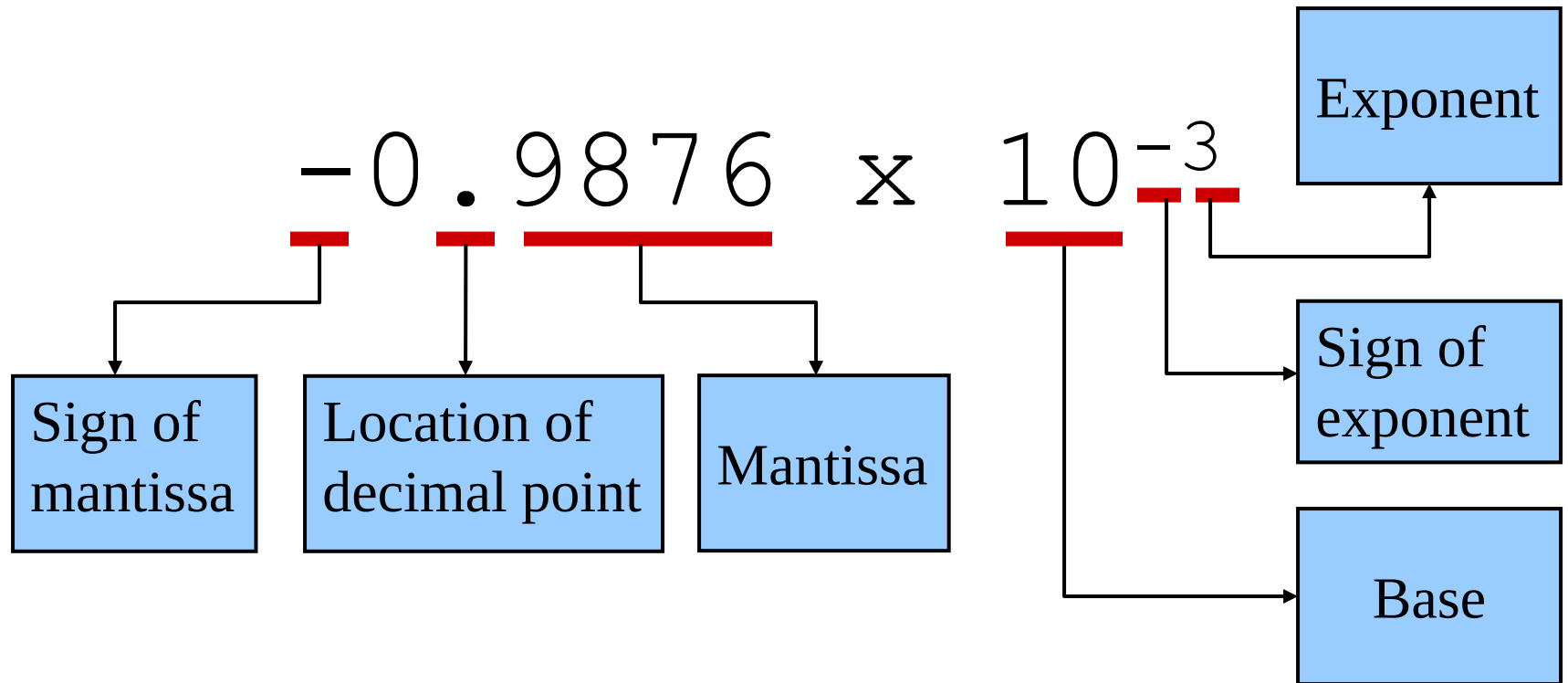
$$12.34 \times 10^2$$

$$1.234 \times 10^3$$

$$0.1234 \times 10^4$$

The representations differ in that the decimal place – the “point” -- “floats” to the left or right (with the appropriate adjustment in the exponent).

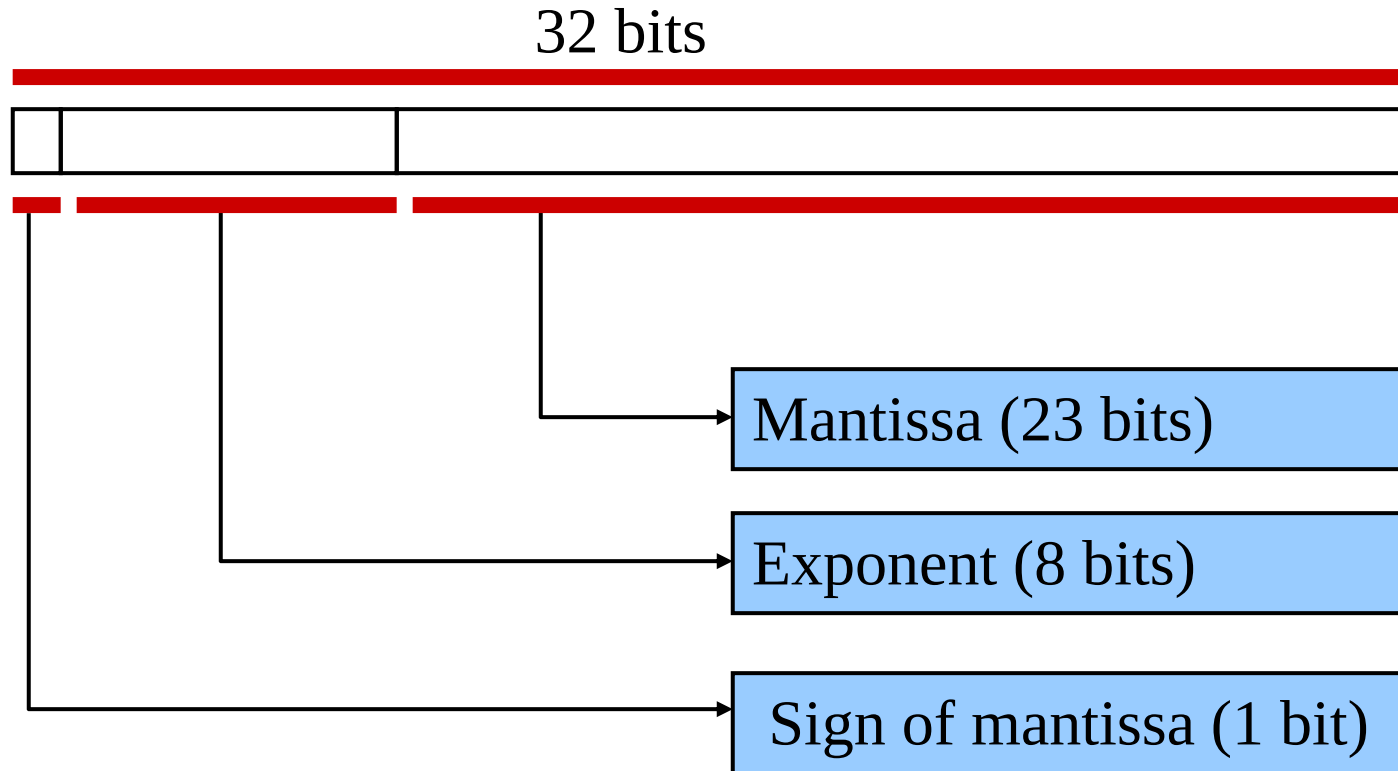
Parts of a Floating Point Number



IEEE 754 Standard

- Most common standard for representing floating point numbers
- Single precision: 32 bits, consisting of:
 - Sign bit (1 bit)
 - Exponent (8 bits)
 - Mantissa (or fraction) (23 bits)
- Double precision: 64 bits, consisting of:
 - Sign bit (1 bit)
 - Exponent (11 bits)
 - Mantissa (or fraction) (52 bits)

Single Precision Format



Normalization

- The mantissa is *normalized*
- Has an implied decimal place on left
- Has an implied “1” on left of the decimal place
- E.g.,
 - Mantissa → 1010000000000000000000000000
 - Represents... $1.101_2 = 1.625_{10}$

Excess Notation

- To include exponents, “excess” notation is used
- Single precision: excess 127
- Double precision: excess 1023
- The value of the exponent stored is larger than the actual exponent
- E.g., excess 127,
 - Exponent → 10000111
 - Represents... $135 - 127 = 8$

Denormalized Values

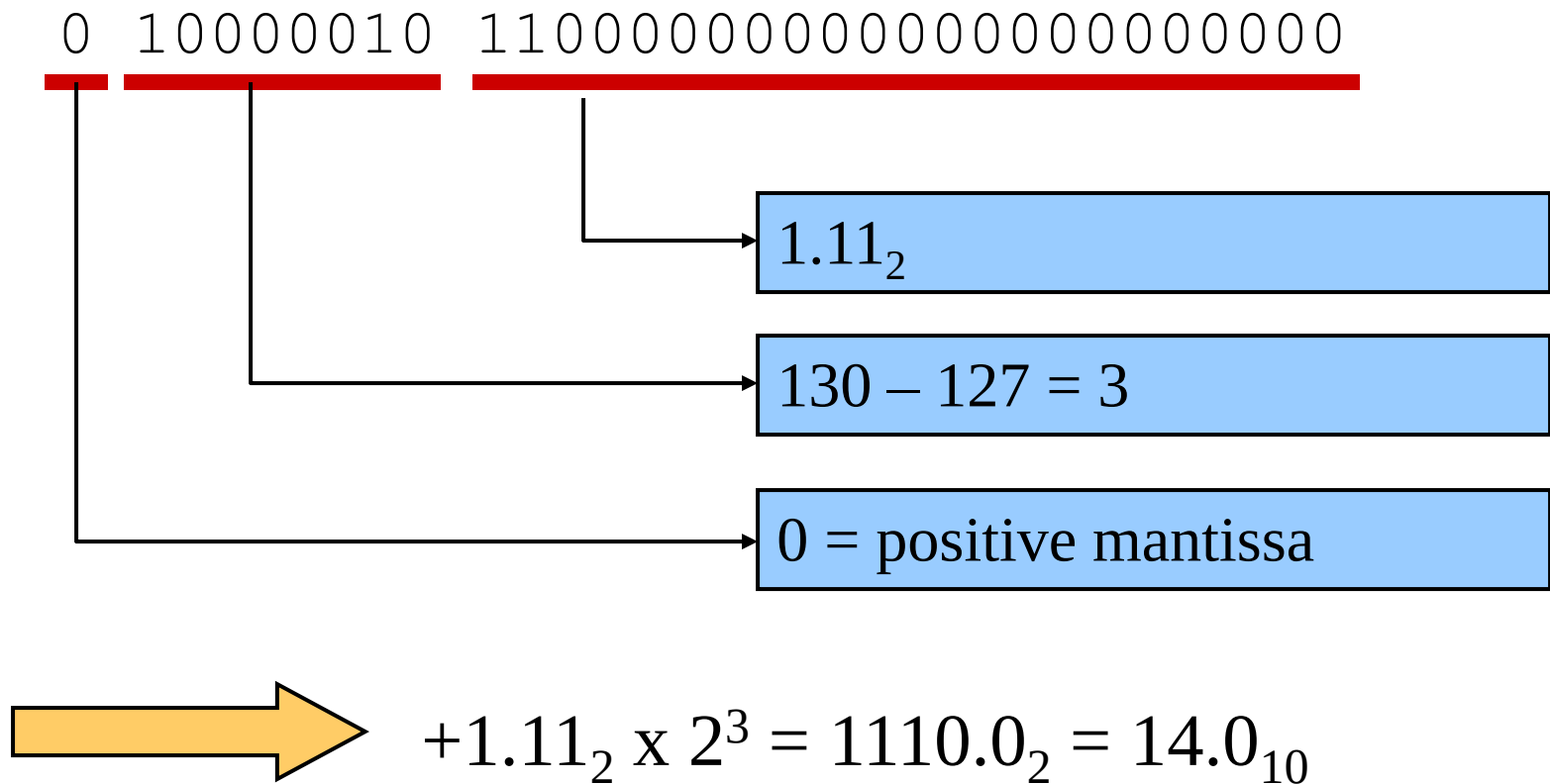
- Condition
 - $\text{exp} = 000\dots 0$
- Value
 - Exponent value $E = -\text{Bias} + 1$
 - Significand value $m = 0.\text{xxx}\dots\text{x}_2$
 - **xxx...x**: bits of `frac`
- Cases
 - $\text{exp} = 000\dots 0, \text{frac} = 000\dots 0$
 - Represents value 0
 - Note that have distinct values +0 and -0
 - $\text{exp} = 000\dots 0, \text{frac} \neq 000\dots 0$
 - Numbers very close to 0.0
 - Lose precision as get smaller
 - “Gradual underflow”

Special Values

- Condition
 - $\text{exp} = 111\dots 1$
- Cases
 - $\text{exp} = 111\dots 1, \text{frac} = 000\dots 0$
 - Represents value ∞ (infinity)
 - Operation that overflows
 - Both positive and negative
 - E.g., $1.0/0.0 = -1.0/-0.0 = +\infty$, $1.0/-0.0 = -\infty$
 - $\text{exp} = 111\dots 1, \text{frac} \neq 000\dots 0$
 - Not-a-Number (NaN)
 - Represents case when no numeric value can be determined
 - E.g., $\text{sqrt}(-1)$, $\infty - \infty$

Example

- Single precision



Hexadecimal

- It is convenient and common to represent the original floating point number in hexadecimal
- The preceding example...

0	100	0001	0	110	0000	0000	0000	0000	0000	0000
4	1	6	0	0	0	0	0	0		

Converting from Floating Point

- E.g., What decimal value is represented by the following 32-bit floating point number?

$C17B0000_{16}$

- Step 1
 - Express in binary and find S, E, and M

$$C17B0000_{16} =$$

1 10000010 11110110000000000000000000000000₂

S E M

↑
1 = negative
0 = positive

C17B0000₁₆ =

1	10000010	11110110000000000000000000000000	2
S	E	M	

- Step 2
 - Find “real” exponent, n
 - $n = E - 127$
 - = 10000010₂ – 127
 - = 130 – 127
 - = 3

$$\text{C17B0000}_{16} =$$

1 10000010 111101100000000000000000₂

S E M

- Step 3

- Put S, M, and n together to form binary result
- (Don't forget the implied "1." on the left of the mantissa.)

$$-1.1111011_2 \times 2^n =$$

$$-1.1111011_2 \times 2^3 =$$

$$-1111.1011_2$$

$$\text{C17B0000}_{16} =$$

$\underline{1}$ $\underline{10000010}$ $\underline{111101100000000000000000}_2$
 S E M

- Step 4

- Express result in decimal

$$\underline{-1111} \cdot \underline{1011}_2$$

-15

$$2^{-1} = 0.5$$
$$2^{-3} = \underline{0.125}$$
$$2^{-4} = 0.0625$$

0.6875

Answer: -15.6875

Converting to Floating Point

- E.g., Express 36.5625_{10} as a 32-bit floating point number (in hexadecimal)

- Step 1
 - Express original value in binary

$$36.5625_{10} =$$

$$100100.1001_2$$

- Step 2
 - Normalize

$$100100.1001_2 =$$

$$1.001001001_2 \times 2^5$$

- Step 3
 - Determine S, E, and M

$$\begin{array}{c} \text{+1} \cdot \text{001001001}_2 \times 2^{\text{5}} \\ \hline \text{S} \qquad \qquad \text{M} \qquad \qquad n \end{array}$$

$$\begin{aligned} E &= n + 127 \\ &= 5 + 127 \\ &= 132 \\ &= 10000100_2 \end{aligned}$$

$S = 0$ (because the value is positive)

$$\begin{array}{c} \text{+1.001001001}_2 \times 2^{\underline{5}} \\ \text{S} \quad \quad \text{M} \quad \quad \quad n \end{array}$$

- Step 4
 - Put S, E, and M together to form 32-bit binary result

$$\begin{array}{c} \underline{0} \quad \underline{10000100} \quad \underline{001001001000000000000000}_2 \\ \text{S} \quad \quad \text{E} \quad \quad \quad \text{M} \end{array}$$

$$\begin{array}{c} \text{+1.001001001}_2 \times 2^{\text{5}} \\ \hline \text{S} \quad \text{M} \quad \text{n} \end{array}$$

- Step 5
 - Express in hexadecimal

$$0 \ 10000100 \ 001001001000000000000000_2 =$$

$$0100 \ 0010 \ 0001 \ 0010 \ 0100 \ 0000 \ 0000 \ 0000_2 =$$

$$4 \quad 2 \quad 1 \quad 2 \quad 4 \quad 0 \quad 0 \quad 0_{16}$$

Answer: 42124000_{16}

Floating Point Operations

- Conceptual View
 - First compute exact result
 - Make it fit into desired precision
 - Possibly overflow if exponent too large
 - Possibly round to fit into `frac`
- Rounding Modes (illustrate with \$ rounding)

	\$1.40	\$1.60	\$1.50	\$2.50	-\$1.50
– Zero	\$1.00	\$1.00	\$1.00	\$2.00	-\$1.00
– Round down ($-\infty$)	\$1.00	\$1.00	\$1.00	\$2.00	-\$2.00
– Round up ($+\infty$)	\$2.00	\$2.00	\$2.00	\$3.00	-\$1.00
– Nearest Even (default)	\$1.00	\$2.00	\$2.00	\$2.00	-\$2.00

Note:

1. **Round down:** rounded result is close to but no greater than true result.
2. **Round up:** rounded result is close to but no less than true result.

FP Multiplication

- Operands
$$(-1)^{s1} M1 2^{E1}$$
$$(-1)^{s2} M2 2^{E2}$$
- Exact Result
$$(-1)^s M 2^E$$
 - Sign s : $s1 \wedge s2$
 - Significand M : $M1 * M2$
 - Exponent E : $E1 + E2$
- Fixing
 - If $M \geq 2$, shift M right, increment E
 - If E out of range, overflow
 - Round M to fit `frac` precision
- Implementation
 - Biggest chore is multiplying significands

FP Addition

- Operands

$$(-1)^{s1} M1 2^{E1}$$

$$(-1)^{s2} M2 2^{E2}$$

- Assume $E1 > E2$

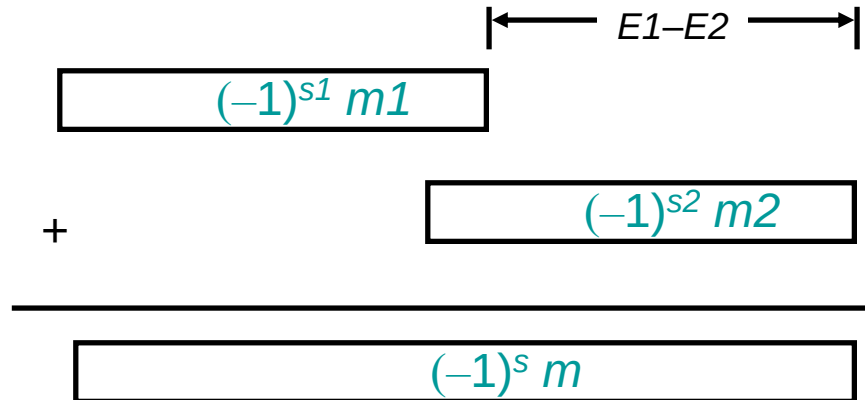
- Exact Result

$$(-1)^s M 2^E$$

- Sign s , significand M :
 - Result of signed align & add
- Exponent E : $E1$

- Fixing

- If $M \geq 2$, shift M right, increment E
- if $M < 1$, shift M left k positions, decrement E by k
- Overflow if E out of range
- Round M to fit `frac` precision



FP Operations on Intel Microprocessor

- Intel 80X87 family coprocessors support floating point numbers.
- 80X87 is an external integrated circuit designed to operate concurrently with the microprocessor.
 - 80486DX–Core2 have internal versions of 80387.
- The processor executes all normal instructions and 80X87 executes coprocessor instructions.
 - 80X87 executes 68 different instructions

SUMMARY

- Representation of real numbers in processors is explained.
- FP representation is introduced.
- IEEE 754 standard is described.
- FP arithmetic is briefly explained.

Extra References

- Lecture Notes, “Introduction to Information Technologies”, York University
- Lecture Notes, Course CS 213.



The Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium, 4, and Core2 with 64-bit Extensions Architecture, Programming, and Interfacing, Eighth Edition
Barry B. Brey

Copyright ©2009 by Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • All rights reserved.

CSE213

MICROCONTROLLER PROGRAMMING

Microprocessor Performance

Performance

- **Response time**: The time between start and finish of the task (e.g., execution time)
- **Throughput**: total amount of work done in a given time

Question

- Suppose that we replace the processor in a computer by a faster model
 - Does this improve the response time?
 - How about the throughput?

Question

- Suppose we add an additional processor to a system that uses separate processors for separate tasks.
 - Does this improve the response time?
 - Does this improve the throughput?

Computer Clock Times

- Computers run according to a clock that runs at a steady rate
- The time interval is called a *clock cycle* (eg, 10ns).
- The *clock rate* is the reciprocal of clock cycle - a frequency, how many cycles per sec (eg, 100MHz).
 - $10\text{ ns} = 1/100,000,000$ (clock cycle), same as:-
 - $1/10\text{ns} = 100,000,000 = 100\text{MHz}$ (clock rate).

Purchasing Decision

- Computer A has a 100MHz processor
- Computer B has a 300MHz processor
- So, B is faster, right?
- **NOT SO SURE!**
- Now, let's get it right.....

CPU Performance

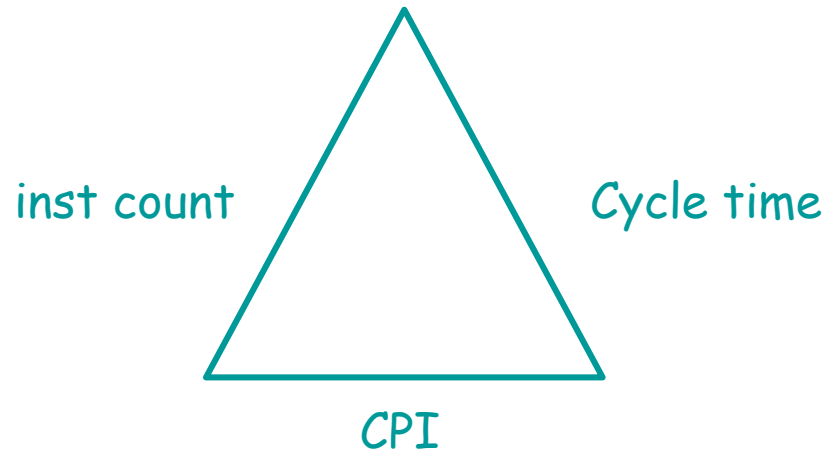
- CPU Execution time for a program

= CPU Clock Cycles for a program x Clock Cycle time

= CPU Clock Cycles for a program / Clock rate

CPU Performance

- CPU clock cycles
= instruction count x CPI (Cycles per Instruction)
- CPU time
= instruction count x CPI x Clock cycle time



Cycles Per Instruction (Throughput)

“Average Cycles per Instruction”

$$\begin{aligned}\text{CPI} &= (\text{CPU Time} * \text{Clock Rate}) / \text{Instruction Count} \\ &= \text{Cycles} / \text{Instruction Count}\end{aligned}$$

$$\text{CPU time} = \text{Cycle Time} \times \sum_{j=1}^n \text{CPI}_j \times I_j$$

$$\text{CPI} = \sum_{j=1}^n \text{CPI}_j \times F_j \quad \text{where } F_j = \frac{I_j}{\text{Instruction Count}}$$

“Instruction Frequency”

Example: Calculating CPI

Base Machine (Reg / Reg)

Op	Freq	Cycles	CPI(i)	(% Time)
ALU	50%	1	.5	(33%)
Load	20%	2	.4	(27%)
Store	10%	2	.2	(13%)
Branch	20%	2	.4	(27%)
			1.5	

Typical Mix of
instruction types
in program

Performance

(Absolute) Performance

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X}$$

Relative Performance

"X is n times faster than Y" means

$$n = \frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X}$$

Some performance measures

- MIPS
 - Million Instructions Per Second
- MFLOPS
 - Million Floating Point Operations Per Second
- Benchmarks: SPECs (Standard Performance Evaluation Corporation)
 - Average Performance over a set of example programs

Amdahl's Law

The execution time after making an improvement to the system is given by

$$\text{Exec time after improvement} = I / A + E$$

I = execution time affected by improvement

A = amount of improvement

E = execution time unaffected

Amdahl's Law

Suppose that program runs 100 seconds on a machine and multiplication instructions take 80% of the total time. How much do I have to improve the speed of multiplication if I want my program to run 5 times faster?

20 seconds = 80 seconds / n + 20 seconds

=> it is impossible!

References

- Measuring Performance, Chris Clack, B261 Systems Architecture.
- CSCE 350 Computer Architecture, Rabi Mahapatra